

# 离散数学

## 从逻辑到计算

命题逻辑 · 谓词逻辑 · 集合论 · 关系与函数  
组合数学 · 图论 · 树 · 代数结构  
布尔代数 · 递推与生成函数 · 初等数论  
算法与计算理论

魏舟

2026 年 7 月 26 日



---

## 目录

|       |                                |    |
|-------|--------------------------------|----|
| 第 1 章 | 命题逻辑                           | 1  |
| 1.1   | 命题与联结词                         | 1  |
| 1.1.1 | 逻辑联结词总览                        | 1  |
| 1.2   | 命题公式与真值表                       | 3  |
| 1.3   | 永真式、永假式、可满足式                   | 5  |
| 1.4   | 逻辑等价                           | 5  |
| 1.4.1 | 重要逻辑等价式                        | 5  |
| 1.4.2 | 逻辑等价的证明方法                      | 6  |
| 1.5   | 范式                             | 7  |
| 1.5.1 | 文字、简单合取式与简单析取式                 | 7  |
| 1.5.2 | 析取范式 DNF                       | 7  |
| 1.5.3 | 合取范式 CNF                       | 7  |
| 1.5.4 | 主范式                            | 7  |
| 1.5.5 | 求主范式的方法                        | 7  |
| 1.6   | 推理理论                           | 8  |
| 1.6.1 | 基本概念                           | 8  |
| 1.6.2 | 基本推理规则                         | 8  |
| 1.6.3 | 形式证明示例                         | 9  |
| 1.6.4 | 附加前提证明法与归谬法                    | 9  |
| 第 2 章 | 谓词逻辑                           | 13 |
| 2.1   | 谓词与个体词                         | 13 |
| 2.1.1 | 个体词                            | 13 |
| 2.1.2 | 谓词                             | 13 |
| 2.1.3 | 自然语言翻译模板                       | 14 |
| 2.2   | 量词                             | 14 |
| 2.2.1 | 全称量词 $\forall$ 与存在量词 $\exists$ | 14 |
| 2.2.2 | 辖域、自由变元与约束变元                   | 14 |
| 2.2.3 | 变元的改名规则                        | 15 |
| 2.3   | 谓词公式与解释                        | 15 |
| 2.3.1 | 项与原子公式                         | 15 |
| 2.3.2 | 合式公式                           | 16 |
| 2.3.3 | 解释与真值                          | 16 |
| 2.4   | 量词的等价与永真式                      | 16 |
| 2.4.1 | 量词的否定律                         | 16 |
| 2.4.2 | 量词的分配律                         | 16 |
| 2.4.3 | 量词作用域的扩张与收缩                    | 17 |
| 2.4.4 | 量词顺序与互换                        | 17 |

|       |                      |    |
|-------|----------------------|----|
| 2.5   | 前束范式                 | 17 |
| 2.5.1 | 求前束范式的四步法            | 17 |
| 2.6   | 谓词逻辑推理               | 18 |
| 2.6.1 | 量词的推理规则              | 18 |
| 2.6.2 | 形式证明示例               | 19 |
| 2.6.3 | 常见推理错误               | 19 |
| 第3章   | 集合论                  | 21 |
| 3.1   | 集合的基本概念              | 21 |
| 3.1.1 | 集合的表示法               | 21 |
| 3.1.2 | 特殊集合                 | 21 |
| 3.1.3 | 基数                   | 21 |
| 3.2   | 子集、真子集、幂集            | 22 |
| 3.2.1 | 子集与集合相等              | 22 |
| 3.2.2 | 幂集                   | 22 |
| 3.3   | 集合运算                 | 23 |
| 3.3.1 | 并集 $\cup$ 与交集 $\cap$ | 23 |
| 3.3.2 | 差集与补集                | 23 |
| 3.3.3 | 对称差 $\Delta$         | 23 |
| 3.3.4 | 广义并和广义交              | 24 |
| 3.4   | 集合恒等式                | 24 |
| 3.4.1 | 恒等式证明示例              | 25 |
| 3.5   | 有序对与笛卡尔积             | 25 |
| 3.5.1 | $n$ 元有序组             | 25 |
| 3.6   | 容斥原理                 | 26 |
| 3.6.1 | 两集合与三集合              | 26 |
| 3.6.2 | 一般 $n$ 个集合的容斥原理      | 27 |
| 3.6.3 | 应用示例                 | 27 |
| 第4章   | 关系与函数                | 29 |
| 4.1   | 二元关系的基本概念            | 29 |
| 4.1.1 | 关系的表示法               | 29 |
| 4.1.2 | 定义域、值域与域             | 30 |
| 4.2   | 关系的运算                | 30 |
| 4.2.1 | 逆关系                  | 30 |
| 4.2.2 | 复合关系                 | 30 |
| 4.2.3 | 关系矩阵的布尔积             | 30 |
| 4.3   | 关系的性质                | 31 |
| 4.3.1 | 自反性与反自反性             | 31 |
| 4.3.2 | 对称性与反对称性             | 31 |
| 4.3.3 | 传递性                  | 31 |
| 4.3.4 | 性质间的关系               | 32 |
| 4.4   | 闭包                   | 32 |
| 4.4.1 | Warshall 算法求传递闭包     | 32 |
| 4.5   | 等价关系与划分              | 34 |
| 4.5.1 | 等价类与商集               | 35 |
| 4.5.2 | 划分与等价关系的一一对应         | 35 |

|       |                 |    |
|-------|-----------------|----|
| 4.6   | 偏序关系            | 35 |
| 4.6.1 | 哈塞图             | 36 |
| 4.6.2 | 极大元/极小元与最大元/最小元 | 36 |
| 4.6.3 | 上界/下界与确界        | 36 |
| 4.6.4 | 全序与良序           | 37 |
| 4.7   | 函数              | 37 |
| 4.7.1 | 单射、满射、双射        | 37 |
| 4.7.2 | 函数的复合与逆函数       | 37 |
| 4.7.3 | 几类特殊函数          | 37 |
| 4.8   | 鸽巢原理            | 37 |
| 第 5 章 | 组合数学            | 41 |
| 5.1   | 计数基本原理          | 41 |
| 5.1.1 | 加法原理            | 41 |
| 5.1.2 | 乘法原理            | 41 |
| 5.1.3 | 减法原理            | 41 |
| 5.2   | 排列              | 42 |
| 5.2.1 | 线排列（不重复排列）      | 42 |
| 5.2.2 | 可重复排列           | 42 |
| 5.2.3 | 圆排列             | 42 |
| 5.2.4 | 多重集的排列          | 43 |
| 5.3   | 组合              | 43 |
| 5.3.1 | 组合数（不重复组合）      | 43 |
| 5.3.2 | 可重复组合（隔板法）      | 43 |
| 5.4   | 二项式定理与组合恒等式     | 44 |
| 5.4.1 | 二项式定理           | 44 |
| 5.4.2 | 杨辉三角（Pascal 三角） | 44 |
| 5.4.3 | 组合恒等式           | 45 |
| 5.5   | 鸽巢原理            | 45 |
| 5.5.1 | 基本形式            | 45 |
| 5.5.2 | 加强形式            | 46 |
| 5.5.3 | Ramsey 理论入门     | 46 |
| 5.6   | 容斥原理            | 46 |
| 5.6.1 | 基本形式            | 46 |
| 5.6.2 | 一般形式            | 47 |
| 5.6.3 | 错位排列            | 47 |
| 5.7   | 递推关系入门          | 48 |
| 5.7.1 | 递推关系的基本概念       | 48 |
| 5.7.2 | 一阶线性递推          | 48 |
| 5.8   | Catalan 数       | 48 |
| 5.9   | 排列组合的编程实现       | 49 |
| 5.10  | 本章小结            | 51 |
| 第 6 章 | 图论基础            | 53 |
| 6.1   | 图的基本概念          | 53 |
| 6.1.1 | 无向图与有向图的定义      | 53 |
| 6.1.2 | 邻接、关联与度         | 53 |
| 6.1.3 | 握手定理            | 53 |

|       |               |    |
|-------|---------------|----|
| 6.2   | 图的分类          | 54 |
| 6.2.1 | 简单图、完全图与二部图   | 54 |
| 6.2.2 | 正则图与补图        | 54 |
| 6.2.3 | 子图类型          | 54 |
| 6.3   | 图的表示          | 55 |
| 6.3.1 | 邻接矩阵          | 55 |
| 6.3.2 | 邻接表与关联矩阵      | 55 |
| 6.4   | 路径与连通性        | 55 |
| 6.4.1 | 通路与回路         | 55 |
| 6.4.2 | 连通性           | 55 |
| 6.4.3 | 点连通度与边连通度     | 56 |
| 6.5   | 欧拉图           | 56 |
| 6.5.1 | Fleury 算法     | 57 |
| 6.6   | 哈密顿图          | 57 |
| 6.7   | 平面图           | 58 |
| 6.7.1 | 欧拉公式          | 58 |
| 6.7.2 | Kuratowski 定理 | 58 |
| 6.8   | 图的着色          | 58 |
| 6.8.1 | 顶点着色与色数       | 58 |
| 6.8.2 | 四色定理          | 59 |
| 6.8.3 | 贪心着色算法        | 59 |
| 6.9   | 图的编程实现        | 60 |
| 6.10  | 本章小结          | 63 |
| 第 7 章 | 树             | 65 |
| 7.1   | 树的基本概念        | 65 |
| 7.1.1 | 树的定义          | 65 |
| 7.1.2 | 树的六个等价条件      | 65 |
| 7.1.3 | 树叶与森林         | 66 |
| 7.2   | 生成树           | 67 |
| 7.2.1 | 定义与存在性        | 67 |
| 7.2.2 | 最小生成树 (MST)   | 67 |
| 7.2.3 | Cayley 定理     | 68 |
| 7.3   | 有根树与二叉树       | 68 |
| 7.3.1 | 有根树           | 68 |
| 7.3.2 | $m$ 叉树        | 69 |
| 7.3.3 | 二叉树           | 69 |
| 7.3.4 | 二叉树的遍历        | 69 |
| 7.4   | 哈夫曼树与哈夫曼编码    | 70 |
| 7.4.1 | 哈夫曼树 (最优二叉树)  | 70 |
| 7.4.2 | 哈夫曼编码         | 71 |
| 7.5   | 编程实现          | 72 |
| 7.6   | 本章小结          | 75 |
| 第 8 章 | 代数结构          | 77 |
| 8.1   | 代数系统的基本概念     | 77 |
| 8.1.1 | 二元运算          | 77 |
| 8.1.2 | 特殊元素：幺元、零元与逆元 | 77 |

|       |              |    |
|-------|--------------|----|
| 8.1.3 | 代数结构层次       | 78 |
| 8.2   | 半群与么半群       | 78 |
| 8.2.1 | 半群           | 78 |
| 8.2.2 | 么半群          | 78 |
| 8.3   | 群            | 79 |
| 8.3.1 | 群的定义         | 79 |
| 8.3.2 | 群的基本性质       | 79 |
| 8.3.3 | 子群与判定定理      | 80 |
| 8.3.4 | 循环群          | 80 |
| 8.3.5 | 陪集与拉格朗日定理    | 81 |
| 8.3.6 | 正规子群与商群      | 81 |
| 8.3.7 | 群同态与同构定理     | 82 |
| 8.4   | 置换群          | 82 |
| 8.4.1 | 置换的表示        | 82 |
| 8.4.2 | 对称群与交错群      | 83 |
| 8.5   | 环与域          | 83 |
| 8.5.1 | 环的定义         | 83 |
| 8.5.2 | 环的类型         | 84 |
| 8.5.3 | 子环与理想        | 84 |
| 8.5.4 | 域            | 84 |
| 8.5.5 | 有限域          | 85 |
| 8.6   | 格            | 85 |
| 8.6.1 | 格作为偏序集       | 85 |
| 8.6.2 | 格作为代数结构      | 85 |
| 8.6.3 | 特殊格          | 86 |
| 8.7   | 编程实现         | 87 |
| 8.8   | 本章小结         | 89 |
| 第 9 章 | 布尔代数         | 91 |
| 9.1   | 布尔代数的定义      | 91 |
| 9.1.1 | 公理化定义        | 91 |
| 9.1.2 | 亨廷顿公理体系      | 92 |
| 9.1.3 | 作为有补分配格的布尔代数 | 92 |
| 9.1.4 | 重要例子         | 92 |
| 9.2   | 布尔代数的基本性质    | 93 |
| 9.2.1 | 幂等律          | 93 |
| 9.2.2 | 零律           | 93 |
| 9.2.3 | 吸收律          | 93 |
| 9.2.4 | 补元的唯一性       | 94 |
| 9.2.5 | 德摩根律         | 94 |
| 9.2.6 | 双重否定律        | 94 |
| 9.2.7 | 性质总表         | 94 |
| 9.3   | 布尔函数         | 95 |
| 9.3.1 | 布尔函数的定义      | 95 |
| 9.3.2 | 极小项与极大项      | 95 |
| 9.4   | 布尔表达式的化简     | 96 |
| 9.4.1 | 代数化简法        | 96 |

|        |                     |     |
|--------|---------------------|-----|
| 9.4.2  | 卡诺图                 | 96  |
| 9.4.3  | Quine-McCluskey 算法  | 97  |
| 9.5    | 逻辑门电路               | 98  |
| 9.5.1  | 基本逻辑门               | 98  |
| 9.5.2  | 真值表                 | 98  |
| 9.5.3  | 功能完备性               | 98  |
| 9.5.4  | 加法器电路               | 99  |
| 9.6    | 编程实现                | 100 |
| 第 10 章 | 递推关系与生成函数           | 105 |
| 10.1   | 递推关系的基本概念           | 105 |
| 10.2   | 线性递推关系的求解           | 105 |
| 10.2.1 | 一阶线性递推              | 105 |
| 10.2.2 | 齐次线性递推的特征方程法        | 106 |
| 10.2.3 | 非齐次线性递推             | 107 |
| 10.2.4 | Fibonacci 数列的完整求解   | 107 |
| 10.3   | 分治递推与 Master 定理     | 108 |
| 10.3.1 | 分治算法的递推形式           | 108 |
| 10.3.2 | Master 定理           | 108 |
| 10.4   | 生成函数 (母函数)          | 109 |
| 10.4.1 | 普通生成函数的定义           | 109 |
| 10.4.2 | 常见序列的生成函数           | 109 |
| 10.4.3 | 生成函数的运算             | 109 |
| 10.4.4 | 用生成函数解 Fibonacci 递推 | 109 |
| 10.4.5 | 指数生成函数              | 110 |
| 10.5   | 生成函数与组合计数           | 110 |
| 10.5.1 | 多重集的组合数             | 110 |
| 10.5.2 | 整数拆分的生成函数           | 111 |
| 10.6   | 编程实现                | 112 |
| 第 11 章 | 初等数论                | 115 |
| 11.1   | 整除理论                | 115 |
| 11.1.1 | 整除的定义与基本性质          | 115 |
| 11.1.2 | 带余除法                | 115 |
| 11.1.3 | 最大公因数与辗转相除法         | 116 |
| 11.1.4 | 最小公倍数               | 116 |
| 11.1.5 | 扩展欧几里得算法与 Bézout 定理 | 117 |
| 11.2   | 素数                  | 117 |
| 11.2.1 | 素数与算术基本定理           | 117 |
| 11.2.2 | 素数有无穷多个             | 118 |
| 11.2.3 | 素数判定与筛法             | 118 |
| 11.2.4 | 素数定理                | 118 |
| 11.3   | 同余与模算术              | 119 |
| 11.3.1 | 同余的定义               | 119 |
| 11.3.2 | 模算术基本性质             | 119 |
| 11.3.3 | 线性同余方程              | 119 |
| 11.3.4 | 模逆元与费马小定理           | 119 |
| 11.4   | 中国剩余定理              | 120 |



|        |                          |     |
|--------|--------------------------|-----|
| 11.5   | 欧拉定理与 RSA 密码体制           | 121 |
| 11.5.1 | 欧拉函数与欧拉定理                | 121 |
| 11.5.2 | RSA 密码体制                 | 121 |
| 11.5.3 | RSA 的安全性                 | 122 |
| 11.6   | 二次剩余                     | 122 |
| 11.7   | 编程实现                     | 123 |
| 第 12 章 | 算法与计算理论初步                | 129 |
| 12.1   | 算法的基本概念                  | 129 |
| 12.1.1 | 算法的定义与特性                 | 129 |
| 12.1.2 | 算法的表示                    | 129 |
| 12.2   | 算法复杂度分析                  | 130 |
| 12.2.1 | 渐进记号                     | 130 |
| 12.2.2 | 常见复杂度排序                  | 130 |
| 12.3   | 经典算法设计策略                 | 132 |
| 12.3.1 | 贪心算法                     | 132 |
| 12.3.2 | 分治算法                     | 132 |
| 12.3.3 | 动态规划                     | 133 |
| 12.4   | 计算理论入门                   | 134 |
| 12.4.1 | 形式语言与 Chomsky 谱系         | 134 |
| 12.4.2 | 有限自动机                    | 135 |
| 12.4.3 | 非确定型有限自动机                | 135 |
| 12.4.4 | 正则表达式                    | 135 |
| 12.4.5 | 图灵机简介                    | 136 |
| 12.5   | P 与 NP 问题简介              | 137 |
| 12.5.1 | P 类与 NP 类                | 137 |
| 12.5.2 | NP 完全性                   | 137 |
| 12.5.3 | Cook-Levin 定理与 $P=NP$ 问题 | 137 |
| 12.6   | 编程实现                     | 138 |

## 命题逻辑

命题逻辑是数理逻辑的基础分支，研究由命题和逻辑联结词构成的复合命题的真值关系与推理规则。本章从命题的基本概念出发，系统介绍联结词、真值表、逻辑等价、范式及推理理论。

§1.1  
命题与联结词

## 命题

**命题**是具有唯一确定真值的陈述句。一个命题要么为真（记作 T 或 1），要么为假（记作 F 或 0），二者必居其一。

命题的三要素：

- 必须是**陈述句**——疑问句、祈使句、感叹句不是命题。
- 必须有唯一确定的**真值**。
- 真值非真即假（排中律）。

| 语句          | 是否为命题   | 说明          |
|-------------|---------|-------------|
| $2 + 2 = 4$ | 是，真值为 T | 数学陈述句       |
| 北京是中国的首都    | 是，真值为 T | 事实陈述        |
| $1 + 1 = 3$ | 是，真值为 F | 假命题仍是命题     |
| 请你开门        | 否       | 祈使句         |
| $x > 5$     | 否       | 含自由变元，真值不确定 |

## 简单命题与复合命题

**简单命题**（原子命题）不能再分解，用小写字母  $p, q, r, \dots$  表示。**复合命题**由简单命题通过逻辑联结词组合而成。

## 1.1.1 逻辑联结词总览

逻辑联结词是将简单命题组合为复合命题的运算符号。以下列出全部基本联结词及其真值表。

否定  $\neg$

$\neg p$ （读作“非  $p$ ”）的真值与  $p$  相反。

| $p$ | $\neg p$ |
|-----|----------|
| T   | F        |
| F   | T        |

合取  $\wedge$

$p \wedge q$ （读作“ $p$  且  $q$ ”）当且仅当  $p, q$  同时为真时为真。

| $p$ | $q$ | $p \wedge q$ |
|-----|-----|--------------|
| T   | T   | T            |
| T   | F   | F            |
| F   | T   | F            |
| F   | F   | F            |

析取  $\vee$

$p \vee q$ （读作“ $p$  或  $q$ ”）当且仅当  $p, q$  至少有一个为真时为真。注意  $\vee$  是**可兼或**。

| $p$ | $q$ | $p \vee q$ |
|-----|-----|------------|
| T   | T   | T          |
| T   | F   | T          |
| F   | T   | T          |
| F   | F   | F          |

异或  $\oplus$ 、与非  $\uparrow$ 、或非  $\downarrow$

异或  $p \oplus q$  当  $p, q$  真值不同时为真；与非  $\uparrow$  即  $\neg(p \wedge q)$ ；或非  $\downarrow$  即  $\neg(p \vee q)$ 。 $\{\uparrow\}$  和  $\{\downarrow\}$  各自都是**功能完备集**——仅用一个联结词即可表达所有逻辑运算。

| $p$ | $q$ | $p \oplus q$ | $p \uparrow q$ | $p \downarrow q$ |
|-----|-----|--------------|----------------|------------------|
| T   | T   | F            | F              | F                |
| T   | F   | T            | T              | F                |
| F   | T   | T            | T              | F                |
| F   | F   | F            | T              | T                |

蕴涵  $\rightarrow$

$p \rightarrow q$ （“如果  $p$  则  $q$ ”）当且仅当  $p$  真而  $q$  假时为假。 $p$  称为**前件**， $q$  称为**后件**。

| $p$ | $q$ | $p \rightarrow q$ |
|-----|-----|-------------------|
| T   | T   | T                 |
| T   | F   | F                 |
| F   | T   | T                 |
| F   | F   | T                 |

关键理解： $p$  为假时  $p \rightarrow q$  恒为真（**虚真**）； $p \rightarrow q \equiv \neg p \vee q$ ；蕴涵不要求因果关系。  
相关蕴涵形式：

- 逆命题:  $q \rightarrow p$
- 否命题:  $\neg p \rightarrow \neg q$
- 逆否命题:  $\neg q \rightarrow \neg p$  (与原命题等价)

等价  $\leftrightarrow$

$p \leftrightarrow q$  (“ $p$  当且仅当  $q$ ”) 当  $p, q$  真值相同时为真。

| $p$ | $q$ | $p \leftrightarrow q$ |
|-----|-----|-----------------------|
| T   | T   | T                     |
| T   | F   | F                     |
| F   | T   | F                     |
| F   | F   | T                     |

联结词优先级

为减少括号, 约定优先级 (从高到低):

$$\neg > \wedge > \vee > \rightarrow > \leftrightarrow$$

## §1.2

### 命题公式与真值表

#### 合式公式

命题逻辑中的合式公式 (WFF) 递归定义如下:

1. (基底) 单个命题变元 ( $p, q, r, \dots$ ) 和真值常量 T, F 是合式公式。
2. (递归步) 若  $A$  是合式公式, 则  $(\neg A)$  是合式公式。
3. 若  $A$  和  $B$  是合式公式, 则  $(A \wedge B)$ 、 $(A \vee B)$ 、 $(A \rightarrow B)$ 、 $(A \leftrightarrow B)$ 、 $(A \oplus B)$  是合式公式。
4. 只有有限次使用上述规则得到的符号串才是合式公式。

**赋值**是对公式中每个命题变元指定真值的函数。含  $n$  个命题变元的公式共有  $2^n$  种不同的赋值。使公式  $A$  为真的赋值称为  $A$  的**成真赋值**, 使  $A$  为假的赋值称为**成假赋值**。

## Python 真值表生成器

```

1 import itertools
2
3 OPERATORS = {
4     '!': (lambda a, _: not a, 4, 1),
5     '&': (lambda a, b: a and b, 3, 2),
6     '|': (lambda a, b: a or b, 2, 2),
7     '^': (lambda a, b: a ^ b, 2, 2),
8     '>': (lambda a, b: not a or b, 1, 2),
9     '=': (lambda a, b: a == b, 0, 2),
10 }
11
12 def infix_to_postfix(expr: str) -> str:
13     output = []
14     stack = []
15     prec = {op: info[1] for op, info in OPERATORS.items()}
16     for ch in expr:
17         if ch.isalpha():
18             output.append(ch)
19         elif ch == '(':
20             stack.append(ch)
21         elif ch == ')':
22             while stack and stack[-1] != '(':
23                 output.append(stack.pop())
24             if stack:
25                 stack.pop()
26         elif ch in OPERATORS:
27             while (stack and stack[-1] != '(' and
28                    prec.get(stack[-1], -1) >= prec[ch]):
29                 output.append(stack.pop())
30             stack.append(ch)
31     while stack:
32         output.append(stack.pop())
33     return ''.join(output)
34
35 def eval_postfix(postfix: str, assign: dict) -> bool:
36     stack = []
37     for ch in postfix:
38         if ch.isalpha():
39             stack.append(assign[ch])
40         else:
41             func = OPERATORS[ch][0]
42             arity = OPERATORS[ch][2]
43             if arity == 1:
44                 a = stack.pop()
45                 stack.append(func(a, None))
46             else:
47                 b = stack.pop()
48                 a = stack.pop()
49                 stack.append(func(a, b))
50     return stack[0]
51
52 def truth_table(expr: str) -> dict:
53     variables = sorted(set(ch for ch in expr if ch.isalpha()))
54     postfix = infix_to_postfix(expr)

```

## § 1.3

## 永真式、永假式、可满足式

## 三种类型

设  $A$  为命题公式：

- **永真式**（重言式，Tautology）：所有赋值下  $A$  均为真，记作  $\models A$ 。
- **永假式**（矛盾式，Contradiction）：所有赋值下  $A$  均为假。
- **可满足式**（Satisfiable）：至少有一个赋值使  $A$  为真。

永真式一定是可满足式；永假式一定不可满足。

**判断方法：** 真值表法——列出所有赋值，逐行判断。等价变换法——将公式化为规范形式。

**【例题 1.1】** 判断  $A: (p \rightarrow q) \leftrightarrow (\neg p \vee q)$  的类型。

| $p$ | $q$ | $p \rightarrow q$ | $\neg p \vee q$ | $A$ |
|-----|-----|-------------------|-----------------|-----|
| T   | T   | T                 | T               | T   |
| T   | F   | F                 | F               | T   |
| F   | T   | T                 | T               | T   |
| F   | F   | T                 | T               | T   |

$A$  在所有赋值下为真，故  $A$  是永真式。

## § 1.4

## 逻辑等价

## 逻辑等价

设  $A, B$  是两个命题公式，若对于任意赋值， $A$  与  $B$  具有相同的真值，则称  $A$  与  $B$  **逻辑等价**，记作  $A \equiv B$ 。注意： $\equiv$  是元语言符号， $\leftrightarrow$  是对象语言符号。 $A \equiv B$  当且仅当  $A \leftrightarrow B$  是永真式。

## 1.4.1 重要逻辑等价式

## 基本等价律

双重否定律：  $\neg \neg p \equiv p$

幂等律：  $p \vee p \equiv p, \quad p \wedge p \equiv p$

交换律：  $p \vee q \equiv q \vee p, \quad p \wedge q \equiv q \wedge p$

结合律：  $(p \vee q) \vee r \equiv p \vee (q \vee r), \quad (p \wedge q) \wedge r \equiv p \wedge (q \wedge r)$

分配律：  $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r), \quad p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$

## 德摩根律与恒等律

$$\neg(p \wedge q) \equiv \neg p \vee \neg q$$

$$\neg(p \vee q) \equiv \neg p \wedge \neg q$$

$$p \vee F \equiv p, \quad p \wedge T \equiv p \quad (\text{同一律})$$

$$p \vee T \equiv T, \quad p \wedge F \equiv F \quad (\text{零律})$$

$$p \vee \neg p \equiv T \quad (\text{排中律}), \quad p \wedge \neg p \equiv F \quad (\text{矛盾律})$$

## 吸收律与蕴涵展开

$$p \vee (p \wedge q) \equiv p, \quad p \wedge (p \vee q) \equiv p$$

$$p \rightarrow q \equiv \neg p \vee q$$

$$p \leftrightarrow q \equiv (p \rightarrow q) \wedge (q \rightarrow p) \equiv (\neg p \vee q) \wedge (p \vee \neg q)$$

$$p \rightarrow q \equiv \neg q \rightarrow \neg p \quad (\text{逆否命题})$$

## 1.4.2 逻辑等价的证明方法

方法一：真值表法——列出所有可能赋值，比较真值列是否一致。

方法二：等价变换法——利用已知等价式，逐步将一个公式变换为另一个公式。

【例题 1.2】证明输出律： $(p \wedge q) \rightarrow r \equiv p \rightarrow (q \rightarrow r)$ 。

$$\begin{aligned} (p \wedge q) \rightarrow r &\equiv \neg(p \wedge q) \vee r \quad (\text{蕴涵展开}) \\ &\equiv (\neg p \vee \neg q) \vee r \quad (\text{德摩根律}) \\ &\equiv \neg p \vee (\neg q \vee r) \quad (\text{结合律}) \\ &\equiv \neg p \vee (q \rightarrow r) \quad (\text{蕴涵展开}) \\ &\equiv p \rightarrow (q \rightarrow r) \quad (\text{蕴涵展开}) \end{aligned}$$

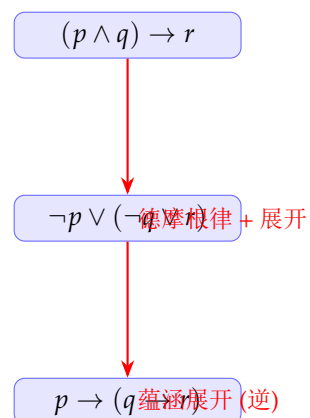


图 1.1: 逻辑等价变换示意——利用已知等价式逐步推导

§ 1.5  
范式

## 1.5.1 文字、简单合取式与简单析取式

文字：命题变元  $p$  或其否定  $\neg p$ 。

简单合取式：若干文字的合取，如  $p \wedge \neg q \wedge r$ 。

简单析取式：若干文字的析取，如  $p \vee \neg q \vee r$ 。

## 1.5.2 析取范式 DNF

## 析取范式

一个命题公式称为析取范式 (DNF)，若它是由一个或多个简单合取式通过析取联结而成的公式：

$$(L_{11} \wedge \cdots \wedge L_{1k_1}) \vee (L_{21} \wedge \cdots \wedge L_{2k_2}) \vee \cdots \vee (L_{m1} \wedge \cdots \wedge L_{mk_m})$$

求 DNF 的步骤：

1. 消去  $\rightarrow$  和  $\leftrightarrow$ ：  $p \rightarrow q \equiv \neg p \vee q$ ；  $p \leftrightarrow q \equiv (\neg p \vee q) \wedge (p \vee \neg q)$
2. 将  $\neg$  移到命题变元前（德摩根律 + 双重否定律）
3. 用分配律  $p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$  将  $\wedge$  提入  $\vee$  中

## 1.5.3 合取范式 CNF

## 合取范式

一个命题公式称为合取范式 (CNF)，若它是由一个或多个简单析取式通过合取联结而成的公式：

$$(L_{11} \vee \cdots \vee L_{1k_1}) \wedge (L_{21} \vee \cdots \vee L_{2k_2}) \wedge \cdots \wedge (L_{m1} \vee \cdots \vee L_{mk_m})$$

求 CNF 的步骤与 DNF 类似，第三步改用分配律  $p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$ 。

## 1.5.4 主范式

若公式含有命题变元  $p_1, p_2, \dots, p_n$ ：

- 极小项  $m_i$ ：每个命题变元或其否定恰好出现一次的简单合取式。编码：变元取 1 对应本身，0 对应否定。例如  $p \wedge \neg q \wedge r \rightarrow$  编码  $(1, 0, 1)_2 = 5 \rightarrow m_5$ 。
- 极大项  $M_i$ ：每个命题变元或其否定恰好出现一次的简单析取式。编码：变元取 0 对应本身，1 对应否定。

主析取范式：由不同极小项析取而成  $\rightarrow \Sigma(i_1, i_2, \dots, i_k)$ 。

主合取范式：由不同极大项合取而成  $\rightarrow \Pi(j_1, j_2, \dots, j_l)$ 。

关系：对含  $n$  个变元的公式，主析取范式中含  $k$  个极小项，则主合取范式含剩余的  $2^n - k$  个极大项。

## 1.5.5 求主范式的方法

方法一：真值表法。列出真值表，成真赋值对应极小项（析取），成假赋值对应极大项（合取）。

方法二：等价变换法。先求一般范式，再对缺少某变元的简单合取式用  $p \vee \neg p \equiv \top$  扩充。



【例题 1.3】求  $(p \rightarrow q) \wedge \neg r$  的主范式。

先求真值表，得成真赋值为  $\{(0,0,0), (0,0,1), (0,1,0), (1,1,0)\}$ （其中  $(p,q,r)$  编码），故：

- 主析取范式： $m_0 \vee m_1 \vee m_2 \vee m_6 = \Sigma(0,1,2,6)$
- 主合取范式： $\Pi(3,4,5,7)$

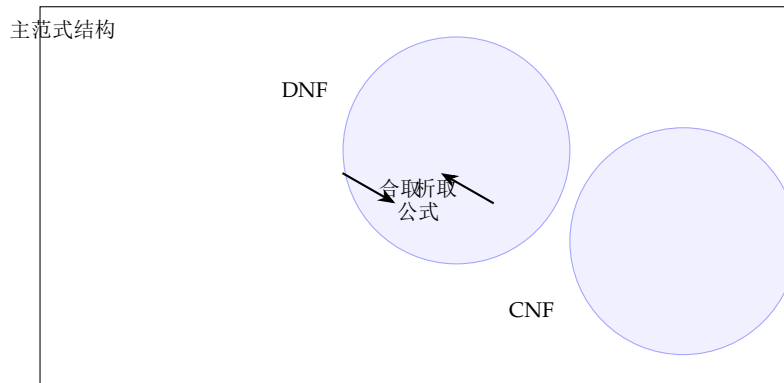


图 1.2: 范式关系示意——DNF 与 CNF 从不同方向刻画同一公式

## § 1.6 推理理论

### 1.6.1 基本概念

#### 有效推理

设  $A_1, A_2, \dots, A_n$  是前提， $C$  是结论。若对于任意赋值，当所有前提为真时结论也为真，则称此推理有效，记作：

$$A_1, A_2, \dots, A_n \models C$$

等价地， $A_1 \wedge A_2 \wedge \dots \wedge A_n \rightarrow C$  是永真式。

### 1.6.2 基本推理规则

| 规则名称  | 推理形式                                                       | 别名                 |
|-------|------------------------------------------------------------|--------------------|
| 假言推理  | $p, p \rightarrow q \models q$                             | Modus Ponens (MP)  |
| 拒取式   | $\neg q, p \rightarrow q \models \neg p$                   | Modus Tollens (MT) |
| 假言三段论 | $p \rightarrow q, q \rightarrow r \models p \rightarrow r$ | HS                 |
| 析取三段论 | $p \vee q, \neg p \models q$                               | DS                 |
| 附加律   | $p \models p \vee q$                                       | Addition           |
| 化简律   | $p \wedge q \models p$                                     | Simplification     |
| 合取律   | $p, q \models p \wedge q$                                  | Conjunction        |
| 消解律   | $p \vee q, \neg p \vee r \models q \vee r$                 | Resolution         |

## 关键推理规则公式

MP:  $p \wedge (p \rightarrow q) \rightarrow q$  是永真式MT:  $\neg q \wedge (p \rightarrow q) \rightarrow \neg p$  是永真式HS:  $(p \rightarrow q) \wedge (q \rightarrow r) \rightarrow (p \rightarrow r)$  是永真式

## 1.6.3 形式证明示例

**形式证明：**从前提出发，每一步使用推理规则或逻辑等价式推出中间结论，直到推出所要求的结论。

**【例题 1.4】** 证明  $p \rightarrow q, q \rightarrow r, p \models r$ 。

1.  $p$  (前提)
2.  $p \rightarrow q$  (前提)
3.  $q$  (1, 2, MP)
4.  $q \rightarrow r$  (前提)
5.  $r$  (3, 4, MP)

**【例题 1.5】** 证明拒取式:  $\neg q, p \rightarrow q \models \neg p$  (用等价变换法)。

1.  $\neg q$  (前提)
2.  $p \rightarrow q$  (前提)
3.  $\neg p \vee q$  (2, 蕴涵展开)
4.  $\neg p$  (1, 3, 析取三段论)

## 1.6.4 附加前提证明法与归谬法

**CP 规则：**要证  $A_1, \dots, A_n \models B \rightarrow C$ ，等价于证  $A_1, \dots, A_n, B \models C$ 。

**归谬法（反证法）：**要证  $A_1, \dots, A_n \models C$ ，证  $A_1, \dots, A_n, \neg C \models F$ （矛盾）。

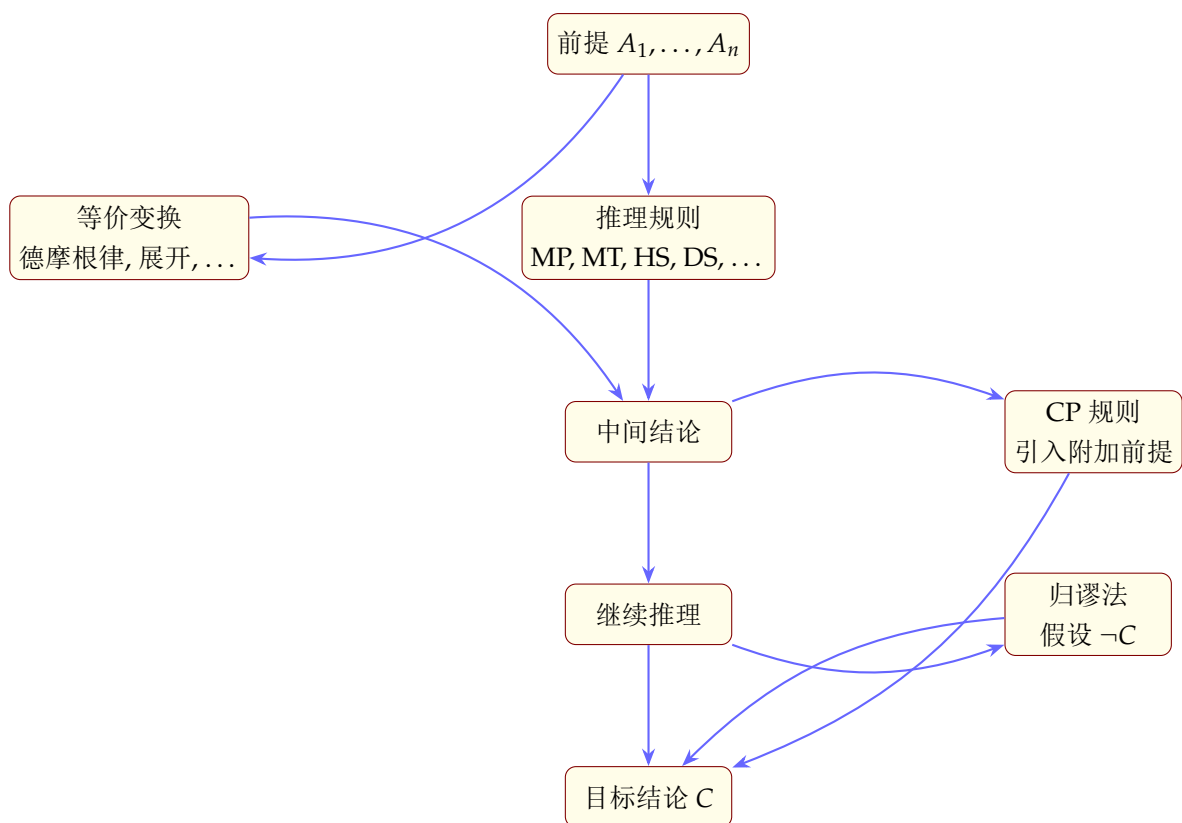


图 1.3: 逻辑推理流程图——多种策略配合使用

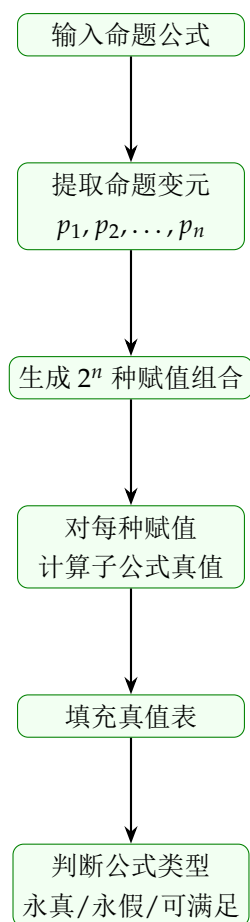


图 1.4: 真值表生成流程图



## 谓词逻辑

谓词逻辑又称一阶逻辑，是对命题逻辑的扩展。命题逻辑将原子命题视为不可分割的整体，忽略其内部结构；谓词逻辑进一步分析命题的内部成分——个体对象和谓词关系，并引入全称量词 $\forall$ 和存在量词 $\exists$ ，使推理表达能力大幅增强。

§2.1  
谓词与个体词

## 2.1.1 个体词

个体词是表示所讨论对象的符号，分为两类：

- 个体常元：表示特定对象的符号，用小写字母  $a, b, c, \dots$  表示。
- 个体变元：表示泛指某类对象的符号，用  $x, y, z, \dots$  表示。

论域 (Domain of Discourse)：个体变元的取值范围，记作  $D$ 。论域可以是有限集或无限集。

## 2.1.2 谓词

谓词是表示个体性质或个体间关系的符号。谓词本身不是命题，但填入个体词后成为命题。

- 一元谓词  $P(x)$ ：表示个体  $x$  的性质。
- 二元谓词  $R(x, y)$ ：表示个体  $x$  与  $y$  之间的关系。
- $n$  元谓词  $P(x_1, \dots, x_n)$ ：表示  $n$  个个体之间的关系。

| 谓词符号         | 含义                 | 元数 |
|--------------|--------------------|----|
| $H(x)$       | $x$ 是人             | 一元 |
| $M(x)$       | $x$ 会死             | 一元 |
| $L(x, y)$    | $x$ 爱 $y$          | 二元 |
| $G(x, y, z)$ | $x$ 在 $y$ 和 $z$ 之间 | 三元 |

命题是 **0 元谓词** (不含量词和自由变元)。

## 2.1.3 自然语言翻译模板

| 自然语言          | 谓词公式                                    |
|---------------|-----------------------------------------|
| 所有 $P$ 是 $Q$  | $\forall x(P(x) \rightarrow Q(x))$      |
| 有些 $P$ 是 $Q$  | $\exists x(P(x) \wedge Q(x))$           |
| 没有 $P$ 是 $Q$  | $\forall x(P(x) \rightarrow \neg Q(x))$ |
| 有些 $P$ 不是 $Q$ | $\exists x(P(x) \wedge \neg Q(x))$      |

**关键规律：**全称量词  $\forall$  后通常跟蕴涵  $\rightarrow$ ，存在量词  $\exists$  后通常跟合取  $\wedge$ 。

**【例题 2.1】** 将“每个人都爱某个人”翻译为谓词公式。

设论域为全体人类， $L(x, y)$  表示“ $x$  爱  $y$ ”：

$$\forall x \exists y L(x, y)$$

注意： $\forall x \exists y L(x, y)$ （每人都爱某个可能不同的人）与  $\exists y \forall x L(x, y)$ （存在一个被所有人爱的人）意义不同！

§2.2  
量词2.2.1 全称量词  $\forall$  与存在量词  $\exists$ 

## 全称量词

$\forall x P(x)$  读作“对任意  $x$ （在论域中）， $P(x)$  成立”。当且仅当论域中的每一个个体都满足  $P$  时， $\forall x P(x)$  为真。

在有限论域  $D = \{a_1, \dots, a_n\}$  中：

$$\forall x P(x) \equiv P(a_1) \wedge P(a_2) \wedge \dots \wedge P(a_n)$$

## 存在量词

$\exists x P(x)$  读作“存在  $x$ （在论域中），使得  $P(x)$  成立”。当且仅当论域中至少有一个个体满足  $P$  时， $\exists x P(x)$  为真。

在有限论域中：

$$\exists x P(x) \equiv P(a_1) \vee P(a_2) \vee \dots \vee P(a_n)$$

**唯一存在量词  $\exists!$ ：** $\exists! x P(x)$  表示“存在唯一的  $x$  使  $P(x)$  成立”。

$$\exists! x P(x) \equiv \exists x [P(x) \wedge \forall y (P(y) \rightarrow y = x)]$$

**量词对偶性：**

$$\neg \forall x P(x) \equiv \exists x \neg P(x), \quad \neg \exists x P(x) \equiv \forall x \neg P(x)$$

## 2.2.2 辖域、自由变元与约束变元

## 辖域

量词的作用范围。在  $\forall x P(x) \rightarrow Q(x)$  中，量词  $\forall x$  的辖域仅为  $P(x)$ ， $Q(x)$  中的  $x$  是自由变元。在  $\forall x (P(x) \rightarrow Q(x))$  中，辖域是整个  $P(x) \rightarrow Q(x)$ 。

- **约束变元**：出现在量词辖域中且与该量词使用的变元名相同的变元。
- **自由变元**：未受任何量词约束的变元。

含自由变元的公式不是命题，其真值依赖于对自由变元的具体赋值。

**【例题 2.2】** 区分公式  $\forall x(P(x) \rightarrow \exists y Q(x, y)) \vee R(z)$  中的自由与约束变元：

- $\forall x$  约束了  $P(x)$  和  $Q(x, y)$  中的  $x$
- $\exists y$  约束了  $Q(x, y)$  中的  $y$
- $z$  是自由变元

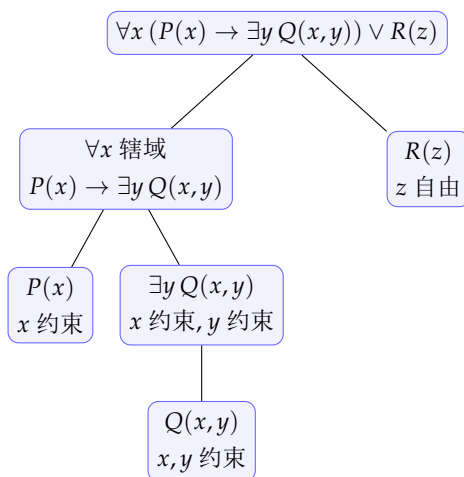


图 2.1: 量词辖域树形图——展示约束变元与自由变元的层次关系

## 2.2.3 变元的改名规则

在谓词逻辑中，约束变元的名称可以更改而不改变公式的含义，但必须遵守：

1. 只对约束变元改名，不能改自由变元。
2. 改名后的变元不能与辖域中已有的其他自由变元同名（避免变量捕获）。

例如： $\forall x P(x) \equiv \forall y P(y)$ （合法）。但  $\forall x (P(x) \wedge Q(y))$  不能改为  $\forall y (P(y) \wedge Q(y))$ （原名自由变元  $y$  被捕获）。

## §2.3 谓词公式与解释

### 2.3.1 项与原子公式

项的递归定义：

1. 个体常元是项；个体变元是项。
2. 若  $f$  是  $n$  元函数符号， $t_1, \dots, t_n$  是项，则  $f(t_1, \dots, t_n)$  是项。

**原子公式**：若  $P$  是  $n$  元谓词符号， $t_1, \dots, t_n$  是项，则  $P(t_1, \dots, t_n)$  是原子公式。



### 2.3.2 合式公式

谓词逻辑的合式公式 (WFF) 递归定义：

1. 每个原子公式是合式公式。
2. 若  $A$  是 WFF, 则  $(\neg A)$  是 WFF。
3. 若  $A, B$  是 WFF, 则  $(A \wedge B)$ 、 $(A \vee B)$ 、 $(A \rightarrow B)$ 、 $(A \leftrightarrow B)$  是 WFF。
4. 若  $A$  是 WFF,  $x$  是变元, 则  $(\forall x A)$  和  $(\exists x A)$  是 WFF。

封闭公式 (Sentence): 不含自由变元的 WFF, 具有确定的真值。

### 2.3.3 解释与真值

谓词公式的解释  $\mathcal{I}$  由以下成分构成：

1. 论域  $D$ : 一个非空集合。
2. 常元指派: 将每个个体常元  $a$  映射到  $D$  中的某个元素。
3. 函数符号指派: 将每个  $n$  元函数符号  $f$  映射到函数  $f^{\mathcal{I}} : D^n \rightarrow D$ 。
4. 谓词符号指派: 将每个  $n$  元谓词符号  $P$  映射到关系  $P^{\mathcal{I}} \subseteq D^n$ 。

封闭公式在给定解释下按 Tarski 语义递归计算真值。

**【例题 2.3】** 设论域  $D = \{1, 2, 3\}$ ,  $P(x)$ : “ $x$  是偶数” ( $P^{\mathcal{I}} = \{2\}$ ),  $Q(x)$ : “ $x > 0$ ” ( $Q^{\mathcal{I}} = \{1, 2, 3\}$ )。

- $\mathcal{I} \models \forall x Q(x)$  (所有数都  $> 0$  ✓)
- $\mathcal{I} \not\models \forall x P(x)$  (1 不是偶数)
- $\mathcal{I} \models \exists x P(x)$  (存在偶数 2 ✓)

## §2.4 量词的等价与永真式

### 2.4.1 量词的否定律

#### 量词否定律

$$\neg \forall x P(x) \equiv \exists x \neg P(x), \quad \neg \exists x P(x) \equiv \forall x \neg P(x)$$

**直观理解:**  $\forall x P(x)$  为假, 意味着并非所有  $x$  都使  $P(x)$  成立  $\rightarrow$  存在某个  $x$  使  $P(x)$  不成立  $\rightarrow \exists x \neg P(x)$  为真。

### 2.4.2 量词的分配律

| 编号 | 等价/蕴涵关系                                                                    | 说明                       |
|----|----------------------------------------------------------------------------|--------------------------|
| Q3 | $\forall x(P(x) \wedge Q(x)) \equiv \forall x P(x) \wedge \forall x Q(x)$  | $\forall$ 对 $\wedge$ 可分配 |
| Q4 | $\exists x(P(x) \vee Q(x)) \equiv \exists x P(x) \vee \exists x Q(x)$      | $\exists$ 对 $\vee$ 可分配   |
| Q5 | $\forall x P(x) \vee \forall x Q(x) \models \forall x(P(x) \vee Q(x))$     | 反向不成立                    |
| Q6 | $\exists x(P(x) \wedge Q(x)) \models \exists x P(x) \wedge \exists x Q(x)$ | 反向不成立                    |

【例题 2.4】反例 (Q5 反向不成立): 论域  $D = \mathbb{N}$ ,  $P(x)$ : “ $x$  是偶数”,  $Q(x)$ : “ $x$  是奇数”。

- $\forall x(P(x) \vee Q(x))$  为真 (每个自然数非偶即奇)
- $\forall x P(x) \vee \forall x Q(x)$  为假 (并非所有数都是偶数, 也并非所有数都是奇数)

### 2.4.3 量词作用域的扩张与收缩

若公式  $B$  不含自由变元  $x$ , 则有以下等价式:

#### 量词作用域扩张律

$$\forall x(A(x) \wedge B) \equiv \forall x A(x) \wedge B$$

$$\forall x(A(x) \vee B) \equiv \forall x A(x) \vee B$$

$$\exists x(A(x) \wedge B) \equiv \exists x A(x) \wedge B$$

$$\exists x(A(x) \vee B) \equiv \exists x A(x) \vee B$$

### 2.4.4 量词顺序与互换

- $\forall x \forall y P(x, y) \equiv \forall y \forall x P(x, y)$  (同种量词可交换)
- $\exists x \exists y P(x, y) \equiv \exists y \exists x P(x, y)$  (同种量词可交换)
- $\exists x \forall y P(x, y) \models \forall y \exists x P(x, y)$  (反向不成立!)

关键区别:  $\forall y \exists x P(x, y)$  (对每个  $y$ , 存在某个可能依赖于  $y$  的  $x$ ) 不能推出  $\exists x \forall y P(x, y)$  (存在一个统一的  $x$  对所有  $y$  成立)。

## §2.5

### 前束范式

#### 前束范式

一个谓词公式称为前束范式, 若其具有形式:

$$Q_1 x_1 Q_2 x_2 \dots Q_n x_n M$$

其中每个  $Q_i$  是  $\forall$  或  $\exists$ ,  $M$  是不含量词的公式 (称为母式), 所有量词都在公式的最前面。

### 2.5.1 求前束范式的四步法

1. 消去  $\rightarrow$  和  $\leftrightarrow$ :  $A \rightarrow B \equiv \neg A \vee B$ ;  $A \leftrightarrow B \equiv (A \rightarrow B) \wedge (B \rightarrow A)$
2. 将否定深入 (德摩根律 + 量词否定律):
  - $\neg \neg A \equiv A$
  - $\neg(A \wedge B) \equiv \neg A \vee \neg B$ ,  $\neg(A \vee B) \equiv \neg A \wedge \neg B$
  - $\neg \forall x A(x) \equiv \exists x \neg A(x)$ ,  $\neg \exists x A(x) \equiv \forall x \neg A(x)$
3. 变元改名: 若不同量词使用了相同的约束变元名, 将其中的约束变元改为新名。确保不存在某变元既是约束的又是自由的。

4. **量词前移**：利用量词作用域扩张律，将量词逐步移到公式前面。

**【例题 2.5】** 求  $\forall xP(x) \rightarrow \exists yQ(y)$  的前束范式。

$$\begin{aligned}\forall xP(x) \rightarrow \exists yQ(y) &\equiv \neg\forall xP(x) \vee \exists yQ(y) && (\text{消去 } \rightarrow) \\ &\equiv \exists x\neg P(x) \vee \exists yQ(y) && (\text{量词否定律}) \\ &\equiv \exists x\exists y(\neg P(x) \vee Q(y)) && (\text{量词前移})\end{aligned}$$

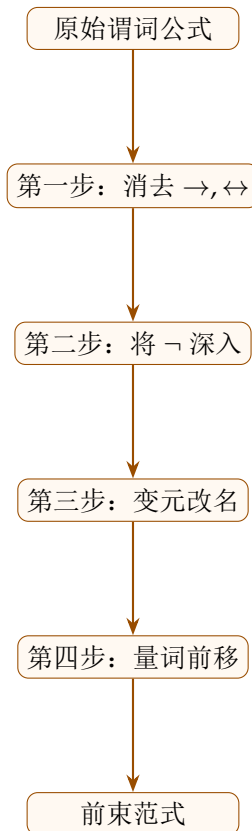


图 2.2: 前束范式转换四步法流程

## §2.6 谓词逻辑推理

### 2.6.1 量词的推理规则

| 规则 | 名称    | 形式                                          |
|----|-------|---------------------------------------------|
| UI | 全称实例化 | $\forall xP(x) \models P(a)$ ( $a$ 为任意个体常元) |
| EI | 存在实例化 | $\exists xP(x) \models P(c)$ ( $c$ 为新常元)    |
| UG | 全称泛化  | $P(a)$ 对任意 $a$ 成立 $\models \forall xP(x)$   |
| EG | 存在泛化  | $P(a) \models \exists xP(x)$                |

**关键约束：**

- EI:  $c$  必须是**新**常元——此前未在前提、已推出的结论中出现过，且不能出现在最终结论中。
- UG:  $a$  不能是已被特殊假设（如来自 EI）限制的个体。

## 2.6.2 形式证明示例

【例题 2.6】经典三段论：所有人都会死，苏格拉底是人，所以苏格拉底会死。

设  $H(x)$ ：“ $x$  是人”， $M(x)$ ：“ $x$  会死”， $s$ ：“苏格拉底”。证明  $\forall x(H(x) \rightarrow M(x)), H(s) \models M(s)$ 。

1.  $\forall x(H(x) \rightarrow M(x))$  (前提)
2.  $H(s)$  (前提)
3.  $H(s) \rightarrow M(s)$  (1, UI)
4.  $M(s)$  (2, 3, MP)

【例题 2.7】证明  $\forall x(P(x) \rightarrow Q(x)), \exists xP(x) \models \exists xQ(x)$ 。

1.  $\forall x(P(x) \rightarrow Q(x))$  (前提)
2.  $\exists xP(x)$  (前提)
3.  $P(c)$  (2, EI: 引入新常元  $c$ )
4.  $P(c) \rightarrow Q(c)$  (1, UI)
5.  $Q(c)$  (3, 4, MP)
6.  $\exists xQ(x)$  (5, EG)

【例题 2.8】证明  $\exists x\forall y L(x, y) \models \forall y\exists x L(x, y)$ 。

注意：其逆  $(\forall y\exists x L(x, y) \models \exists x\forall y L(x, y))$  不成立。直观：每人爱某个人（依赖  $y$  的  $x$ ） $\not\models$  存在一个被所有人爱的人（统一的  $x$ ）。

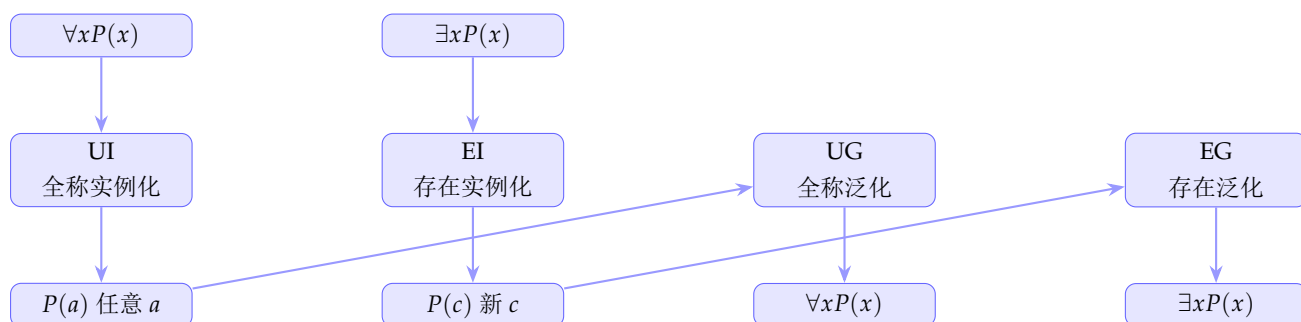


图 2.3: 谓词逻辑四大约束规则关系图——UI 与 UG 配对，EI 与 EG 配对

## 2.6.3 常见推理错误

| 错误      | 描述                                                             |
|---------|----------------------------------------------------------------|
| 逆命题错误   | 从 $P(x) \rightarrow Q(x)$ 推出 $Q(x) \rightarrow P(x)$           |
| 量词跳跃    | 从 $\forall y\exists xP(x, y)$ 错误推出 $\exists x\forall yP(x, y)$ |
| EI 使用不当 | 引入的常元在前提中已出现                                                   |
| UG 使用不当 | 对由 EI 引入的特定个体使用全称泛化                                            |

## 有限论域量词评估

```

1 from typing import Callable, TypeVar, List
2
3 T = TypeVar('T')
4
5 def forall(domain: List[T], pred: Callable[[T], bool]) -> bool:
6     """评估  $\forall x P(x)$  在有限论域上的真值"""
7     return all(pred(x) for x in domain)
8
9 def exists(domain: List[T], pred: Callable[[T], bool]) -> bool:
10    """评估  $\exists x P(x)$  在有限论域上的真值"""
11    return any(pred(x) for x in domain)
12
13 def exists_unique(domain: List[T], pred: Callable[[T], bool]) -> bool:
14    """评估  $\exists! x P(x)$ """
15    return sum(1 for x in domain if pred(x)) == 1
16
17 # 示例：论域  $D = \{1, 2, 3, 4, 5\}$ 
18 D = [1, 2, 3, 4, 5]
19 is_even = lambda x: x % 2 == 0
20 is_positive = lambda x: x > 0
21
22 print(f" $\forall x (x > 0) = \{forall(D, is\_positive)\}$ ") # True
23 print(f" $\forall x (x \text{ 是偶数}) = \{forall(D, is\_even)\}$ ") # False
24 print(f" $\exists x (x \text{ 是偶数}) = \{exists(D, is\_even)\}$ ") # True
25
26 # 验证量词否定律:  $\neg \forall x Q(x) \equiv \exists x \neg Q(x)$ 
27 lhs = not forall(D, is_even)
28 rhs = exists(D, lambda x: not is_even(x))
29 print(f" $\neg \forall x Q(x) \equiv \exists x \neg Q(x): \{lhs == rhs\}$ ") # True
30
31 # 验证 Q6 反向不成立:  $(\forall x P(x) \wedge \forall x Q(x)) \rightarrow \forall x (P(x) \wedge Q(x))$ 
32 P = lambda x: x % 2 == 0
33 Q = lambda x: x % 3 == 0
34 fwd = exists(D, lambda x: P(x) and Q(x))
35 bwd = exists(D, P) and exists(D, Q)
36 print(f"反向不成立验证:  $\{not bwd or fwd\}$ ") # False

```

集合论是现代数学的基石，几乎所有数学对象都可以用集合来定义。本章讨论朴素集合论的核心概念，包括集合的基本定义、运算、关系以及容斥原理，为后续的关系、函数和图论奠定基础。

### §3.1 集合的基本概念

#### 集合与元素

集合是由确定对象构成的整体，构成集合的对象称为该集合的**元素**。用  $a \in A$  表示  $a$  是  $A$  的元素， $a \notin A$  表示  $a$  不是  $A$  的元素。

集合的基本特征：

- 确定性**：对任意对象  $x$ ， $x \in A$  和  $x \notin A$  二者必居其一。
- 无序性**： $\{1, 2, 3\} = \{3, 1, 2\}$ 。
- 互异性**： $\{1, 1, 2, 2, 3\} = \{1, 2, 3\}$ 。

#### 3.1.1 集合的表示法

**列举法**： $A = \{1, 2, 3, 4, 5\}$ ； $\mathbb{N} = \{0, 1, 2, 3, \dots\}$ 。

**描述法**： $A = \{x \mid P(x)\}$ ，读作“满足  $P(x)$  的所有  $x$  构成的集合”。例如  $A = \{x \mid x \in \mathbb{R}, x > 2\}$ （大于 2 的所有实数）。

**文氏图**：用平面上的封闭曲线直观表示集合及其关系。用矩形表示全集  $U$ ，内部用圆表示各集合。

#### 3.1.2 特殊集合

- 空集**  $\emptyset = \{\}$ ：不含任何元素的集合。 $|\emptyset| = 0$ ，对任意集合  $A$  有  $\emptyset \subseteq A$ 。
- 单元素集**  $\{a\}$ ：仅含一个元素的集合。注意  $a \in \{a\}$  但  $a \neq \{a\}$ 。
- 全集**  $U$ ：在特定讨论范围内包含所有考虑的元素集合。

#### 3.1.3 基数

集合  $A$  的**基数**记作  $|A|$ 。有限集基数是非负整数；无限集中  $\mathbb{N}$  的基数为  $\aleph_0$ （可数无穷）， $\mathbb{R}$  的基数为  $\mathfrak{c} = 2^{\aleph_0}$ （不可数）。

## 外延公理

$$\forall A \forall B (A = B \leftrightarrow \forall x (x \in A \leftrightarrow x \in B))$$

集合由它的元素唯一确定，不依赖于描述方式。

## §3.2

## 子集、真子集、幂集

## 3.2.1 子集与集合相等

## 子集

$A \subseteq B$  当且仅当  $\forall x (x \in A \rightarrow x \in B)$ 。

子集关系的三条基本性质（偏序关系）：

1. 自反性：  $A \subseteq A$
2. 反对称性：若  $A \subseteq B$  且  $B \subseteq A$ ，则  $A = B$
3. 传递性：若  $A \subseteq B$  且  $B \subseteq C$ ，则  $A \subseteq C$

## 真子集

$A \subset B$  当且仅当  $A \subseteq B$  且  $A \neq B$ 。

判定集合相等的标准策略：证明  $A \subseteq B$  且  $B \subseteq A$ ，由外延公理得  $A = B$ 。

## 3.2.2 幂集

## 幂集

集合  $A$  的所有子集构成的集合称为  $A$  的幂集，记作  $\mathcal{P}(A)$  或  $2^A$ ：

$$\mathcal{P}(A) = \{X \mid X \subseteq A\}$$

## 幂集的基数

若  $|A| = n$ （有限集），则  $|\mathcal{P}(A)| = 2^n$ 。

**证明.** 组合论证：对  $A$  中的  $n$  个元素，每个元素在子集中有“出现”和“不出现”两种选择，因此子集总数为  $2^n$ 。□

**【例题 3.1】** •  $A = \{1, 2\}$ ,  $\mathcal{P}(A) = \{\emptyset, \{1\}, \{2\}, \{1, 2\}\}$ ,  $|\mathcal{P}(A)| = 4$

- $A = \emptyset$ ,  $\mathcal{P}(\emptyset) = \{\emptyset\}$ ,  $|\mathcal{P}(\emptyset)| = 1$

幂集的性质：

- $\emptyset \in \mathcal{P}(A)$  且  $A \in \mathcal{P}(A)$
- 若  $A \subseteq B$ ，则  $\mathcal{P}(A) \subseteq \mathcal{P}(B)$

- $\mathcal{P}(A \cap B) = \mathcal{P}(A) \cap \mathcal{P}(B)$

### §3.3 集合运算

#### 3.3.1 并集 $\cup$ 与交集 $\cap$

##### 并集

$$A \cup B = \{x \mid x \in A \vee x \in B\}$$

##### 交集

$$A \cap B = \{x \mid x \in A \wedge x \in B\}$$

若  $A \cap B = \emptyset$ , 则称  $A$  和  $B$  不相交。

并集性质：幂等律、交换律、结合律、同一律 ( $A \cup \emptyset = A$ )、零律 ( $A \cup U = U$ )。

交集性质：幂等律、交换律、结合律、同一律 ( $A \cap U = A$ )、零律 ( $A \cap \emptyset = \emptyset$ )。

#### 3.3.2 差集与补集

##### 差集与补集

$$A - B = A \setminus B = \{x \mid x \in A \wedge x \notin B\}.$$

当给定全集  $U$  时,  $A$  的补集为  $\bar{A} = U - A = \{x \in U \mid x \notin A\}$ 。

补集性质:

- $\overline{\bar{A}} = A$  (双重补)
- $\bar{U} = \emptyset, \bar{\emptyset} = U$
- $A \cup \bar{A} = U$  (排中律)
- $A \cap \bar{A} = \emptyset$  (矛盾律)
- $\overline{A \cup B} = \bar{A} \cap \bar{B}$  (德摩根律)
- $\overline{A \cap B} = \bar{A} \cup \bar{B}$  (德摩根律)

#### 3.3.3 对称差 $\triangle$

##### 对称差

$$A \triangle B = (A - B) \cup (B - A) = (A \cup B) - (A \cap B)$$

性质：交换律、结合律、 $A \triangle \emptyset = A$ 、 $A \triangle A = \emptyset$ 、 $A \triangle B = \emptyset \iff A = B$ 。



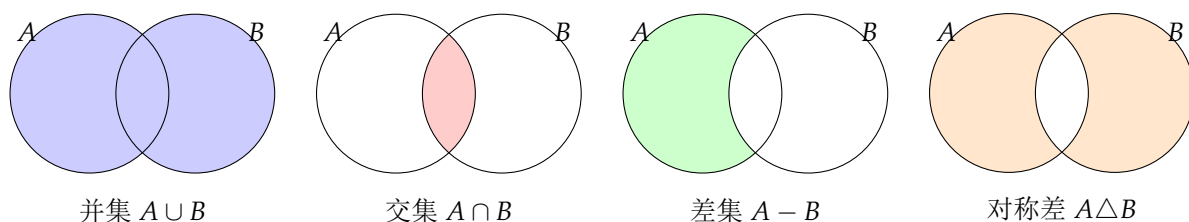


图 3.1: 文氏图——两集合的四种基本运算

## 3.3.4 广义并和广义交

$$\bigcup_{i=1}^n A_i = \{x \mid \exists i, x \in A_i\}, \quad \bigcap_{i=1}^n A_i = \{x \mid \forall i, x \in A_i\}$$

§3.4  
集合恒等式

以下列出常用的集合恒等式，每个可用子集法（证明  $\subseteq$  且  $\supseteq$ ）或命题逻辑等价法证明。

| 编号 | 名称  | 恒等式                                                                                                  |
|----|-----|------------------------------------------------------------------------------------------------------|
| S1 | 幂等律 | $A \cup A = A; A \cap A = A$                                                                         |
| S2 | 交换律 | $A \cup B = B \cup A; A \cap B = B \cap A$                                                           |
| S3 | 结合律 | $(A \cup B) \cup C = A \cup (B \cup C)$                                                              |
| S4 | 分配律 | $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$<br>$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$ |
| S5 | 同一律 | $A \cup \emptyset = A; A \cap U = A$                                                                 |
| S6 | 零律  | $A \cup U = U; A \cap \emptyset = \emptyset$                                                         |
| S7 | 排中律 | $A \cup \overline{A} = U$                                                                            |
| S8 | 矛盾律 | $A \cap \overline{A} = \emptyset$                                                                    |

## 德 摩 根 律

$$\overline{A \cup B} = \overline{A} \cap \overline{B}, \quad \overline{A \cap B} = \overline{A} \cup \overline{B}$$

| 编号  | 名称    | 恒等式                                            |
|-----|-------|------------------------------------------------|
| S10 | 吸收律   | $A \cup (A \cap B) = A; A \cap (A \cup B) = A$ |
| S11 | 补余律   | $\overline{\overline{A}} = A$                  |
| S12 | 差集恒等式 | $A - B = A \cap \overline{B}$                  |
| S13 | 对称差   | $A \Delta B = (A \cup B) - (A \cap B)$         |
| S14 |       | $A \cup (B - A) = A \cup B$                    |
| S15 |       | $A \cap (B - A) = \emptyset$                   |
| S16 |       | $A - (B \cup C) = (A - B) \cap (A - C)$        |
| S17 |       | $A - (B \cap C) = (A - B) \cup (A - C)$        |

## 3.4.1 恒等式证明示例

【例题 3.2】证明 S4 (分配律):  $A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$ 。

(一) 证  $\subseteq$ : 任取  $x \in A \cup (B \cap C)$ 。若  $x \in A$ , 则  $x \in A \cup B$  且  $x \in A \cup C$ , 故  $x \in (A \cup B) \cap (A \cup C)$ 。  
若  $x \in B \cap C$ , 则  $x \in B$  且  $x \in C$ , 故  $x \in A \cup B$  且  $x \in A \cup C$ 。

(二) 证  $\supseteq$ : 任取  $x \in (A \cup B) \cap (A \cup C)$ 。若  $x \in A$ , 则  $x \in A \cup (B \cap C)$ 。若  $x \notin A$ , 则由  $x \in A \cup B$  知  $x \in B$ ; 由  $x \in A \cup C$  知  $x \in C$ , 故  $x \in B \cap C \subseteq A \cup (B \cap C)$ 。□

【例题 3.3】证明 S9 (德摩根律之一):  $\overline{A \cup B} = \overline{A} \cap \overline{B}$ 。

$$\begin{aligned} x \in \overline{A \cup B} &\iff x \notin A \cup B \iff \neg(x \in A \vee x \in B) \\ &\iff \neg(x \in A) \wedge \neg(x \in B) \iff x \in \overline{A} \wedge x \in \overline{B} \\ &\iff x \in \overline{A} \cap \overline{B} \end{aligned}$$

## §3.5

## 有序对与笛卡尔积

## 有序对

$(a, b) = \{\{a\}, \{a, b\}\}$  (库拉托夫斯基定义)。核心性质:

$$(a, b) = (c, d) \iff a = c \wedge b = d$$

注意  $\{a, b\} = \{b, a\}$  但  $(a, b) \neq (b, a)$  (除非  $a = b$ )。

## 笛卡尔积

集合  $A$  与  $B$  的笛卡尔积为:

$$A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$$

$$|A \times B| = |A| \cdot |B|。$$

笛卡尔积的性质:

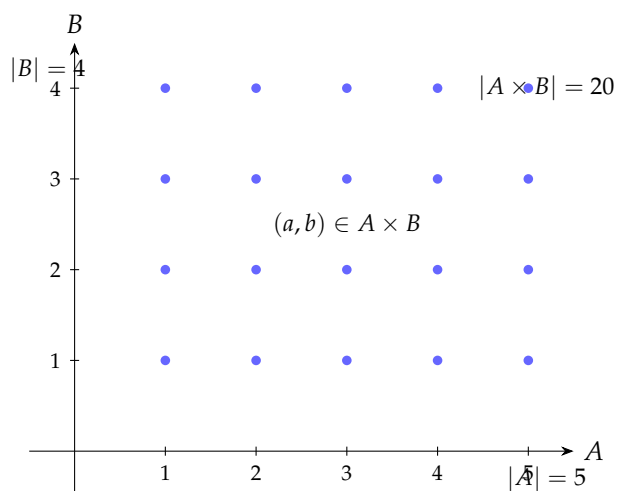
- $A \times \emptyset = \emptyset \times A = \emptyset$
- 一般  $A \times B \neq B \times A$  (不满足交换律)
- $A \times (B \cup C) = (A \times B) \cup (A \times C)$  (分配律)
- $A \times (B \cap C) = (A \times B) \cap (A \times C)$

3.5.1  $n$  元有序组

递归定义:  $(a_1, \dots, a_n) = ((a_1, \dots, a_{n-1}), a_n)$ 。

$$A_1 \times A_2 \times \dots \times A_n = \{(a_1, \dots, a_n) \mid a_i \in A_i\}; \quad A^n = \underbrace{A \times A \times \dots \times A}_{n \text{ 个}}$$

例如  $\mathbb{R}^2 = \mathbb{R} \times \mathbb{R}$  (平面),  $\mathbb{B}^n = \{0, 1\}^n$  ( $2^n$  个  $n$  位二进制串)。

图 3.2: 笛卡尔积  $A \times B$  的几何表示——每个点对应一个有序对

### §3.6 容斥原理

#### 3.6.1 两集合与三集合

##### 两集合容斥原理

$$|A \cup B| = |A| + |B| - |A \cap B|$$

直观解释：分别计算  $|A|$  和  $|B|$  时， $A \cap B$  被计算了两次，因此需要减去一次。

##### 三集合容斥原理

$$\begin{aligned} |A \cup B \cup C| &= |A| + |B| + |C| \\ &\quad - |A \cap B| - |A \cap C| - |B \cap C| \\ &\quad + |A \cap B \cap C| \end{aligned}$$

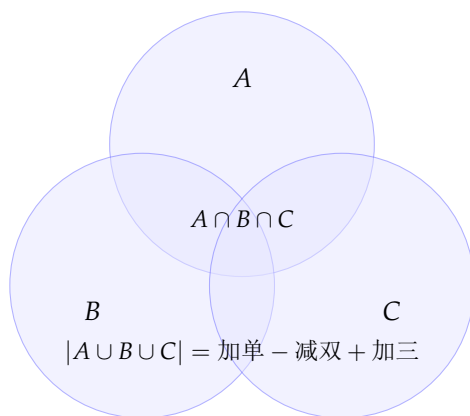


图 3.3: 三集合容斥原理文氏图——先加单集，减去两两交集，再加回三集交集

3.6.2 一般  $n$  个集合的容斥原理

## 容斥原理一般形式

$$\left| \bigcup_{i=1}^n A_i \right| = \sum_{k=1}^n (-1)^{k-1} \sum_{1 \leq i_1 < \dots < i_k \leq n} |A_{i_1} \cap A_{i_2} \cap \dots \cap A_{i_k}|$$

符号规则：奇数个集合的交集前符号为  $+$ ，偶数个为  $-$ 。

补集形式：

$$\left| \bigcap_{i=1}^n \overline{A_i} \right| = |U| - \left| \bigcup_{i=1}^n A_i \right|$$

## 3.6.3 应用示例

**【例题 3.4】** 1 到 100 中能被 3 或 5 整除的整数有多少个？

$$|A| = \lfloor 100/3 \rfloor = 33, \quad |B| = \lfloor 100/5 \rfloor = 20, \quad |A \cap B| = \lfloor 100/15 \rfloor = 6.$$

$$|A \cup B| = 33 + 20 - 6 = 47.$$

**【例题 3.5】** 三集合问题：某班 50 人，学英语 30 人，学日语 20 人，学法语 15 人；学英 + 日 10 人，学英 + 法 7 人，学日 + 法 5 人；三门都学 3 人。问至少学一门的有多少人？

$$|E \cup J \cup F| = 30 + 20 + 15 - 10 - 7 - 5 + 3 = 46 \text{ 人}.$$

$\mathcal{P}(\{a, b, c\})$  的哈塞图（按  $\subseteq$  偏序）

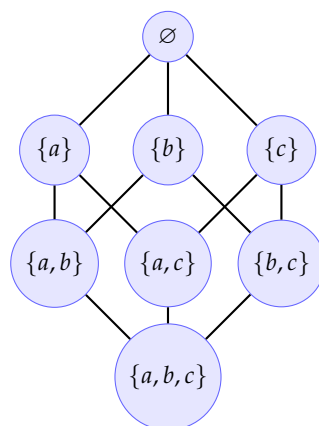


图 3.4: 幂集  $\mathcal{P}(\{a, b, c\})$  的哈塞图——共  $2^3 = 8$  个子集

## Python 集合运算与容斥原理

```

1 from itertools import combinations
2 from typing import Set, List, Any
3
4 A = {1, 2, 3, 4, 5}
5 B = {4, 5, 6, 7, 8}
6
7 # 基本运算
8 union = A | B           # {1,2,3,4,5,6,7,8}
9 inter = A & B           # {4,5}
10 diff = A - B           # {1,2,3}
11 sym = A ^ B            # {1,2,3,6,7,8}
12 print(f"A   B: {A <= B}") # False
13
14 def cartesian_product(*sets: Set[Any]) -> Set[tuple]:
15     result = {(x,) for x in sets[0]}
16     for s in sets[1:]:
17         result = {r + (y,) for r in result for y in s}
18     return result
19
20 def powerset(s: Set[Any]) -> Set[frozenset]:
21     elements = list(s)
22     n = len(elements)
23     return {frozenset(elements[i] for i in range(n)
24         if (mask >> i) & 1) for mask in range(1 << n)}
25
26 def inclusion_exclusion(sets: List[Set[Any]]) -> int:
27     n = len(sets)
28     total = 0
29     for k in range(1, n + 1):
30         sign = (-1) ** (k - 1)
31         for indices in combinations(range(n), k):
32             inter = sets[indices[0]]
33             for idx in indices[1:]:
34                 inter = inter & sets[idx]
35             total += sign * len(inter)
36     return total
37
38 # 验证容斥原理
39 div3 = {x for x in range(1, 101) if x % 3 == 0}
40 div5 = {x for x in range(1, 101) if x % 5 == 0}
41 print(f"|3的倍数   5的倍数| = {inclusion_exclusion([div3, div5])}") # 47

```

关系与函数是离散数学的核心主题。关系刻画了集合中元素之间的联系，函数则是特殊的关系（满足单值性）。等价关系和偏序关系是两类最重要的关系，分别刻画了“相等”和“大小”这两种最基本的数学概念。

### §4.1 二元关系的基本概念

#### 二元关系

设  $A$  和  $B$  是两个集合，从  $A$  到  $B$  的二元关系  $R$  是笛卡尔积  $A \times B$  的一个子集：

$$R \subseteq A \times B = \{(a, b) \mid a \in A, b \in B\}$$

若  $(a, b) \in R$ ，记作  $aRb$ 。当  $A = B$  时，称  $R$  是  $A$  上的关系。

特殊关系：

- 空关系：  $R = \emptyset$
- 全域关系：  $R = A \times B$
- 恒等关系：  $I_A = \{(a, a) \mid a \in A\}$

#### 关系总数

设  $|A| = m$ ， $|B| = n$ ，则从  $A$  到  $B$  的不同关系有  $2^{mn}$  个。

证明：  $A \times B$  有  $mn$  个元素，每个元素可选入或不选入  $R$ ，故总数为  $2^{mn}$ 。

#### 4.1.1 关系的表示法

##### 关系矩阵

设  $A = \{a_1, \dots, a_m\}$ ， $B = \{b_1, \dots, b_n\}$ ，关系  $R \subseteq A \times B$  的关系矩阵  $M_R = (m_{ij})_{m \times n}$  定义为：

$$m_{ij} = \begin{cases} 1, & \text{若 } (a_i, b_j) \in R \\ 0, & \text{若 } (a_i, b_j) \notin R \end{cases}$$

##### 关系图

将集合  $A$  中的元素用点（顶点）表示，若  $(a_i, a_j) \in R$ ，则从  $a_i$  到  $a_j$  画一条有向边（弧）。当  $R$  是  $A$  上的关系时，关系图是一个有向图。

### 4.1.2 定义域、值域与域

$\text{dom}(R) = \{a \in A \mid \exists b \in B, (a, b) \in R\}$ ,  $\text{ran}(R) = \{b \in B \mid \exists a \in A, (a, b) \in R\}$ ,  $\text{fld}(R) = \text{dom}(R) \cup \text{ran}(R)$ 。

## §4.2 关系的运算

### 4.2.1 逆关系

#### 逆关系

$R^{-1} = \{(b, a) \mid (a, b) \in R\}$ , 且  $R^{-1} \subseteq B \times A$ 。

性质:  $(R^{-1})^{-1} = R$ ;  $M_{R^{-1}} = (M_R)^T$ ;  $\text{dom}(R^{-1}) = \text{ran}(R)$ 。

### 4.2.2 复合关系

#### 复合关系

设  $R \subseteq A \times B$ ,  $S \subseteq B \times C$ , 则:

$$R \circ S = \{(a, c) \mid \exists b \in B, (a, b) \in R \wedge (b, c) \in S\}$$

性质:

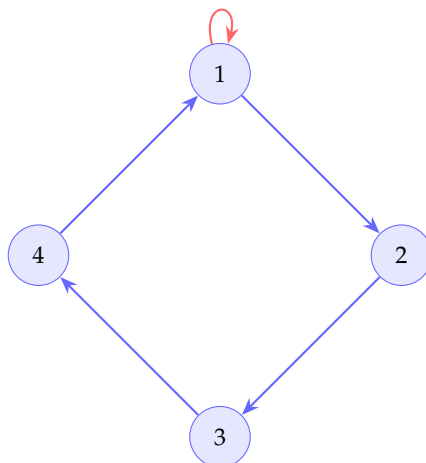
- 结合律:  $(R \circ S) \circ T = R \circ (S \circ T)$
- $R \circ I_B = R$ ,  $I_A \circ R = R$
- $(R \circ S)^{-1} = S^{-1} \circ R^{-1}$  (次序颠倒)
- 一般  $R \circ S \neq S \circ R$  (不满足交换律)

### 4.2.3 关系矩阵的布尔积

关系复合的矩阵表示使用布尔积:

$$M_{R \circ S} = M_R \odot M_S, \quad (M_R \odot M_S)_{ij} = \bigvee_{k=1}^n (m_{ik}^{(R)} \wedge m_{kj}^{(S)})$$

即将普通矩阵乘法中的  $+$  替换为  $\vee$ ,  $\times$  替换为  $\wedge$ 。



关系图  $R = \{(1,1), (1,2), (2,3), (3,4), (4,1)\}$

图 4.1: 关系图——有向图表示关系

### §4.3 关系的性质

设  $R$  是集合  $A$  上的关系。

#### 4.3.1 自反性与反自反性

##### 自反性

若对每个  $a \in A$ ，都有  $(a,a) \in R$ ，则称  $R$  是自反的。矩阵判断：主对角线全为 1。关系图判断：每个顶点都有自环。

若对每个  $a \in A$ ，都有  $(a,a) \notin R$ ，则称  $R$  是反自反的。矩阵判断：主对角线全为 0。

#### 4.3.2 对称性与反对称性

##### 对称性与反对称性

对称的：  $(a,b) \in R \Rightarrow (b,a) \in R$ 。矩阵判断：  $M_R^T = M_R$ （对称矩阵）。

反对称的：  $(a,b) \in R \wedge (b,a) \in R \Rightarrow a = b$ 。矩阵判断：若  $i \neq j$ ，则  $m_{ij}$  与  $m_{ji}$  不能同时为 1。

#### 4.3.3 传递性

##### 传递性

$(a,b) \in R \wedge (b,c) \in R \Rightarrow (a,c) \in R$ 。矩阵判断：  $M_{R \circ R} \preceq M_R$ （即  $R \circ R \subseteq R$ ）。



## 4.3.4 性质间的关系

| 性质   | $R^{-1}$ 保持 | $R \cap S$ 保持 | $R \cup S$ 保持 | $R \circ S$ 保持 |
|------|-------------|---------------|---------------|----------------|
| 自反性  | ✓           | ✓             | ✓             | ✓              |
| 反自反性 | ✓           | ✓             | ✓             |                |
| 对称性  | ✓           | ✓             | ✓             |                |
| 反对称性 | ✓           | ✓             |               |                |
| 传递性  | ✓           | ✓             |               |                |

## 关系性质与矩阵特征

自反:  $\forall i, m_{ii} = 1$

反自反:  $\forall i, m_{ii} = 0$

对称:  $\forall i, j, m_{ij} = m_{ji}$

反对称:  $\forall i \neq j, m_{ij} \cdot m_{ji} = 0$

传递:  $R \circ R \subseteq R$

§4.4  
闭包

设  $R$  是  $A$  上的关系,  $R$  关于性质  $P$  的**闭包**  $R_P$  是满足  $R \subseteq R_P$ 、 $R_P$  具有性质  $P$ 、且具有最小性的关系。

| 符号     | 名称   | 构造方法                                |
|--------|------|-------------------------------------|
| $r(R)$ | 自反闭包 | $r(R) = R \cup I_A$                 |
| $s(R)$ | 对称闭包 | $s(R) = R \cup R^{-1}$              |
| $t(R)$ | 传递闭包 | $t(R) = \bigcup_{i=1}^{\infty} R^i$ |

## 4.4.1 Warshall 算法求传递闭包

**思想:** 逐步构造矩阵序列  $W_0, \dots, W_n$ , 其中  $W_k[i][j] = 1$  当且仅当存在从  $i$  到  $j$  的路径且内部顶点编号不超过  $k$ 。

**递推公式:**

$$W_k[i][j] = W_{k-1}[i][j] \vee (W_{k-1}[i][k] \wedge W_{k-1}[k][j])$$

$W_0 = M_R$ ,  $W_n = M_{t(R)}$ 。时间复杂度  $O(n^3)$ 。

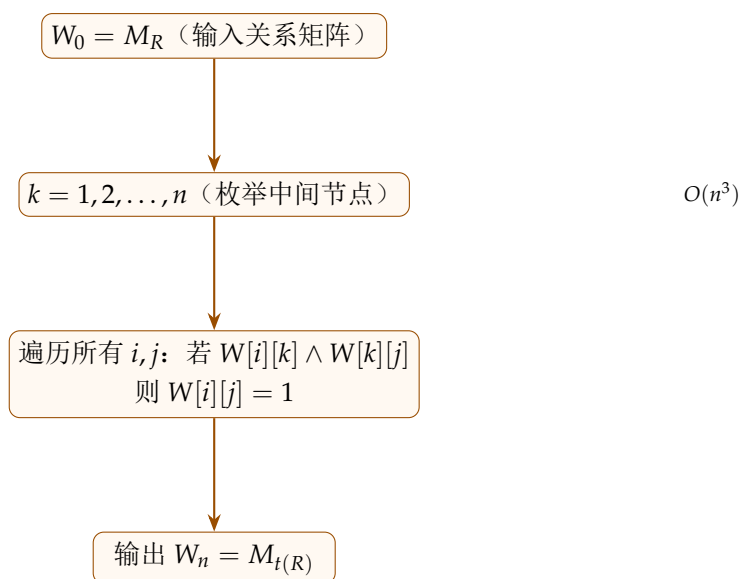


图 4.2: Warshall 算法流程图——三重循环求传递闭包

## Warshall 算法

```

1 def warshall(W):
2     """Warshall 算法求传递闭包"""
3     n = len(W)
4     for k in range(n):
5         for i in range(n):
6             if W[i][k]:
7                 for j in range(n):
8                     W[i][j] = W[i][j] or W[k][j]
9     return W
10
11 # 示例
12 W = [
13     [1, 0, 1, 0],
14     [0, 1, 0, 1],
15     [0, 0, 1, 0],
16     [0, 0, 0, 1]
17 ]
18 result = warshall([row[:] for row in W])
19 for row in result:
20     print(row)
  
```

## Warshall 算法 C++ 版本

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void warshall(vector<vector<int>>& W) {
6      int n = W.size();
7      for (int k = 0; k < n; k++) {
8          for (int i = 0; i < n; i++) {
9              if (W[i][k]) {
10                 for (int j = 0; j < n; j++) {
11                     W[i][j] = W[i][j] || W[k][j];
12                 }
13             }
14         }
15     }
16 }
17
18 int main() {
19     int n = 4;
20     vector<vector<int>> W = {
21         {1, 0, 1, 0},
22         {0, 1, 0, 1},
23         {0, 0, 1, 0},
24         {0, 0, 0, 1}
25     };
26     warshall(W);
27     for (auto& row : W) {
28         for (int v : row) cout << v << " ";
29         cout << endl;
30     }
31     return 0;
32 }

```

§4.5  
等价关系与划分

## 等价关系

集合  $A$  上的关系  $R$  若满足自反性、对称性、传递性，则称  $R$  为一个等价关系，通常记作  $\sim$ 。

**【例题 4.1】** 模  $n$  同余关系：在  $\mathbb{Z}$  上， $a \equiv b \pmod{n}$ （即  $n \mid (a - b)$ ）是等价关系。

- 自反： $a - a = 0$  被任何  $n$  整除 ✓
- 对称：若  $n \mid (a - b)$ ，则  $n \mid (b - a)$  ✓
- 传递：若  $n \mid (a - b)$  且  $n \mid (b - c)$ ，则  $n \mid (a - b) + (b - c) = a - c$  ✓

## 4.5.1 等价类与商集

## 等价类与商集

$a$  的等价类:  $[a]_{\sim} = \{x \in A \mid x \sim a\}$ ,  $a$  称为该等价类的代表元。

$A$  关于  $\sim$  的商集:  $A/\sim = \{[a]_{\sim} \mid a \in A\}$ 。

## 等价类的性质

设  $\sim$  是  $A$  上的等价关系,  $a, b \in A$ , 则下列三命题等价:

1.  $a \sim b$
2.  $[a] = [b]$
3.  $[a] \cap [b] \neq \emptyset$

## 4.5.2 划分与等价关系的一一对应

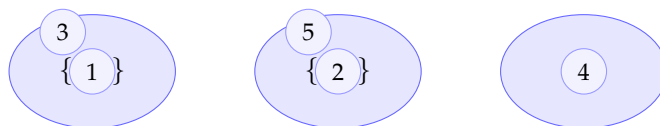
## 划分

集合  $A$  的一个划分是一个由  $A$  的非空子集构成的集合族  $\mathcal{P} = \{A_1, \dots, A_k\}$ , 满足  $\bigcup A_i = A$  且  $A_i \cap A_j = \emptyset$  ( $i \neq j$ )。

## 核心定理

$A$  上的等价关系与  $A$  的划分之间存在一一对应:

- 等价关系  $\rightarrow$  划分: 等价类的集合  $A/\sim$  构成  $A$  的一个划分。
- 划分  $\rightarrow$  等价关系: 定义  $a \sim b$  当  $a$  与  $b$  属于划分的同一块。



$A = \{1, 2, 3, 4, 5\}$ , 划分  $\{\{1, 3\}, \{2, 5\}, \{4\}\}$

图 4.3: 等价类的划分示意——同一等价类内的元素相互等价

§ 4.6  
偏序关系

## 偏序关系

集合  $A$  上的关系  $R$  若满足自反性、反对称性、传递性, 则称  $R$  为一个偏序关系, 记作  $\preceq$ 。称  $(A, \preceq)$  为偏序集。

典型偏序:  $(\mathbb{R}, \leq)$ 、 $(\mathcal{P}(X), \subseteq)$  (集合包含)、 $(\mathbb{N}, |)$  (整除关系)。

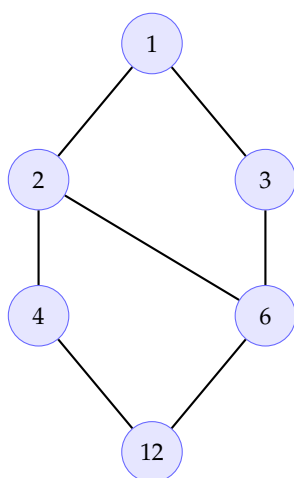
**可比性**

在偏序集  $(A, \preceq)$  中，若  $a \preceq b$  或  $b \preceq a$ ，称  $a$  与  $b$  可比；否则称不可比。

**4.6.1 哈塞图**

哈塞图绘制规则：

1. 用小圆圈表示每个元素。
2. 若  $a \prec b$  且不存在  $c$  使  $a \prec c \prec b$ （即  $b$  覆盖  $a$ ），将  $a$  放在  $b$  下方并用线段连接。
3. 不画自环（自反性）和传递边（传递性）。



{1, 2, 3, 4, 6, 12} 上整除关系的哈塞图

图 4.4: 整除偏序的哈塞图——12 覆盖 4 和 6，4 和 6 覆盖 2，2 和 3 覆盖 1

**4.6.2 极大元/极小元与最大元/最小元**

| 概念  | 定义                      | 存在性     |
|-----|-------------------------|---------|
| 极大元 | 不存在 $b$ 使 $a \prec b$   | 必有，可能多个 |
| 极小元 | 不存在 $b$ 使 $b \prec a$   | 必有，可能多个 |
| 最大元 | 对所有 $b$ , $b \preceq a$ | 至多一个    |
| 最小元 | 对所有 $b$ , $a \preceq b$ | 至多一个    |

**4.6.3 上界/下界与确界**

设  $(A, \preceq)$  是偏序集， $B \subseteq A$ ：

- 上界  $a \in A$ ：对所有  $b \in B$ ,  $b \preceq a$ 。
- 下界  $a \in A$ ：对所有  $b \in B$ ,  $a \preceq b$ 。
- 上确界  $\sup B$ ： $B$  的最小上界。
- 下确界  $\inf B$ ： $B$  的最大下界。

4.6.4 全序与良序

- **全序集**（线性序）：偏序集中任意两元素都可比。如  $(\mathbb{R}, \leq)$  是全序。
- **良序集**：全序集中每个非空子集都有最小元。如  $(\mathbb{N}, \leq)$  是良序， $(\mathbb{Z}, \leq)$  不是。

§ 4.7

函数

函数

设  $f \subseteq A \times B$  是一个关系，若对每个  $a \in A$ ，存在**唯一的**  $b \in B$  使  $(a, b) \in f$ ，则称  $f$  为从  $A$  到  $B$  的**函数**，记作  $f: A \rightarrow B$ ， $f(a) = b$ 。

$A$  为定义域， $B$  为陪域， $f(A) = \{f(a) \mid a \in A\}$  为值域。

4.7.1 单射、满射、双射

| 类型 | 定义                                            | 等价条件                                     |
|----|-----------------------------------------------|------------------------------------------|
| 单射 | $a_1 \neq a_2 \Rightarrow f(a_1) \neq f(a_2)$ | $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$  |
| 满射 | $f(A) = B$                                    | 对每个 $b \in B$ ，存在 $a \in A$ 使 $f(a) = b$ |
| 双射 | 既单射又满射                                        | $A$ 与 $B$ 之间一一对应                         |

4.7.2 函数的复合与逆函数

函数复合： $(g \circ f)(a) = g(f(a))$ ，满足结合律。  
若  $f$  是双射，则存在**逆函数**  $f^{-1}: B \rightarrow A$ ， $f^{-1}(b) = a \iff f(a) = b$ 。  
性质： $f^{-1} \circ f = I_A$ ， $f \circ f^{-1} = I_B$ ； $(g \circ f)^{-1} = f^{-1} \circ g^{-1}$ 。

4.7.3 几类特殊函数

- **恒等函数**  $I_A(a) = a$
- **特征函数**  $\chi_A: U \rightarrow \{0, 1\}$ ， $\chi_A(x) = 1$  当  $x \in A$ ，否则 0
- **取整函数**：下取整  $\lfloor x \rfloor$ ，上取整  $\lceil x \rceil$

§ 4.8

鸽巢原理

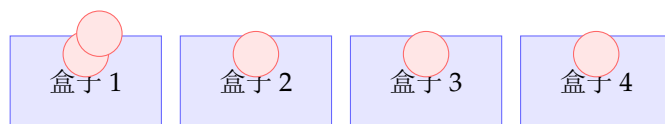
鸽巢原理基本形式

若把  $n + 1$  个物体放入  $n$  个盒子，则至少有一个盒子包含至少 2 个物体。

**证明.** 反证法。若每个盒子至多包含 1 个物体，则  $n$  个盒子至多包含  $n$  个物体，与有  $n + 1$  个物体矛盾。□

**加强形式:** 若把  $q_1 + q_2 + \cdots + q_n - n + 1$  个物体放入  $n$  个盒子，则存在某个  $i$  使第  $i$  个盒子包含至少  $q_i$  个物体。

**平均形式：**若  $m$  个物体放入  $n$  个盒子，则至少有一个盒子包含至少  $\lceil m/n \rceil$  个物体。



$n = 4$  个盒子， $n + 1 = 5$  个物体，盒子 1 必定含  $\geq 2$  个

图 4.5: 鸽巢原理示意图——5 个物体放入 4 个盒子

**【例题 4.2】** 在任意 13 个人中，至少有 2 人生日在同一个月。(12 个月为 12 个盒子，13 个人为物体)

## 关系性质检查与等价类求解

```

1 class Relation:
2     def __init__(self, n):
3         self.n = n
4         self.matrix = [[0] * n for _ in range(n)]
5
6     def add(self, a, b):
7         self.matrix[a][b] = 1
8
9     def is_reflexive(self):
10        return all(self.matrix[i][i] for i in range(self.n))
11
12    def is_irreflexive(self):
13        return all(not self.matrix[i][i] for i in range(self.n))
14
15    def is_symmetric(self):
16        return all(self.matrix[i][j] == self.matrix[j][i]
17                    for i in range(self.n) for j in range(self.n))
18
19    def is_antisymmetric(self):
20        for i in range(self.n):
21            for j in range(self.n):
22                if i != j and self.matrix[i][j] and self.matrix[j][i]:
23                    return False
24        return True
25
26    def is_transitive(self):
27        for i in range(self.n):
28            for j in range(self.n):
29                for k in range(self.n):
30                    if (self.matrix[i][j] and self.matrix[j][k]
31                        and not self.matrix[i][k]):
32                        return False
33        return True
34
35    def is_equivalence(self):
36        return (self.is_reflexive() and self.is_symmetric()
37                and self.is_transitive())
38
39    def is_partial_order(self):
40        return (self.is_reflexive() and self.is_antisymmetric()
41                and self.is_transitive())
42
43    def transitive_closure(self):
44        W = [row[:] for row in self.matrix]
45        for k in range(self.n):
46            for i in range(self.n):
47                if W[i][k]:
48                    for j in range(self.n):
49                        W[i][j] = W[i][j] or W[k][j]
50        return W
51
52    def equivalence_classes(self):
53        closure = self.transitive_closure()
54        reach = [[closure[i][j] or closure[j][i] for j in range(self.n)]

```





组合数学研究离散对象的计数、排列、组合与存在性问题，是离散数学的核心分支之一。

### §5.1 计数基本原理

#### 5.1.1 加法原理

设  $A_1, A_2, \dots, A_k$  是两两不交的有限集，则

$$\left| \bigcup_{i=1}^k A_i \right| = \sum_{i=1}^k |A_i|$$

通俗地说：若有  $k$  项互斥的任务，第  $i$  项有  $n_i$  种完成方式，则从中任选一项完成共有  $n_1 + n_2 + \dots + n_k$  种方式。

**【例题 5.1】** 某班有男生 20 人、女生 18 人，选 1 名代表，共有  $20 + 18 = 38$  种选法。

#### 5.1.2 乘法原理

若  $A_1, A_2, \dots, A_k$  是有限集，则

$$|A_1 \times A_2 \times \dots \times A_k| = \prod_{i=1}^k |A_i|$$

通俗地说：若一项任务可分解为  $k$  个独立步骤，第  $i$  步有  $n_i$  种选择，则完成该任务共有  $n_1 \times n_2 \times \dots \times n_k$  种方式。

**【例题 5.2】** 车牌号格式为“字母-字母-数字-数字”，字母用 26 个大写字母，数字用 0-9，共有  $26 \times 26 \times 10 \times 10 = 67600$  种不同车牌。

**【例题 5.3】** 从  $A$  到  $B$  有 3 条路，从  $B$  到  $C$  有 4 条路，则从  $A$  经  $B$  到  $C$  有  $3 \times 4 = 12$  种走法。

#### 5.1.3 减法原理

在全集  $U$  中求满足某性质的元素个数，可以先求不具有该性质的元素个数，然后从总数中减去：

$$|A| = |U| - |\overline{A}|$$

这本质上是容斥原理的最简形式。

**【例题 5.4】** 从 1 到 100 中, 有多少个数既不含因子 2 也不含因子 3? 含因子 2 的有  $\lfloor 100/2 \rfloor = 50$  个, 含因子 3 的有  $\lfloor 100/3 \rfloor = 33$  个, 同时含因子 2 和 3 (即含因子 6) 的有  $\lfloor 100/6 \rfloor = 16$  个。所以不含 2 也不含 3 的有  $100 - 50 - 33 + 16 = 23$  个。

## §5.2 排列

### 5.2.1 线排列 (不重复排列)

从  $n$  个不同元素中取出  $r$  个 ( $0 \leq r \leq n$ ) 按顺序排成一列, 称为一个  $r$ -排列, 其总数记为  $P(n, r)$  或  $P_n^r$ :

$$P(n, r) = n \cdot (n-1) \cdot (n-2) \cdots (n-r+1) = \frac{n!}{(n-r)!}$$

**推导:** 第一位有  $n$  种选择, 第二位有  $n-1$  种 (不能重复),  $\cdots$ , 第  $r$  位有  $n-r+1$  种。由乘法原理即得。

**特例:**  $r = n$  时,  $P(n, n) = n!$  (全排列)。约定  $0! = 1$ ,  $P(n, 0) = 1$ 。

**【例题 5.5】** 5 个人排队, 前 3 人领奖的安排方式有  $P(5, 3) = 5 \times 4 \times 3 = 60$  种。

### 5.2.2 可重复排列

#### 可重复排列

从  $n$  个不同元素中取出  $r$  个排成一列, 允许重复选取, 则排列总数为

$$n^r$$

**【例题 5.6】** 3 位数密码 (每位 0-9) 共有  $10^3 = 1000$  种组合。

**【例题 5.7】**  $n$  个元素的集合  $A$  到  $m$  个元素的集合  $B$  的函数总数为  $m^n$  (每个  $a \in A$  有  $m$  种映射选择)。

### 5.2.3 圆排列

#### 圆排列公式

$n$  个不同元素的圆排列 (环形排列) 数为

$$(n-1)!$$

**推导:** 将  $n$  个元素排在一个圆圈上, 旋转后重合视为同一排列。首先选定一个元素固定在某位置 (打破旋转对称性), 剩下  $n-1$  个元素全排列, 得  $(n-1)!$ 。

**注.** 若圆排列可以翻转 (即翻转后重合视为同一), 则数量为  $\frac{(n-1)!}{2}$  (当  $n \geq 3$ )。

**【例题 5.8】** 6 个人围桌而坐, 有  $(6-1)! = 120$  种坐法。

## 5.2.4 多重集的排列

多重集有  $k$  种不同类型的元素，第  $i$  种有  $n_i$  个相同元素，总元素数  $n = n_1 + n_2 + \cdots + n_k$ 。则多重集的排列数为

$$\frac{n!}{n_1! n_2! \cdots n_k!}$$

【例题 5.9】“MISSISSIPPI”的字母排列数：M=1, I=4, S=4, P=2，共 11 个字母。

$$\frac{11!}{1! \times 4! \times 4! \times 2!} = 34650$$

§ 5.3  
组合

## 5.3.1 组合数（不重复组合）

从  $n$  个不同元素中取出  $r$  个 ( $0 \leq r \leq n$ ) 不计顺序组成一组，称为一个  $r$ -组合，其总数记为  $C(n, r)$ 、 $\binom{n}{r}$  或  $C_n^r$ ：

$$C(n, r) = \binom{n}{r} = \frac{P(n, r)}{r!} = \frac{n!}{r! (n-r)!}$$

推导：每个  $r$ -组合对应  $r!$  个不同的  $r$ -排列，故  $P(n, r) = r! \cdot C(n, r)$ 。

【例题 5.10】从 10 人中选 3 人组成委员会，共有  $\binom{10}{3} = 120$  种选法。

## 5.3.2 可重复组合（隔板法）

## 可重复组合公式

$n$  种元素中选出  $r$  个的可重复组合数为

$$\binom{n+r-1}{r} = \binom{n+r-1}{n-1}$$

隔板法证明：将  $r$  个相同球放入  $n$  个不同的盒子（允许空盒）。用  $r$  个星  $\star$  和  $(n-1)$  条竖线  $|$  排列表示分配方案。总共有  $r+n-1$  个位置，选其中  $n-1$  个放竖线，故有  $\binom{r+n-1}{n-1}$  种方案。

【例题 5.11】方程  $x_1 + x_2 + x_3 = 10$  的非负整数解个数为

$$\binom{10+3-1}{3-1} = \binom{12}{2} = 66$$

## §5.4 二项式定理与组合恒等式

### 5.4.1 二项式定理

#### 二项式定理

对任意实数  $x, y$  和正整数  $n$ ,

$$(x+y)^n = \sum_{k=0}^n \binom{n}{k} x^{n-k} y^k = \sum_{k=0}^n \binom{n}{k} x^k y^{n-k}$$

证明（组合意义）： $(x+y)^n = \underbrace{(x+y)(x+y)\cdots(x+y)}_{n \text{ 个}}$ ，展开后每项形如  $x^{n-k}y^k$ ，系数等于从  $n$  个因子中选  $k$  个取  $y$  的方案数  $\binom{n}{k}$ 。

常用推论（取特殊值）：

$$\begin{aligned} x=1, y=1 &\Rightarrow \sum_{k=0}^n \binom{n}{k} = 2^n \\ x=1, y=-1 &\Rightarrow \sum_{k=0}^n (-1)^k \binom{n}{k} = 0 \end{aligned}$$

### 5.4.2 杨辉三角（Pascal 三角）

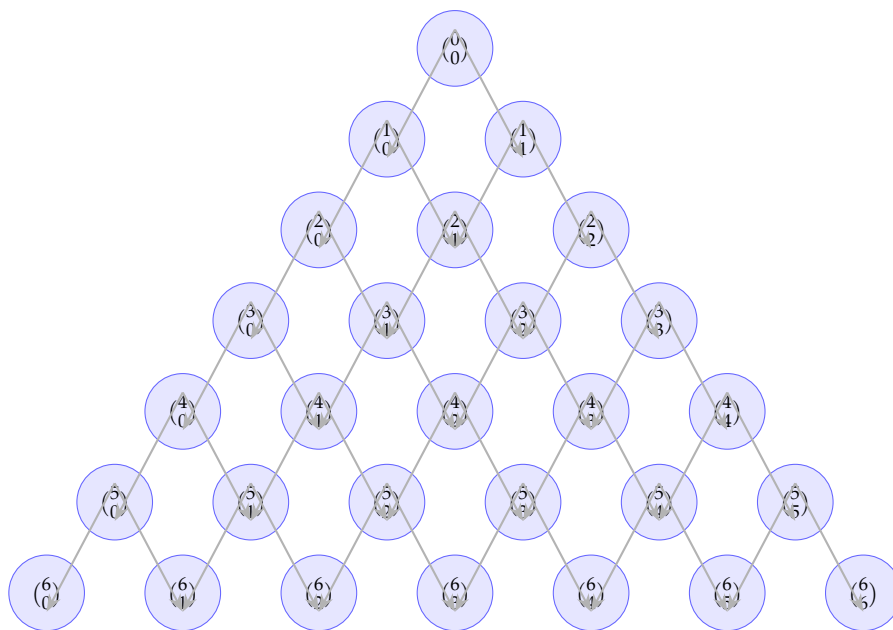
#### Pascal 恒等式

$$\binom{n}{r} = \binom{n-1}{r} + \binom{n-1}{r-1} \quad (0 < r < n)$$

证明（组合意义）：从  $n$  个中选  $r$  个，考虑是否包含某个固定元素：

- 不包含：从其余  $n-1$  个中选  $r$  个，共  $\binom{n-1}{r}$  种
- 包含：从其余  $n-1$  个中选  $r-1$  个，共  $\binom{n-1}{r-1}$  种

下面用 TikZ 绘制杨辉三角，直观展示 Pascal 恒等式的递推结构：



杨辉三角：每个数是上方两数之和（Pascal 恒等式）

### 5.4.3 组合恒等式

除 Pascal 恒等式外，还有以下重要恒等式：

对称性： $\binom{n}{r} = \binom{n}{n-r}$

朱世杰恒等式（Vandermonde 卷积）：

$$\sum_{k=0}^r \binom{m}{k} \binom{n}{r-k} = \binom{m+n}{r}$$

证明（组合意义）：从  $m$  男  $n$  女共  $m+n$  人中选  $r$  人，按选出的男生数  $k$  分类，第  $k$  类有  $\binom{m}{k} \times \binom{n}{r-k}$  种选法，求和即得。

其他常用恒等式：

$$\sum_{k=0}^n \binom{n}{k} = 2^n \quad (5.1)$$

$$\sum_{k=0}^n (-1)^k \binom{n}{k} = 0 \quad (n \geq 1) \quad (5.2)$$

$$\sum_{k=0}^n k \binom{n}{k} = n \cdot 2^{n-1} \quad (5.3)$$

## §5.5 鸽巢原理

### 5.5.1 基本形式

#### 鸽巢原理

若把  $n+1$  个物体放入  $n$  个盒子中，则至少有一个盒子包含至少 2 个物体。

证明. 反证法。设每个盒子至多放 1 个，则  $n$  个盒子至多容纳  $n$  个物体，矛盾。 □

【例题 5.12】任意 13 个人中，至少有 2 人在同一个月出生。

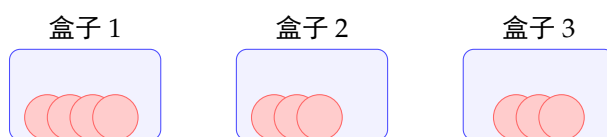
【例题 5.13】从 1 到  $2n$  中任取  $n+1$  个数，必有两数互质（取相邻两数）。

## 5.5.2 加强形式

### 鸽巢原理（加强形式）

设  $q_1, q_2, \dots, q_n$  都是正整数，若将  $q_1 + q_2 + \dots + q_n - n + 1$  个物体放入  $n$  个盒子，则第 1 个盒子至少有  $q_1$  个，或第 2 个盒子至少有  $q_2$  个，……，或第  $n$  个盒子至少有  $q_n$  个。

推论（平均形式）： $m$  个物体放入  $n$  个盒子，则至少有一个盒子包含至少  $\lceil m/n \rceil$  个物体。



10 个鸽子放入 3 个盒子，由鸽巢原理，至少有一个盒子包含  $\lceil 10/3 \rceil = 4$  个鸽子

## 5.5.3 Ramsey 理论入门

### Ramsey 数

对任意正整数  $k, l$ ，存在最小的正整数  $R(k, l)$ ，使得任何  $R(k, l)$  个人的群体中，要么有  $k$  个人彼此认识，要么有  $l$  个人彼此不认识。

已知结果： $R(3, 3) = 6$ ， $R(4, 4) = 18$ ，而  $R(5, 5)$  至今未知（已知在 43 到 48 之间）。

【例题 5.14】 $R(3, 3) = 6$  的证明：6 人中必存在 3 人彼此都认识或彼此都不认识。任取一人  $A$ ，其余 5 人中由鸽巢原理，至少有 3 人与  $A$  认识或至少有 3 人与  $A$  不认识。若 3 人与  $A$  认识，这 3 人中若有 2 人认识，则加上  $A$  构成 3 人彼此认识；若 3 人彼此不认识，则构成 3 人彼此不认识。

## §5.6 容斥原理

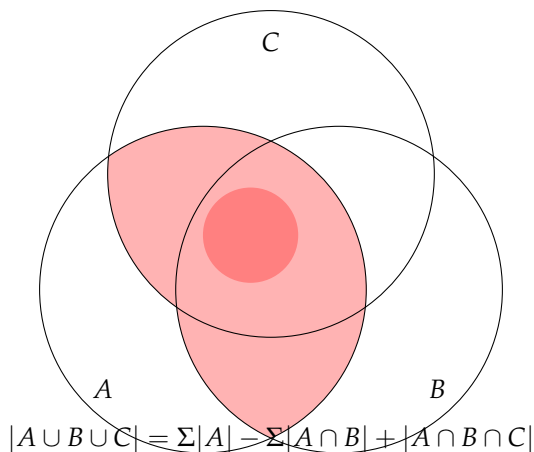
### 5.6.1 基本形式

两集合容斥：

$$|A \cup B| = |A| + |B| - |A \cap B|$$

三集合容斥：

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |C \cap A| + |A \cap B \cap C|$$



### 5.6.2 一般形式

#### 容斥原理

设  $A_1, A_2, \dots, A_n$  是有限集  $U$  的子集, 则

$$\begin{aligned} \left| \bigcup_{i=1}^n A_i \right| &= \sum_{i=1}^n |A_i| - \sum_{1 \leq i < j \leq n} |A_i \cap A_j| \\ &\quad + \sum_{1 \leq i < j < k \leq n} |A_i \cap A_j \cap A_k| - \dots + (-1)^{n-1} |A_1 \cap \dots \cap A_n| \end{aligned}$$

等价的补集形式 (求不在任何  $A_i$  中的元素数) 更常用:

$$\left| \overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_n} \right| = |U| - \sum |A_i| + \sum |A_i \cap A_j| - \dots + (-1)^n |A_1 \cap \dots \cap A_n|$$

### 5.6.3 错位排列

$n$  个元素的排列中, 若每个元素都不在其原始位置 ( $a_i \neq i$ ), 则称为一个 **错位排列**。错位排列数记为  $D_n$  或  $!n$ 。

#### 错位排列公式

$$D_n = n! \sum_{i=0}^n \frac{(-1)^i}{i!} = n! \left( 1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + \frac{(-1)^n}{n!} \right)$$

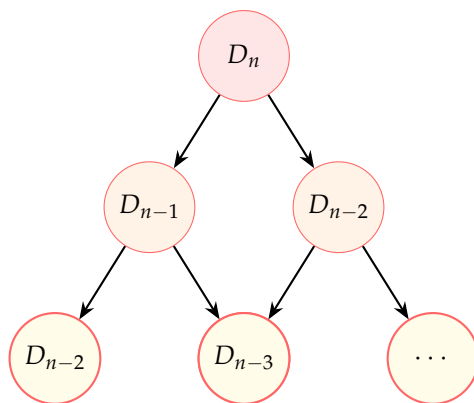
**容斥原理推导:** 设  $A_i$  为第  $i$  个元素在正确位置的排列集合。  $|A_i| = (n-1)!$ ,  $|A_i \cap A_j| = (n-2)!$ , 一般地  $|A_{i_1} \cap \dots \cap A_{i_k}| = (n-k)!$ 。由容斥原理:

$$D_n = n! - \sum_{k=1}^n (-1)^{k-1} \binom{n}{k} (n-k)! = n! \sum_{k=0}^n \frac{(-1)^k}{k!}$$

**递推关系:**  $D_n = (n-1)(D_{n-1} + D_{n-2})$ ,  $D_1 = 0, D_2 = 1$ 。

**渐近公式:**  $D_n \approx \frac{n!}{e}$  (当  $n$  较大时, 约 36.79% 的排列是错位排列)。





错位排列递推结构:  $D_n = (n-1)(D_{n-1} + D_{n-2})$  前几项:  $D_1 = 0, D_2 = 1, D_3 = 2, D_4 = 9, D_5 = 44$

## §5.7 递推关系入门

### 5.7.1 递推关系的基本概念

一个序列  $\{a_n\}$  的递推关系将  $a_n$  表示为该序列中前若干项的函数。给定初始条件，即可唯一确定整个序列。

**【例题 5.15】** 斐波那契数列:  $F_n = F_{n-1} + F_{n-2}$ ,  $F_0 = 0, F_1 = 1$ 。序列:  $0, 1, 1, 2, 3, 5, 8, 13, 21, \dots$

### 5.7.2 一阶线性递推

形式:  $a_n = c \cdot a_{n-1} + g(n)$ , 其中  $c$  为常数。

齐次情形 ( $g(n) = 0$ ):  $a_n = c \cdot a_{n-1}$ , 通解为  $a_n = a_0 \cdot c^n$ 。

**【例题 5.16】** 汉诺塔问题:  $T_n = 2T_{n-1} + 1$ ,  $T_1 = 1$ 。通解:  $T_n = 2^n - 1$ 。

**【例题 5.17】** 求  $n$  位二进制串中没有连续两个 0 的个数。设  $a_n$  为满足条件的  $n$  位串数。递推:  $a_n = a_{n-1} + a_{n-2}$  (最后一位为 1 则前  $n-1$  位任意; 最后一位为 0 则第  $n-1$  位必须为 1),  $a_1 = 2, a_2 = 3$ 。即  $a_n = F_{n+2}$ 。

## §5.8 Catalan 数

### 卡特兰数

第  $n$  个 Catalan 数  $C_n$  为:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

前几项： $C_0 = 1, C_1 = 1, C_2 = 2, C_3 = 5, C_4 = 14, C_5 = 42, C_6 = 132$ 。

满足的递推：

$$C_0 = 1, \quad C_{n+1} = \sum_{i=0}^n C_i C_{n-i}$$

经典应用：

- $n$  对括号的正确匹配方式数： $C_n$
- $n+1$  个矩阵相乘的加括号方式数： $C_n$
- $n+2$  边形的三角剖分数： $C_n$
- $n \times n$  网格中不越对角线的路径数： $C_n$

**【例题 5.18】**  $n=3$  对括号的正确匹配有  $C_3 = 5$  种： $((()))$ ,  $(()())$ ,  $(())()$ ,  $()(())$ ,  $()()()$ 。

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n-1}$$

## §5.9

### 排列组合的编程实现

#### 排列组合计算

```

1 from math import comb, perm
2 from itertools import permutations, combinations
3
4 # 排列数
5 print(f"P(10, 3) = {perm(10, 3)}") # 720
6
7 # 组合数
8 print(f"C(10, 3) = {comb(10, 3)}") # 120
9
10 # 全排列
11 for p in permutations([1, 2, 3]):
12     print(p)
13
14 # 组合
15 for c in combinations([1, 2, 3, 4], 2):
16     print(c)

```

## 杨辉三角 (Pascal Triangle)

```
1 def pascal_triangle(n):
2     tri = [[1]]
3     for i in range(1, n + 1):
4         row = [1]
5         for j in range(1, i):
6             row.append(tri[i - 1][j - 1] + tri[i - 1][j])
7         row.append(1)
8         tri.append(row)
9     return tri
10
11 for row in pascal_triangle(8):
12     print(" ".join(f"{v:3d}" for v in row))
```

## 错位排列计数

```
1 def derangement(n):
2     if n == 0:
3         return 1
4     if n == 1:
5         return 0
6     d0, d1 = 1, 0 # D(0)=1, D(1)=0
7     for i in range(2, n + 1):
8         d0, d1 = d1, (i - 1) * (d0 + d1)
9     return d1
10
11 for i in range(11):
12     print(f"D({i}) = {derangement(i)}")
13 # D(0)=1, D(1)=0, D(2)=1, D(3)=2,
14 # D(4)=9, D(5)=44, D(6)=265, ...
```

## Catalan 数计算

```

1 from math import comb
2
3 def catalan(n):
4     return comb(2 * n, n) // (n + 1)
5
6 # 递推方法
7 def catalan_dp(n):
8     C = [0] * (n + 1)
9     C[0] = 1
10    for i in range(n):
11        C[i + 1] = sum(C[j] * C[i - j] for j in range(i + 1))
12    return C
13
14 print([catalan(i) for i in range(11)])
15 # [1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796]

```

## §5.10

## 本章小结

## 组合数学核心公式一览

| 主题        | 核心公式                                                           |
|-----------|----------------------------------------------------------------|
| 加法原理      | $ A \cup B  =  A  +  B $ (互斥时)                                 |
| 乘法原理      | $ A \times B  =  A  \cdot  B $                                 |
| 排列 (不重复)  | $P(n, r) = \frac{n!}{(n-r)!}$                                  |
| 排列 (可重复)  | $n^r$                                                          |
| 圆排列       | $(n-1)!$                                                       |
| 多重集排列     | $\frac{n!}{n_1! n_2! \dots n_k!}$                              |
| 组合        | $\binom{n}{r} = \frac{n!}{r!(n-r)!}$                           |
| 可重复组合     | $\binom{n+r-1}{r}$                                             |
| 二项式定理     | $(x+y)^n = \sum \binom{n}{k} x^{n-k} y^k$                      |
| 鸽巢原理      | $n+1$ 入 $n$ , 至少一盒 $\geq 2$                                    |
| 容斥原理      | $ \cap \overline{A_i}  = \sum (-1)^{ I }  \cap_{i \in I} A_i $ |
| 错位排列      | $D_n = n! \sum_{i=0}^n \frac{(-1)^i}{i!}$                      |
| Catalan 数 | $C_n = \frac{1}{n+1} \binom{2n}{n}$                            |



图论研究由顶点和连接顶点的边构成的离散结构，起源于 1736 年欧拉对哥尼斯堡七桥问题的研究，如今已成为计算机科学和网络分析等领域的基础工具。

### §6.1 图的基本概念

#### 6.1.1 无向图与有向图的定义

一个无向图是一个有序二元组  $G = (V, E)$ ，其中：

- $V$  是非空的顶点集，元素称为顶点
- $E$  是边集，每条边是  $V$  中的无序对  $\{u, v\}$  ( $u, v \in V$  且通常  $u \neq v$ )

一个有向图是一个有序二元组  $D = (V, E)$ ，其中：

- $V$  是非空的顶点集
- $E$  是弧集，每条弧是  $V$  中的有序对  $(u, v)$ ， $u$  称为起点， $v$  称为终点

图的阶： $|V|$ ，即顶点数，常用  $n$  表示。 图的规模： $|E|$ ，即边数，常用  $m$  表示。

#### 6.1.2 邻接、关联与度

- 邻接：无向图中若  $\{u, v\} \in E$ ，称  $u$  与  $v$  邻接。有向图中若  $(u, v) \in E$ ，称  $u$  邻接到  $v$ 。
- 关联：边  $\{u, v\}$  与顶点  $u$  和  $v$  关联。
- 度  $\deg(v)$ ：与  $v$  关联的边数。有向图中有入度  $\deg^-(v)$  和出度  $\deg^+(v)$ 。
- 孤立点：度为 0 的顶点。悬挂点：度为 1 的顶点。

#### 6.1.3 握手定理

##### 握手定理

对任意无向图  $G = (V, E)$ ，

$$\sum_{v \in V} \deg(v) = 2|E|$$

**证明.** 每条边  $\{u, v\}$  在计算度数时被计数两次（一次给  $u$ ，一次给  $v$ ）。因此所有顶点的度数之和等于边数的两倍。  $\square$

**【推论 6.1】** 任何无向图中，度数为奇数的顶点个数必为偶数。

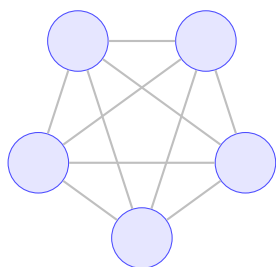
对有向图，有：

$$\sum_{v \in V} \deg^+(v) = \sum_{v \in V} \deg^-(v) = |E|$$

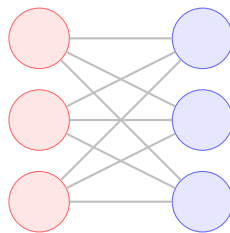
## §6.2 图的分类

### 6.2.1 简单图、完全图与二部图

- **简单图**：无自环且任意两顶点之间至多一条边的无向图。
- **完全图  $K_n$** ： $n$  个顶点的简单图，任意两不同顶点之间都有一条边。 $|E(K_n)| = \binom{n}{2} = \frac{n(n-1)}{2}$ 。
- **二部图**：存在顶点划分  $V = X \cup Y$  ( $X \cap Y = \emptyset$ )，使得每条边都连接  $X$  和  $Y$  中的顶点。
- **完全二部图  $K_{m,n}$** ： $|E(K_{m,n})| = m \cdot n$ 。



$K_5$  (5 阶完全图, 10 条边)



$K_{3,3}$  (完全二部图, 9 条边)

#### 二部图的判定

$G$  是二部图  $\iff G$  中无奇数长度的回路。

### 6.2.2 正则图与补图

- **$k$ -正则图**：所有顶点的度都等于  $k$  的简单图。 $K_n$  是  $(n-1)$ -正则图；Petersen 图是 3-正则图。
- **补图  $\bar{G}$** ：图  $G = (V, E)$  的补图  $\bar{G} = (V, \bar{E})$ ，其中

$$\bar{E} = \{\{u, v\} \mid u, v \in V, u \neq v, \{u, v\} \notin E\}$$

### 6.2.3 子图类型

- **子图**： $V_H \subseteq V$  且  $E_H \subseteq E$ 。
- **生成子图**： $V_H = V$  的子图（覆盖所有顶点）。
- **导出子图  $G[S]$** ：由顶点子集  $S \subseteq V$  导出，包含  $S$  中的所有顶点以及两端点都在  $S$  中的所有边。

### § 6.3 图的表示

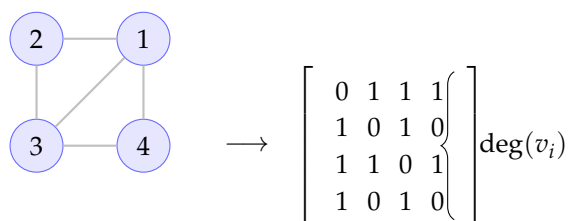
#### 6.3.1 邻接矩阵

设  $G = (V, E)$  的顶点集  $V = \{v_1, v_2, \dots, v_n\}$ , 则  $G$  的邻接矩阵  $A = (a_{ij})_{n \times n}$  定义为:

$$a_{ij} = \begin{cases} 1, & \text{若 } \{v_i, v_j\} \in E \\ 0, & \text{否则} \end{cases}$$

性质:

1. 无向图的邻接矩阵是对称矩阵:  $A^T = A$
2. 第  $i$  行 (或第  $i$  列) 的和 =  $\deg(v_i)$
3.  $A^k$  的意义:  $(A^k)_{ij}$  = 从  $v_i$  到  $v_j$  的长度为  $k$  的通路数



#### 6.3.2 邻接表与关联矩阵

邻接表为每个顶点维护一个邻接顶点链表。对于稀疏图 ( $|E| \ll \binom{n}{2}$ ), 邻接表比邻接矩阵更省空间。

关联矩阵  $B = (b_{ij})_{n \times m}$ :  $b_{ij} = 1$  若  $v_i$  与  $e_j$  关联, 否则  $b_{ij} = 0$ 。有向图中  $b_{ij} = 1$  (起点)、 $b_{ij} = -1$  (终点)。

### § 6.4 路径与连通性

#### 6.4.1 通路与回路

- 通路: 顶点和边交替出现的序列, 长度为边数  $k$ 。
- 迹: 边不重复的通路。
- 路径 (简单通路): 顶点不重复的通路。
- 回路 (圈): 起点 = 终点的长度至少为 1 的通路。

#### 6.4.2 连通性

无向图中, 若存在从  $u$  到  $v$  的通路, 则称  $u$  与  $v$  连通。连通关系是顶点集上的一个等价关系。



- **连通图**：任意两顶点都连通的图。
- **连通分量**：极大连通子图。
- 有向图中：**强连通**（任意两顶点双向可到达）、**弱连通**（忽略方向后连通）。
- **割点**：删除该顶点后连通分量数增加的顶点。
- **桥**：删除该边后连通分量数增加的边。 $e$  是桥  $\iff e$  不在任何简单回路中。

### 6.4.3 点连通度与边连通度

- **点连通度**  $\kappa(G)$ ：使  $G$  不连通或成为平凡图所需删除的最少顶点数。
- **边连通度**  $\lambda(G)$ ：使  $G$  不连通所需删除的最少边数。
- **最小度**  $\delta(G)$ ： $\min_{v \in V} \deg(v)$ 。

#### Whitney 不等式

$$\kappa(G) \leq \lambda(G) \leq \delta(G)$$

## §6.5 欧拉图

- **欧拉通路**：经过图中每条边恰好一次的通路。
- **欧拉回路**：经过图中每条边恰好一次的回路。
- **欧拉图**：存在欧拉回路的图。

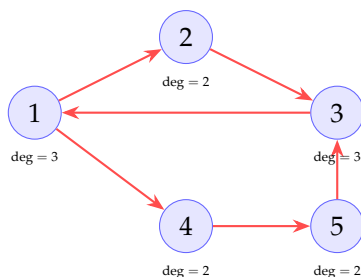
#### 欧拉回路存在条件

连通无向图  $G$  存在欧拉回路  $\iff G$  中所有顶点的度均为偶数。

#### 欧拉通路存在条件

连通无向图  $G$  存在欧拉通路（但非欧拉回路） $\iff G$  中恰有两个奇数度顶点（这两个顶点为通路的起点和终点）。

**【例题 6.2】** 哥尼斯堡七桥：四个区域的度分别为 3, 3, 3, 5（全为奇数），因此不存在欧拉通路。



欧拉通路示例：度序列  $(3, 2, 3, 2, 2)$ ，恰有两个奇数度顶点 1 和 3

欧拉通路： $1 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 4 \rightarrow 5 \rightarrow 3$

### 6.5.1 Fleury 算法

#### Fleury 算法求欧拉回路

1. 检查图是否满足欧拉条件（所有度为偶数 + 连通）。
2. 从合适起点开始（欧拉通路从奇数度顶点开始）。
3. 每步选择一条边，除非别无选择，否则不选桥（割边）。
4. 删除已走过的边，继续在剩余子图中走，直到所有边都被走过。

## §6.6 哈密顿图

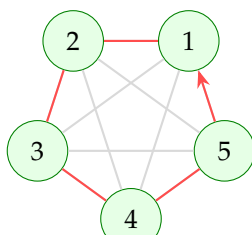
- 哈密顿通路：经过图中每个顶点恰好一次的通路。
- 哈密顿回路：经过图中每个顶点恰好一次的回路。
- 哈密顿图：存在哈密顿回路的图。

#### Dirac 定理（1952）

若  $G$  是  $n \geq 3$  个顶点的简单图且  $\delta(G) \geq n/2$ ，则  $G$  是哈密顿图。

#### Ore 定理（1960）

若  $G$  是  $n \geq 3$  个顶点的简单图，且对任意不相邻的  $u, v \in V$  有  $\deg(u) + \deg(v) \geq n$ ，则  $G$  是哈密顿图。



$K_5$  中的哈密顿回路（红色）

**注.** 判断哈密顿性是一个 **NP**-完全问题。Petersen 图（10 个顶点，3-正则）是经典的极小非哈密顿图反例。

## §6.7 平面图

### 6.7.1 欧拉公式

一个图若能在平面上画出，使得任意两条边只在顶点处相交（没有边交叉），则称为平面图。

#### 欧拉公式

设  $G$  是连通的平面图，有  $n$  个顶点、 $m$  条边，且其平面嵌入将平面划分为  $f$  个面（含外部的无限面），则

$$n - m + f = 2$$

**【推论 6.3】** 1. 对  $n \geq 3$  的简单连通平面图： $m \leq 3n - 6$

2. 对  $n \geq 3$  且无三角形的简单连通平面图： $m \leq 2n - 4$

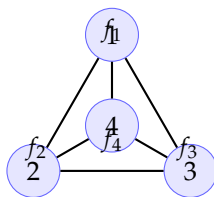
3.  $K_5$  不满足  $m \leq 3n - 6$  ( $10 \not\leq 9$ )，故  $K_5$  非平面图

4.  $K_{3,3}$  不满足  $m \leq 2n - 4$  ( $9 \not\leq 8$ )，故  $K_{3,3}$  非平面图

### 6.7.2 Kuratowski 定理

#### Kuratowski 定理 (1930)

一个图是平面图  $\iff$  它不包含与  $K_5$  或  $K_{3,3}$  同胚的子图（等价地：不以  $K_5$  或  $K_{3,3}$  为子式）。



$K_4$  的平面嵌入： $n = 4$ ,  $m = 6$ ,  $f = 4$ ，验证  $4 - 6 + 4 = 2$

## §6.8 图的着色

### 6.8.1 顶点着色与色数

图  $G$  的一个正常  $k$ -着色是一个函数  $c: V \rightarrow \{1, 2, \dots, k\}$ ，使得相邻顶点的颜色不同。色数  $\chi(G)$  是使  $G$  可正常着色的最小颜色数  $k$ 。

**【例题 6.4】**  $\chi(K_n) = n$  (每个顶点颜色必须不同),  $\chi(\text{二部图}) = 2$ ,  $\chi(C_{2k}) = 2$ ,  $\chi(C_{2k+1}) = 3$ 。

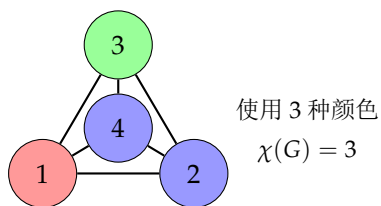
**基本界:**  $\chi(G) \geq \omega(G)$  (团数),  $\chi(G) \leq \Delta(G) + 1$ 。

## 6.8.2 四色定理

### 四色定理

任何平面图的颜色不超过 4。即任何地图用 4 种颜色即可正常着色。

这是第一个主要依靠计算机辅助证明的数学定理 (Appel 和 Haken, 1976)。



## 6.8.3 贪心着色算法

**贪心着色:** 按某个顺序遍历所有顶点, 给每个顶点分配当前已遍历的邻点中未使用的最小颜色编号。  
贪心着色使用的颜色数  $\leq \Delta(G) + 1$ 。

## §6.9 图的编程实现

### 图的邻接矩阵与遍历

```

1 from collections import deque
2
3 class Graph:
4     def __init__(self, n):
5         self.n = n
6         self.adj_matrix = [[0] * n for _ in range(n)]
7         self.adj_list = [[] for _ in range(n)]
8
9     def add_edge(self, u, v):
10        self.adj_matrix[u][v] = self.adj_matrix[v][u] = 1
11        self.adj_list[u].append(v)
12        self.adj_list[v].append(u)
13
14    def dfs(self, start):
15        visited = set()
16        def _dfs(u):
17            visited.add(u)
18            for v in self.adj_list[u]:
19                if v not in visited:
20                    _dfs(v)
21        _dfs(start)
22        return visited
23
24    def bfs(self, start):
25        visited = {start}
26        q = deque([start])
27        while q:
28            u = q.popleft()
29            for v in self.adj_list[u]:
30                if v not in visited:
31                    visited.add(v)
32                    q.append(v)
33        return visited
34
35    def connected_components(self):
36        visited = set()
37        comps = []
38        for v in range(self.n):
39            if v not in visited:
40                comp = self.bfs(v)
41                visited.update(comp)
42                comps.append(comp)
43        return comps
44
45 # 示例
46 g = Graph(5)
47 edges = [(0,1),(0,2),(1,2),(1,3),(2,3),(3,4)]
48 for u, v in edges:
49     g.add_edge(u, v)
50
51 print("邻接矩阵: ")

```

## Fleury 算法求欧拉回路

```

1 def fleury_eulerian_path(adj, n):
2     edge_count = [{ } for _ in range(n)]
3     for u in range(n):
4         for v in adj[u]:
5             edge_count[u][v] = edge_count[u].get(v, 0) + 1
6
7     # 找起点：检查奇数度顶点
8     odd = [v for v in range(n) if len(adj[v]) % 2 == 1]
9     if len(odd) not in (0, 2):
10         return None
11     start = odd[0] if len(odd) == 2 else 0
12
13     path = []
14     stack = [start]
15     while stack:
16         u = stack[-1]
17         # 找下一个未走的邻居
18         next_vertex = None
19         for v in adj[u]:
20             if edge_count[u].get(v, 0) > 0:
21                 next_vertex = v
22                 break
23         if next_vertex is not None:
24             edge_count[u][next_vertex] -= 1
25             edge_count[next_vertex][u] -= 1
26             stack.append(next_vertex)
27         else:
28             path.append(stack.pop())
29     return path[::-1]
30
31 # 测试
32 adj = {0:[1,2], 1:[0,2,3], 2:[0,1,3], 3:[1,2,4], 4:[3]}
33 print(fleury_eulerian_path(adj, 5))

```

## Kosaraju 算法求强连通分量

```
1 def kosaraju(adj, n):
2     # 第一遍 DFS 获得完成顺序
3     visited = [False] * n
4     order = []
5     def dfs1(u):
6         visited[u] = True
7         for v in adj[u]:
8             if not visited[v]:
9                 dfs1(v)
10        order.append(u)
11
12    for u in range(n):
13        if not visited[u]:
14            dfs1(u)
15
16    # 反转图
17    rev_adj = [[] for _ in range(n)]
18    for u in range(n):
19        for v in adj[u]:
20            rev_adj[v].append(u)
21
22    # 第二遍 DFS
23    visited = [False] * n
24    sccs = []
25    def dfs2(u, comp):
26        visited[u] = True
27        comp.append(u)
28        for v in rev_adj[u]:
29            if not visited[v]:
30                dfs2(v, comp)
31
32    for u in reversed(order):
33        if not visited[u]:
34            comp = []
35            dfs2(u, comp)
36            sccs.append(comp)
37    return sccs
38
39 adj = [[1],[2],[0,3],[4],[5],[3]]
40 print(f"强连通分量: {kosaraju(adj, 6)}")
```

§6.10

本章小结

图论基础知识体系

| 主题   | 核心要点                                               |
|------|----------------------------------------------------|
| 基本概念 | 顶点、边、度、握手定理 $\sum \deg(v) = 2 E $                  |
| 图的分类 | 简单图、完全图 $K_n$ 、二部图、正则图、子图                          |
| 图的表示 | 邻接矩阵（对称、 $A^k$ 含义）、邻接表、关联矩阵                        |
| 连通性  | 连通分量、强/弱连通、割点与桥、 $\kappa \leq \lambda \leq \delta$ |
| 欧拉图  | 全部偶数度 $\iff$ 欧拉回路;Fleury 算法                        |
| 哈密顿图 | Dirac/Ore 充分条件;NP-完全问题                             |
| 平面图  | 欧拉公式 $n - m + f = 2$ ;Kuratowski 定理                |
| 着色   | 色数 $\chi(G)$ 、四色定理、贪心着色                            |





树是无向图中最重要的一类——它是最“薄”的连通图，也是图论中应用最广泛的结构之一。任何连通无回路的无向图都称为树。

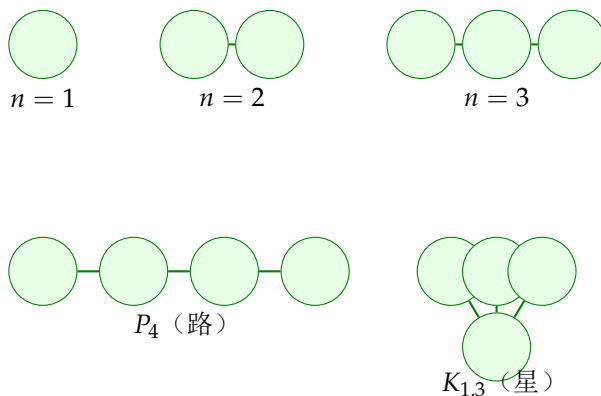
### §7.1 树的基本概念

#### 7.1.1 树的定义

一个无回路的连通无向图称为树，通常记作  $T$ 。

树 = 连通 + 无回路

- 树的边数恰好比顶点数少 1。
- 树是极小连通图：删去任意一条边，图就不再连通。
- 树是极大无回路图：添加任意一条边，图就产生回路。



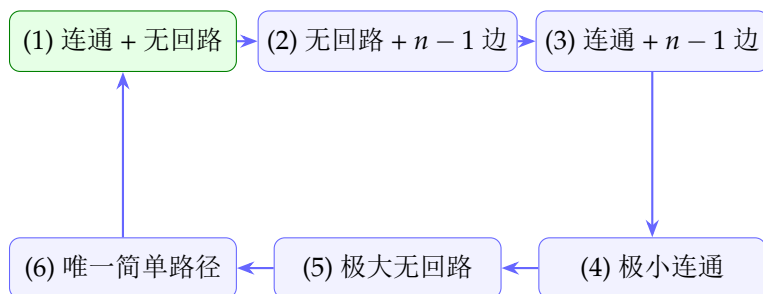
#### 7.1.2 树的六个等价条件

##### 树的等价刻画

设  $T$  是  $n$  阶无向简单图 ( $n \geq 1$ )，以下命题等价：

1.  $T$  连通且无回路（树的原始定义）
2.  $T$  无回路且有  $n-1$  条边
3.  $T$  连通且有  $n-1$  条边

4.  $T$  连通, 但删去任一边后不再连通 (极小连通图)
5.  $T$  无回路, 但添加任一边后恰有一个回路 (极大无回路图)
6.  $T$  中任意两顶点间有唯一简单路径



**证明.** 我们证明  $(1) \Rightarrow (2) \Rightarrow (3) \Rightarrow (4) \Rightarrow (5) \Rightarrow (6) \Rightarrow (1)$ 。

**(1) $\Rightarrow$ (2):** 对  $n$  归纳。 $n = 1$  时边数为  $0 = 1 - 1$ 。假设对少于  $n$  个顶点的树成立。在  $T$  中任取一边  $e$ , 删去  $e$  后图分为两个连通分支  $T_1, T_2$ , 分别有  $n_1, n_2$  个顶点 ( $n_1 + n_2 = n$ )。由归纳假设, 边数分别为  $n_1 - 1, n_2 - 1$ , 故总边数  $m = (n_1 - 1) + (n_2 - 1) + 1 = n - 1$ 。

**(2) $\Rightarrow$ (3):** 设  $T$  有  $c$  个连通分支, 每个分支是树, 故边数为  $n_i - 1$ 。总边数  $m = \sum (n_i - 1) = n - c$ 。已知  $m = n - 1$ , 得  $c = 1$ , 即  $T$  连通。

**(3) $\Rightarrow$ (4):**  $T$  连通且有  $n - 1$  条边。任删一边  $e$ , 得  $T - e$  (仍有  $n$  个顶点,  $n - 2$  条边)。由连通图的边数下界,  $T - e$  不可能连通, 故  $T$  极小连通。

**(4) $\Rightarrow$ (5):** 先证无回路: 若  $T$  有回路  $C$ , 删去  $C$  上任一边  $e$ ,  $T - e$  仍连通, 矛盾。再证极大无回路: 任加一边  $e = (u, v)$ , 由于  $T$  连通,  $u$  和  $v$  间已有路径  $P$ ,  $P \cup \{e\}$  形成回路。

**(5) $\Rightarrow$ (6):** 先证连通性: 若  $T$  不连通, 则存在分属不同分支的顶点  $u, v$ , 添加边  $(u, v)$  不会产生回路, 矛盾。再证唯一性:  $T$  连通保证了任意两点间至少有一条路径。若有两条不同路径, 则它们的对称差含有回路, 与无回路矛盾。

**(6) $\Rightarrow$ (1):** 连通性显然。无回路: 若存在回路  $C$ , 则  $C$  上任意相邻两顶点间的边和回路其余部分给出两条不同路径, 矛盾。  $\square$

### 7.1.3 树叶与森林

- 树中度数为 1 的顶点称为**树叶**。度数为 0 的孤立顶点也视为树叶 (在平凡树中)。
- 度  $\geq 2$  的顶点称为**分支结点** (内结点)。
- 一个无回路 (但可以不通) 的无向图称为**森林**。森林的每个连通分支都是树。
- 若森林有  $n$  个顶点、 $c$  个连通分支, 则边数  $m = n - c$ 。

#### 树叶数量

任何非平凡的树 ( $n \geq 2$ ) 至少有 2 片树叶。

**证明.** 设树  $T$  有  $n$  个顶点,  $m = n - 1$  条边。设树叶数为  $k$ , 非树叶数为  $n - k$ 。由握手定理:

$$2m = \sum_{v \in V} \deg(v) \geq k \cdot 1 + (n - k) \cdot 2 = 2n - k$$

代入  $m = n - 1$  得:  $2(n - 1) \geq 2n - k$ , 即  $2n - 2 \geq 2n - k$ , 故  $k \geq 2$ 。  $\square$

§7.2  
生成树

## 7.2.1 定义与存在性

设  $G$  是连通无向图。 $G$  的一个**生成树**是  $G$  的一个生成子图  $T$ ，且  $T$  是树。等价地说，从  $G$  中删去一些边，使得剩下的图连通且无回路。

## 存在性

每个连通无向图至少有一棵生成树。

构造方法：**破圈法**（反复找回路并删去一条边）或**避圈法**（从空边集开始，每次选取一条不产生回路的新边加入，直到选满  $n - 1$  条边）。

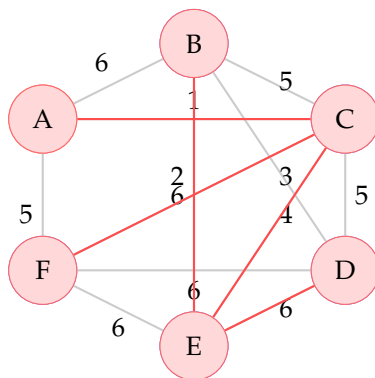
## 7.2.2 最小生成树 (MST)

给定连通带权无向图  $G = (V, E, w)$ ， $G$  的**最小生成树**是  $G$  的一棵生成树  $T$ ，使得总权值  $w(T) = \sum_{e \in E(T)} w(e)$  在所有生成树中最小。

## Prim 算法

Prim 算法 ( $O(n^2)$  朴素实现)

1. 任选起点  $v_0 \in V$ ，设  $S = \{v_0\}$ ， $T = \emptyset$ 。
2. 重复  $n - 1$  次：在所有跨  $S$  和  $V \setminus S$  的边中，选权值最小的边  $e = (u, v)$  ( $u \in S, v \in V \setminus S$ )，将  $e$  加入  $T$ ，将  $v$  加入  $S$ 。
3. 返回  $T$ 。



Prim 算法示例：从 A 开始，红色边构成 MST，总权值  $= 1 + 2 + 4 + 6 + 3 = 16$

## Kruskal 算法

Kruskal 算法 ( $O(m \log m)$ )

1. 将所有边按权值从小到大排序。
2. 初始化并查集：每个顶点自成一个集合。
3. 按排序顺序遍历每条边  $e = (u, v)$ ：若  $u$  和  $v$  不在同一集合，则选取  $e$ ，合并  $u$  和  $v$  所在的集合。
4. 当选取了  $n - 1$  条边时停止。

## 7.2.3 Cayley 定理

## Cayley 定理

$n$  个标号顶点的完全图  $K_n$  中，不同生成树的个数为：

$$T(n) = n^{n-2}$$

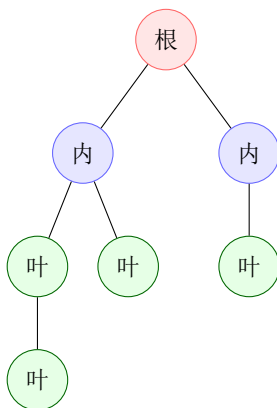
例如  $n = 3$ :  $3^1 = 3$ ;  $n = 4$ :  $4^2 = 16$ 。此定理可通过 Prüfer 编码证明：Prüfer 序列与标号树一一对应，而 Prüfer 序列的每一位可取 1 到  $n$  的任意值，长度为  $n - 2$ ，故有  $n^{n-2}$  种。

### §7.3 有根树与二叉树

## 7.3.1 有根树

选定树的一个顶点作为**根**，并规定所有边的方向为远离根的方向，得到的向树称为**有根树**。

- 根画在最上方，边朝下。
- 顶点  $v$  的**深度**是从根到  $v$  的唯一路径长度（边数）。
- 树的**高度**是最大深度。
- 父结点、子结点、兄弟、祖先、后代、子树——参见图例。



有根树结构：根（红色）、内结点（蓝色）、树叶（绿色）

7.3.2  $m$  叉树

有根树中，每个顶点最多有  $m$  个子结点的称为  $m$  叉树。

- 正则  $m$  叉树：每个分支结点恰好有  $m$  个子结点。
- 完全  $m$  叉树：树叶都在同一层，且每个内结点恰有  $m$  个子结点。

正则  $m$  叉树的树叶与分支结点关系

设  $i$  为分支结点数， $l$  为树叶数，则

$$l = (m - 1)i + 1$$

7.3.3 二叉树

一棵二叉树是有限个顶点的集合，它或者为空，或者由一个根顶点和两棵互不相交的、分别称为左子树和右子树的二叉树组成。

注. 二叉树不是普通的 2 叉树。在二叉树中，子树有左右之分，即使一个子树为空。在普通的有根树中，子结点是无序的。

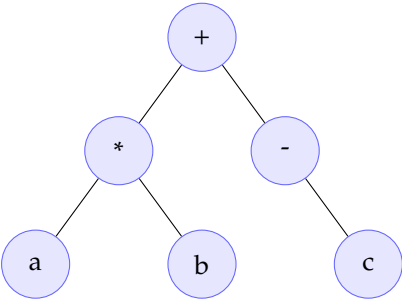
二叉树的基本性质：

- 1. 二叉树第  $i$  层 ( $i \geq 1$ ) 最多有  $2^{i-1}$  个结点。
- 2. 深度为  $k$  的二叉树最多有  $2^k - 1$  个结点。
- 3. 对任意二叉树，若度为 2 的结点个数为  $n_2$ ，树叶个数为  $n_0$ ，则

$n_0 = n_2 + 1$

证明. 设有  $n$  个结点， $m$  条边， $n_1$  个度为 1 的结点。则  $n = n_0 + n_1 + n_2$ 。所有结点度数之和等于边数的 2 倍： $n_1 + 2n_2 = 2m$ 。在树中  $m = n - 1$ ，代入得  $n_1 + 2n_2 = 2(n_0 + n_1 + n_2 - 1)$ 。化简得  $2n_0 = n_1 + 2$ ，又  $n_1 + 2n_2 = 2(n - 1)$ ，可以推出  $n_0 = n_2 + 1$ 。□

7.3.4 二叉树的遍历



| 遍历方式       | 次序        | 结果                     |
|------------|-----------|------------------------|
| 前序遍历 (DLR) | 根 → 左 → 右 | $+ * a b - c$ (波兰表示法)  |
| 中序遍历 (LDR) | 左 → 根 → 右 | $a * b + - c$ 需加括号     |
| 后序遍历 (LRD) | 左 → 右 → 根 | $a b * c - +$ (逆波兰表示法) |

## 遍历序列重建二叉树

- 前序 + 中序可以唯一确定一棵二叉树。
- 后序 + 中序可以唯一确定一棵二叉树。
- 仅有前序和后序无法唯一确定二叉树。

## §7.4

## 哈夫曼树与哈夫曼编码

## 7.4.1 哈夫曼树（最优二叉树）

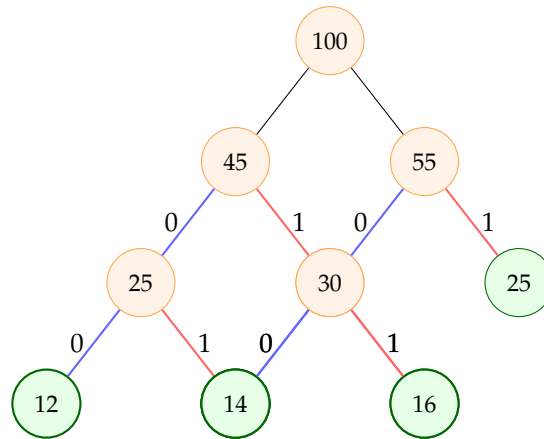
设二叉树有  $n$  个带权树叶，权值分别为  $w_1, w_2, \dots, w_n$ ，树叶的路径长度（从根到树叶的边数）为  $l_1, l_2, \dots, l_n$ 。则二叉树的带权路径长度 (WPL) 定义为：

$$\text{WPL} = \sum_{i=1}^n w_i \cdot l_i$$

给定一组权值，使得 WPL 最小的二叉树称为**最优二叉树**或**哈夫曼树**。

哈夫曼算法 ( $O(n \log n)$ )

1. 构造  $n$  棵单结点二叉树（每个结点携带一个权值），放入集合  $F$ 。
2. 重复  $n - 1$  次：
  - (1). 从  $F$  中取出两棵根权值最小的树作为左右子树。
  - (2). 构造一棵新二叉树，新树的根权值为左右子树根权值之和。
  - (3). 将新树加入  $F$ 。
3.  $F$  中只剩一棵树，即为哈夫曼树。



哈夫曼树示例（权值 9, 11, 12, 13, 14, 16, 25）：左分支标记 0，右分支标记 1

### 7.4.2 哈夫曼编码

一个编码集合，其中没有任何一个编码是另一个编码的前缀，称为**前缀码**。

构造方法：

1. 以字符频率为权值构建哈夫曼树。
2. 左分支标记为 0，右分支标记为 1。
3. 每个字符的编码为从根到该字符树叶的路径标记序列。

哈夫曼编码是**最优前缀码**——对于给定的字符频率，其平均码长最短。



## §7.5 编程实现

### 哈夫曼编码

```
1 import heapq
2
3 class HuffNode:
4     def __init__(self, ch, freq):
5         self.ch = ch
6         self.freq = freq
7         self.left = self.right = None
8     def __lt__(self, other):
9         return self.freq < other.freq
10
11 def build_huffman(freq_map):
12     heap = [HuffNode(ch, f) for ch, f in freq_map.items()]
13     heapq.heapify(heap)
14     while len(heap) > 1:
15         left = heapq.heappop(heap)
16         right = heapq.heappop(heap)
17         parent = HuffNode('', left.freq + right.freq)
18         parent.left, parent.right = left, right
19         heapq.heappush(heap, parent)
20     return heap[0]
21
22 def encode(root, code='', code_map=None):
23     if code_map is None:
24         code_map = {}
25     if root:
26         if not root.left and not root.right:
27             code_map[root.ch] = code
28             encode(root.left, code + '0', code_map)
29             encode(root.right, code + '1', code_map)
30     return code_map
31
32 freq = {'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45}
33 root = build_huffman(freq)
34 codes = encode(root)
35 for ch, code in codes.items():
36     print(f"{ch}: {code}")
```

## Prim 与 Kruskal 算法

```

1  # Prim 算法
2  def prim(graph, n):
3      INF = float('inf')
4      dist = [INF] * n
5      visited = [False] * n
6      dist[0] = 0
7      total = 0
8      for _ in range(n):
9          u = min((j for j in range(n) if not visited[j]),
10                 key=lambda j: dist[j], default=-1)
11          if u == -1:
12              break
13          visited[u] = True
14          total += dist[u]
15          for v in range(n):
16              if not visited[v] and graph[u][v] < dist[v]:
17                  dist[v] = graph[u][v]
18      return total
19
20 # Kruskal 算法
21 class DSU:
22     def __init__(self, n):
23         self.p = list(range(n))
24         self.r = [0] * n
25     def find(self, x):
26         if self.p[x] != x:
27             self.p[x] = self.find(self.p[x])
28         return self.p[x]
29     def union(self, x, y):
30         x, y = self.find(x), self.find(y)
31         if x == y: return
32         if self.r[x] < self.r[y]:
33             x, y = y, x
34         self.p[y] = x
35         if self.r[x] == self.r[y]:
36             self.r[x] += 1
37
38 def kruskal(edges, n):
39     edges.sort(key=lambda e: e[2])
40     dsu = DSU(n)
41     total = 0
42     for u, v, w in edges:
43         if dsu.find(u) != dsu.find(v):
44             dsu.union(u, v)
45             total += w
46     return total

```

## 二叉树遍历

```
1 from collections import deque
2
3 class TreeNode:
4     def __init__(self, val, left=None, right=None):
5         self.val = val
6         self.left = left
7         self.right = right
8
9 def preorder(root):
10     return [root.val] + preorder(root.left) + preorder(root.right) if root
11     else []
12
13 def inorder(root):
14     return inorder(root.left) + [root.val] + inorder(root.right) if root else
15     []
16
17 def postorder(root):
18     return postorder(root.left) + postorder(root.right) + [root.val] if root
19     else []
20
21 def level_order(root):
22     if not root:
23         return []
24     res, q = [], deque([root])
25     while q:
26         node = q.popleft()
27         res.append(node.val)
28         if node.left: q.append(node.left)
29         if node.right: q.append(node.right)
30     return res
31
32 # 示例
33 root = TreeNode('+',
34                 TreeNode('*', TreeNode('a'), TreeNode('b')),
35                 TreeNode('-', None, TreeNode('c')))
36 print(f"前序: {' '.join(preorder(root))}")
37 print(f"中序: {' '.join(inorder(root))}")
38 print(f"后序: {' '.join(postorder(root))}")
```

§7.6

本章小结

树的核心知识体系

| 概念        | 核心要点                                          |
|-----------|-----------------------------------------------|
| 树         | $n$ 顶点, $n - 1$ 边的连通无回路图。六个等价条件。              |
| 树叶        | 非平凡树至少 2 片树叶。                                 |
| 生成树       | 连通图的极小连通生成子图。                                 |
| MST       | Prim $O(n^2)$ 稠密图, Kruskal $O(m \log m)$ 稀疏图。 |
| Cayley 定理 | $n$ 标号顶点的不同树有 $n^{n-2}$ 个。                    |
| 二叉树性质     | $n_0 = n_2 + 1$ , 第 $i$ 层最多 $2^{i-1}$ 个结点。    |
| 遍历        | 前序/中序/后序 (DLR/LDR/LRD)。前中或后中可重建二叉树。           |
| 哈夫曼树      | 带权路径长度最小的二叉树, $O(n \log n)$ 构建。               |
| 哈夫曼编码     | 最优前缀码, 平均码长最短。                                |



代数结构是数学抽象化的核心——  
将具体运算（加法、乘法、函数复合等）  
抽象为集合上的二元运算，研究其公理化  
性质。

### §8.1 代数系统的基本概念

#### 8.1.1 二元运算

设  $S$  是一个非空集合。一个二元运算  $*$  是从  $S \times S$  到  $S$  的一个映射：

$$*: S \times S \rightarrow S, (a, b) \mapsto a * b$$

运算的基本性质：

| 性质  | 定义                                                 |
|-----|----------------------------------------------------|
| 封闭性 | $\forall a, b \in S, a * b \in S$ （二元运算定义内含封闭性）    |
| 结合律 | $\forall a, b, c \in S, (a * b) * c = a * (b * c)$ |
| 交换律 | $\forall a, b \in S, a * b = b * a$                |

【例题 8.1】 $(\mathbb{N}, +)$ ：封闭、结合、交换。 $(\mathbb{N}, -)$ ：不封闭。 $M_n(\mathbb{R})$  上矩阵乘法：封闭、结合，但不交换。

#### 8.1.2 特殊元素：幺元、零元与逆元

##### 幺元（单位元）

元素  $e \in S$  称为幺元，若  $\forall a \in S, e * a = a * e = a$ 。如果幺元存在，则幺元唯一。

##### 零元

元素  $\theta \in S$  称为零元，若  $\forall a \in S, \theta * a = a * \theta = \theta$ 。如果零元存在，则零元唯一。

##### 逆元

设  $(S, *)$  有幺元  $e$ 。对于  $a \in S$ ，若存在  $b \in S$  使得  $a * b = b * a = e$ ，则  $b$  称为  $a$  的逆元，记作  $a^{-1}$ 。  
若运算满足结合律且有幺元，则每个元素最多有一个逆元。

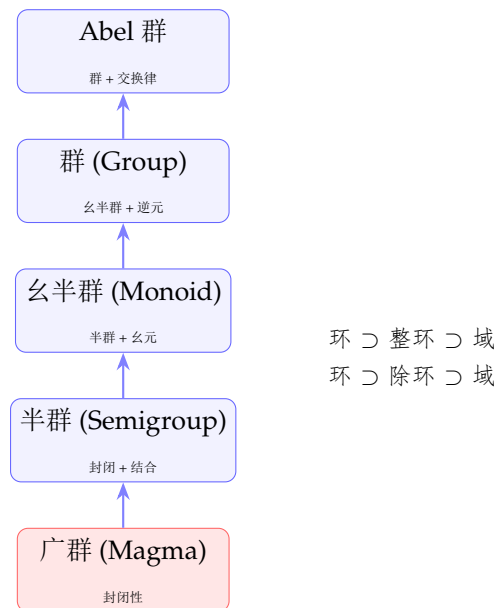
逆元的唯一性。设  $b, c$  都是  $a$  的逆元：

$$b = b * e = b * (a * c) = (b * a) * c = e * c = c$$

□

## 8.1.3 代数结构层次

| 结构             | 条件            | 英文               |
|----------------|---------------|------------------|
| 广群 (Magma)     | 封闭性           | Magma / Groupoid |
| 半群 (Semigroup) | 封闭 + 结合律      | Semigroup        |
| 么半群 (Monoid)   | 半群 + 么元       | Monoid           |
| 群 (Group)      | 么半群 + 每个元素有逆元 | Group            |
| 交换群 (Abel 群)   | 群 + 交换律       | Abelian Group    |

§8.2  
半群与么半群

## 8.2.1 半群

$(S, *)$  是半群当且仅当  $*$  封闭且满足结合律。

常见例子： $(\mathbb{N}, +)$ 、 $(\mathbb{N}, \times)$ 、 $(M_n(\mathbb{R}), \times)$ （矩阵乘法）、 $(\Sigma^+, \circ)$ （字符串拼接）、 $(\{0, 1\}, \wedge)$ （布尔运算）。

## 循环半群

半群中，若存在元素  $a$  使得  $S = \{a^n \mid n \in \mathbb{Z}^+\}$ ，则称  $(S, *)$  为循环半群， $a$  称为生成元。循环半群总是交换的。

## 8.2.2 么半群

$(M, *)$  是么半群当且仅当  $(M, *)$  是半群且存在么元  $e \in M$ 。

常见幺半群：

| 幺半群                         | 运算 | 幺元            |
|-----------------------------|----|---------------|
| $(\mathbb{N}_0, +)$         |    | 0             |
| $(\mathbb{N}, \times)$      |    | 1             |
| $(\Sigma^*, \circ)$         |    | $\varepsilon$ |
| $(\mathcal{P}(X), \cup)$    |    | $\emptyset$   |
| $(M_n(\mathbb{R}), \times)$ |    | $I_n$         |

### 可逆元

幺半群  $M$  中，元素  $a$  若存在  $b$  使得  $a * b = b * a = e$ ，则  $a$  称为可逆元。幺半群中所有可逆元的集合构成一个群。

## §8.3

### 群

#### 8.3.1 群的定义

代数系统  $(G, *)$  称为群，当满足：

1. 封闭性： $\forall a, b \in G, a * b \in G$
2. 结合律： $\forall a, b, c \in G, (a * b) * c = a * (b * c)$
3. 幺元存在： $\exists e \in G, \forall a \in G, e * a = a * e = a$
4. 逆元存在： $\forall a \in G, \exists a^{-1} \in G, a * a^{-1} = a^{-1} * a = e$

若还满足交换律  $a * b = b * a$ ，则称为交换群（Abel 群）。

#### 8.3.2 群的基本性质

##### 消去律

在群  $(G, *)$  中，对任意  $a, b, c \in G$ ：

1. 左消去律：若  $a * b = a * c$ ，则  $b = c$ 。
2. 右消去律：若  $b * a = c * a$ ，则  $b = c$ 。

##### 方程可解性

在群  $(G, *)$  中，对任意  $a, b \in G$ ：

- 方程  $a * x = b$  有唯一解  $x = a^{-1} * b$ 。
- 方程  $y * a = b$  有唯一解  $y = b * a^{-1}$ 。

群中恒有： $(a^{-1})^{-1} = a$ ，且  $(a * b)^{-1} = b^{-1} * a^{-1}$ 。幺元是唯一的幂等元。



## 8.3.3 子群与判定定理

群  $(G, *)$  的非空子集  $H$ , 若  $(H, *)$  也构成群, 则  $H$  称为  $G$  的子群, 记作  $H \leq G$ 。  $\{e\}$  和  $G$  本身称为平凡子群。

## 子群判定定理 I: 一步检验

$G$  的非空子集  $H$  是子群当且仅当:

$$\forall a, b \in H, a * b^{-1} \in H$$

## 子群判定定理 II: 两步检验

$H \leq G$  当且仅当 (1)  $H$  非空; (2)  $\forall a, b \in H, a * b \in H$ ; (3)  $\forall a \in H, a^{-1} \in H$ 。

## 有限子群判定

若  $H$  是  $G$  的有限非空子集且对运算封闭, 则  $H \leq G$ 。

## 8.3.4 循环群

群  $G$  若存在元素  $a \in G$  使得  $G = \{a^k \mid k \in \mathbb{Z}\}$ , 则称  $G$  为循环群,  $a$  称为生成元, 记作  $G = \langle a \rangle$ 。

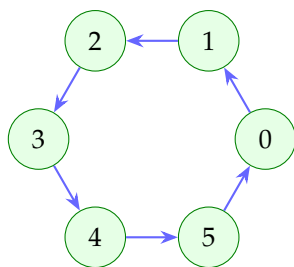
## 元素的阶

群中元素  $a$  的阶  $|a|$  是满足  $a^k = e$  的最小正整数  $k$ 。若不存在这样的  $k$ , 则称  $a$  的阶为无穷大。

性质:

1. 若  $|a| = n$ , 则  $\langle a \rangle = \{e, a, a^2, \dots, a^{n-1}\}$ ,  $|\langle a \rangle| = n$ 。
2.  $a^k = e$  当且仅当  $n \mid k$  ( $n$  为  $a$  的阶)。
3.  $a^k$  的阶为  $\frac{n}{\gcd(n, k)}$ ;  $a^k$  是生成元当且仅当  $\gcd(n, k) = 1$ 。
4. 循环群的子群仍然是循环群。
5. 对  $n$  阶循环群, 对  $n$  的每个正因子  $d$ , 恰有一个  $d$  阶子群。

重要例子:  $(\mathbb{Z}, +)$  (无限循环群, 生成元为 1 和  $-1$ );  $(\mathbb{Z}_n, \oplus_n)$  ( $n$  阶循环群, 生成元有  $\varphi(n)$  个);  $n$  次单位根群 (复数乘法下是循环群)。



$\mathbb{Z}_6$  的 Cayley 图 (生成元 1)

## 8.3.5 陪集与拉格朗日定理

## 左陪集

设  $H \leq G$ ,  $a \in G$ 。则  $aH = \{a * h \mid h \in H\}$  称为  $H$  在  $G$  中的左陪集。

性质:

1.  $|aH| = |H|$  (双射  $h \mapsto ah$ )。
2. 任意两个左陪集要么相等, 要么不相交。
3. 所有左陪集的并等于  $G$ 。因此左陪集构成  $G$  的一个划分。

## 拉格朗日定理

若  $G$  是有限群,  $H \leq G$ , 则

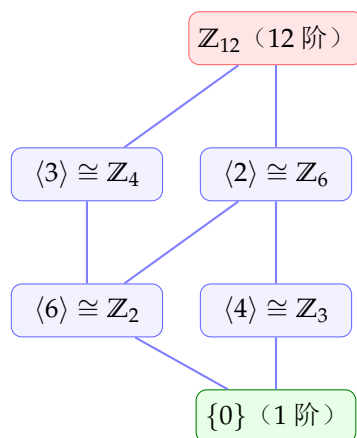
$$|H| \mid |G|$$

且  $[G : H] = |G|/|H|$  称为  $H$  在  $G$  中的指数。

**证明.** 左陪集构成  $G$  的划分, 每个陪集大小等于  $|H|$ 。设共有  $k$  个不同左陪集, 则  $|G| = k \cdot |H|$ , 即  $|H|$  整除  $|G|$ 。□

**【推论 8.2】** 有限群中任意元素  $a$  的阶  $|a|$  整除  $|G|$ 。

**【推论 8.3】** 素数阶群必为循环群 (从而必为 Abel 群)。



$\mathbb{Z}_{12}$  的子群格 (Hasse 图): 每个因子对应一个子群

## 8.3.6 正规子群与商群

子群  $N \leq G$  称为正规子群, 若对任意  $g \in G$ :  $gNg^{-1} = N$  (即  $\forall g \in G, \forall n \in N, gng^{-1} \in N$ )。记作  $N \trianglelefteq G$ 。

- Abel 群的任何子群都是正规子群。
- 对任意同态  $\varphi : G \rightarrow H$ ,  $\ker \varphi \trianglelefteq G$ 。
- 指数为 2 的子群必为正规子群。

**商群**

设  $N \trianglelefteq G$ 。在左陪集的集合  $G/N = \{aN \mid a \in G\}$  上定义运算：

$$(aN) * (bN) = (ab)N$$

则  $(G/N, *)$  构成群，称为  $G$  关于  $N$  的**商群**。若  $G$  有限， $|G/N| = |G|/|N| = [G : N]$ 。

例如  $n\mathbb{Z} \trianglelefteq \mathbb{Z}$ ，商群  $\mathbb{Z}/n\mathbb{Z} \cong \mathbb{Z}_n$ 。

**8.3.7 群同态与同构定理**

设  $(G, *)$  和  $(H, \circ)$  是群。映射  $\varphi : G \rightarrow H$  称为**群同态**，若：

$$\forall a, b \in G, \varphi(a * b) = \varphi(a) \circ \varphi(b)$$

**核与像**

- **核**： $\ker \varphi = \{g \in G \mid \varphi(g) = e_H\}$ ， $\ker \varphi \trianglelefteq G$ 。
- **像**： $\operatorname{im} \varphi = \{\varphi(g) \mid g \in G\}$ ， $\operatorname{im} \varphi \leq H$ 。

$\varphi$  是单射当且仅当  $\ker \varphi = \{e_G\}$ 。若  $\varphi$  是双射，则称  $\varphi$  为**同构**，记作  $G \cong H$ 。

**第一同构定理**

设  $\varphi : G \rightarrow H$  是群同态，则

$$G / \ker \varphi \cong \operatorname{im} \varphi$$

**Cayley 定理**

任意群  $G$  同构于某个对称群的子群。特别地，若  $|G| = n$ ，则  $G$  同构于  $S_n$  的某个子群。

## §8.4 置换群

**8.4.1 置换的表示**

有限集合  $X = \{1, 2, \dots, n\}$  上的**置换**是一个双射  $\sigma : X \rightarrow X$ 。常用双行表示法：

$$\sigma = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 3 & 1 & 5 & 2 & 4 \end{pmatrix}$$

**轮换**

长度为  $k$  的轮换  $(a_1 a_2 \cdots a_k)$  表示置换：

$$a_1 \mapsto a_2, a_2 \mapsto a_3, \dots, a_{k-1} \mapsto a_k, a_k \mapsto a_1$$

其余元素不动。 $k=2$  时为对换。

**轮换分解定理**

每个置换可以唯一（不计轮换次序和循环移位）地分解为不相交轮换的乘积。

**【例题 8.4】** 
$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 3 & 1 & 5 & 6 & 2 & 4 \end{pmatrix} = (1\ 3\ 5\ 2)(4\ 6)$$

每个置换可以表示为对换的乘积。表示不唯一，但对换个数的奇偶性唯一。

**8.4.2 对称群与交错群**

$n$  次对称群  $S_n$ ： $n$  元集合上所有置换关于复合构成的群。 $|S_n| = n!$ 。

- 偶置换：可分解为偶数个对换的乘积。
- 奇置换：可分解为奇数个对换的乘积。
- 交错群  $A_n$ ： $S_n$  中所有偶置换构成的子群。 $|A_n| = \frac{n!}{2}$  ( $n \geq 2$ )。  $A_n \trianglelefteq S_n$ 。

$S_3 \cong D_3$  (6 阶非 Abel 群)； $A_n$  ( $n \geq 5$ ) 是单群（无非平凡正规子群）。

§8.5  
环与域**8.5.1 环的定义**

代数系统  $(R, +, \cdot)$  称为环，若：

1.  $(R, +)$  是 Abel 群（加法幺元记作 0， $a$  的加法逆元记作  $-a$ ）。
2.  $(R, \cdot)$  是半群（乘法满足封闭性和结合律）。
3. 分配律成立： $a \cdot (b + c) = a \cdot b + a \cdot c$ ,  $(a + b) \cdot c = a \cdot c + b \cdot c$ 。

环中的基本计算： $0 \cdot a = a \cdot 0 = 0$ ,  $(-a) \cdot b = a \cdot (-b) = -(a \cdot b)$ ,  $(-a) \cdot (-b) = a \cdot b$ 。

## 8.5.2 环的类型

| 环的类型   | 额外条件                                    |
|--------|-----------------------------------------|
| 交换环    | 乘法满足交换律                                 |
| 含幺环    | 乘法有幺元 1, 且 $1 \neq 0$                   |
| 无零因子环  | 若 $a \cdot b = 0$ , 则 $a = 0$ 或 $b = 0$ |
| 整环     | 交换含幺环 + 无零因子                            |
| 除环 (体) | 含幺环 + 每个非零元有乘法逆元                        |
| 域      | 交换的除环                                   |

域  $\subset$  整环  $\subset$  交换含幺环  $\subset$  环

域  $\subset$  除环  $\subset$  含幺环

## 零因子

环  $R$  中, 非零元  $a$  称为零因子, 若存在  $b \neq 0$  使得  $a \cdot b = 0$  (或  $b \cdot a = 0$ )。在  $\mathbb{Z}_6$  中,  $\bar{2} \cdot \bar{3} = \bar{0}$ , 故  $\bar{2}$  和  $\bar{3}$  是零因子。

$\mathbb{Z}_n$  是整环  $\iff \mathbb{Z}_n$  是域  $\iff n$  是素数。有限整环必为域。

常见例子:

| 结构      | 例子                                                 |
|---------|----------------------------------------------------|
| 非交换环    | $M_n(\mathbb{R})$ , 四元数 $\mathbb{H}$               |
| 整环 (非域) | $\mathbb{Z}$ , $\mathbb{R}[x]$ (多项式环)              |
| 除环 (非域) | 四元数 $\mathbb{H}$                                   |
| 域       | $\mathbb{Q}, \mathbb{R}, \mathbb{C}, \mathbb{Z}_p$ |

## 8.5.3 子环与理想

环  $R$  的子环  $I$  称为理想, 若  $\forall r \in R, \forall i \in I, r \cdot i \in I$  且  $i \cdot r \in I$  (吸收性)。

例如:  $n\mathbb{Z} = \{nk \mid k \in \mathbb{Z}\}$  是  $\mathbb{Z}$  的理想。多项式环  $\mathbb{R}[x]$  中以 0 为根的多项式集合也是理想。

## 8.5.4 域

代数系统  $(F, +, \cdot)$  称为域, 若:

1.  $(F, +)$  是 Abel 群 (幺元 0)
2.  $(F \setminus \{0\}, \cdot)$  是 Abel 群 (幺元 1)
3. 乘法对加法满足分配律

## 域的特征

域  $F$  的特征  $\text{char}(F)$  是满足  $n \cdot 1 = 0$  的最小正整数  $n$ ; 若无此类  $n$ , 则  $\text{char}(F) = 0$ 。域的特征要么为 0, 要么为素数。

## 8.5.5 有限域

有限域的阶（元素个数）必为  $p^n$ ，其中  $p$  是素数（域的特征）， $n$  是正整数。

对每个素数幂  $q = p^n$ ，存在唯一（同构意义下）的  $q$  元有限域，记作  $\text{GF}(q)$  或  $\mathbb{F}_q$ 。 $\text{GF}(p) \cong \mathbb{Z}_p$ 。 $\text{GF}(p^n)$  的乘法群是  $p^n - 1$  阶循环群。

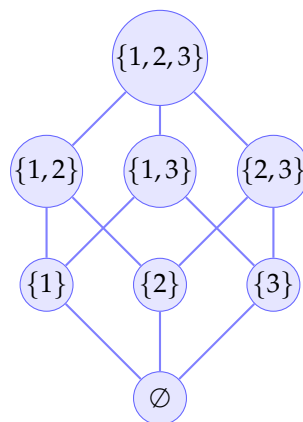
例如构造  $\text{GF}(4) \cong \mathbb{Z}_2[x]/(x^2 + x + 1)$ ，元素为  $\{0, 1, \alpha, \alpha + 1\}$ ，其中  $\alpha^2 + \alpha + 1 = 0$ （在  $\mathbb{Z}_2$  中）。

§8.6  
格

## 8.6.1 格作为偏序集

偏序集  $(L, \preceq)$  称为**格**，若任意两个元素  $a, b \in L$  都有：

- 最小上界（并）  $a \vee b$ ：满足  $a \preceq a \vee b$ ,  $b \preceq a \vee b$ ，且对任意上界  $c$  有  $a \vee b \preceq c$ 。
- 最大下界（交）  $a \wedge b$ ：满足  $a \wedge b \preceq a$ ,  $a \wedge b \preceq b$ ，且对任意下界  $c$  有  $c \preceq a \wedge b$ 。



$(\mathcal{P}(\{1,2,3\}), \subseteq)$  的 Hasse 图：布尔格  $2^3$

## 8.6.2 格作为代数结构

代数系统  $(L, \vee, \wedge)$  称为**格**，若对任意  $a, b, c \in L$ ：

1. 交换律：  $a \vee b = b \vee a$ ,  $a \wedge b = b \wedge a$
2. 结合律：  $(a \vee b) \vee c = a \vee (b \vee c)$ ,  $(a \wedge b) \wedge c = a \wedge (b \wedge c)$
3. 吸收律：  $a \vee (a \wedge b) = a$ ,  $a \wedge (a \vee b) = a$

两种定义等价：通过  $a \preceq b \iff a \wedge b = a \iff a \vee b = b$  建立联系。

## 8.6.3 特殊格

| 类型  | 额外条件                                                   |
|-----|--------------------------------------------------------|
| 有界格 | 存在最大元 1 和最小元 0                                         |
| 分配格 | $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ |
| 有补格 | 有界 + 每个元素有补元 $a'$ ( $a \vee a' = 1, a \wedge a' = 0$ ) |
| 布尔格 | 有补分配格 (即布尔代数)                                          |

**Birkhoff 刻画**

一个格是分配格当且仅当它不包含同构于  $N_5$  (五边形格) 或  $M_3$  (菱形格) 的子格。

§8.7  
编程实现模  $n$  剩余类群

```

1 import math
2
3 class Z_n:
4     def __init__(self, n):
5         self.n = n
6
7     def add(self, a, b):
8         return (a + b) % self.n
9
10    def mul(self, a, b):
11        return (a * b) % self.n
12
13    def inv_add(self, a):
14        return (-a) % self.n
15
16    def inv_mul(self, a):
17        def egcd(a, b):
18            if b == 0: return (a, 1, 0)
19            g, x, y = egcd(b, a % b)
20            return (g, y, x - (a // b) * y)
21        g, x, _ = egcd(a, self.n)
22        if g != 1:
23            raise ValueError(f"{a} 与 {self.n} 不互质")
24        return x % self.n
25
26    def mul_group(self):
27        return {a for a in range(1, self.n)
28                if math.gcd(a, self.n) == 1}
29
30    def is_field(self):
31        if self.n <= 1: return False
32        return all(self.n % i != 0
33                    for i in range(2, int(math.isqrt(self.n)) + 1))
34
35 # 示例
36 z7 = Z_n(7)
37 print(f"Z_7 是域? {z7.is_field()}")
38 print(f"乘法群元素: {z7.mul_group()}") # {1,2,3,4,5,6}
39 print(f"3 的乘法逆元 mod 7: {z7.inv_mul(3)}") # 5

```



## 置换的运算与分解

```

1 import math
2
3 class Permutation:
4     def __init__(self, mapping):
5         self.n = max(mapping.keys(), default=0)
6         self.mapping = {i: mapping.get(i, i)
7                         for i in range(1, self.n + 1)}
8
9     def __call__(self, x):
10        return self.mapping.get(x, x)
11
12    def __mul__(self, other):
13        new_map = {i: self(other(i))
14                  for i in range(1, max(self.n, other.n) + 1)}
15        return Permutation(new_map)
16
17    def inverse(self):
18        inv_map = {v: k for k, v in self.mapping.items()}
19        return Permutation(inv_map)
20
21    def to_cycles(self):
22        visited = set()
23        cycles = []
24        for start in range(1, self.n + 1):
25            if start in visited:
26                continue
27            cycle = []
28            cur = start
29            while cur not in visited:
30                visited.add(cur)
31                cycle.append(cur)
32                cur = self.mapping[cur]
33            if len(cycle) > 1:
34                cycles.append(tuple(cycle))
35        return cycles
36
37    def parity(self):
38        trans = 0
39        for c in self.to_cycles():
40            trans += len(c) - 1
41        return trans % 2 # 0=偶, 1=奇
42
43    def order(self):
44        lengths = [len(c) for c in self.to_cycles()]
45        if not lengths:
46            return 1
47        result = lengths[0]
48        for L in lengths[1:]:
49            result = result * L // math.gcd(result, L)
50        return result
51
52    def __repr__(self):
53        cycles = self.to_cycles()
54        if not cycles:
55            return "id"
56        return "(" + " ".join(str(c) for c in cycles) + ")"

```

## 循环群的子群分析

```

1 import math
2
3 def generators_of_Z_n(n):
4     return [k for k in range(1, n) if math.gcd(k, n) == 1]
5
6 def order_of_element(n, a):
7     if a % n == 0:
8         return 1
9     return n // math.gcd(a, n)
10
11 def subgroups_of_Z_n(n):
12     subgroups = {}
13     for d in range(1, n + 1):
14         if n % d == 0:
15             g = n // d
16             subgroup = sorted((k * g) % n for k in range(d))
17             subgroups[d] = subgroup
18     return subgroups
19
20 def cyclic_group_info(n):
21     print(f" $Z_{\{n\}}$  是  $\{n\}$  阶循环群")
22     gens = generators_of_Z_n(n)
23     print(f" 生成元 ( ( $\{n\}$ )= $\{\text{len}(\text{gens})\}$  个) :  $\{\text{gens}\}$ ")
24     print(" 子群: ")
25     for order, elements in subgroups_of_Z_n(n).items():
26         print(f"       $\{\text{order}\}$  阶:  $\{\text{elements}\}$ ")
27
28 cyclic_group_info(12)
29 #  $Z_{12}$  的子群对应 12 的每个因子

```

§8.8  
本章小结

## 代数结构体系总览

| 结构     | 定义关键条件                | 经典例子                                               |
|--------|-----------------------|----------------------------------------------------|
| 半群     | 封闭 + 结合               | $(\mathbb{N}, +), (\Sigma^+, \circ)$               |
| 么半群    | 半群 + 么元               | $(\mathbb{N}_0, +), (\Sigma^*, \circ)$             |
| 群      | 么半群 + 逆元              | $(\mathbb{Z}, +), S_n, GL_n(\mathbb{R})$           |
| Abel 群 | 群 + 交换律               | $(\mathbb{Z}, +), (\mathbb{Z}_n, \oplus)$          |
| 环      | Abel 加法群 + 乘法半群 + 分配律 | $(\mathbb{Z}, +, \times)$                          |
| 整环     | 交换含么环 + 无零因子          | $\mathbb{Z}, \mathbb{Z}[x]$                        |
| 域      | 交换除环                  | $\mathbb{Q}, \mathbb{R}, \mathbb{C}, \mathbb{Z}_p$ |
| 格      | 任意两元有上下确界             | $(\mathcal{P}(X), \subseteq)$                      |

五大核心定理：

| 定理        | 内容                                          |
|-----------|---------------------------------------------|
| 拉格朗日定理    | 子群阶整除群阶; 元素阶整除群阶                            |
| Cayley 定理 | 每个群同构于某个对称群的子群                              |
| 第一同构定理    | $G / \ker \varphi \cong \text{im } \varphi$ |
| 循环群结构定理   | $n$ 阶循环群对每个 $d \mid n$ 恰有一个 $d$ 阶子群         |
| 有限整环即域    | 有限整环必为域                                     |

## 布尔代数

逻辑推理的代数化——布尔代数用一套简洁的公理描述“真”与“假”的运算规则，是数字电路设计的数学基础。

### §9.1 布尔代数的定义

#### 9.1.1 公理化定义

##### 布尔代数

一个布尔代数是代数系统  $(B, +, \cdot, ', 0, 1)$ ，其中  $B$  是至少包含两个元素 0 和 1 的集合， $+$  和  $\cdot$  是  $B$  上的二元运算， $'$  是  $B$  上的一元运算，满足以下公理。

对任意  $a, b, c \in B$ :

**B1:** 交换律:  $a + b = b + a$

**B1'**  $a \cdot b = b \cdot a$

**B2:** 结合律:  $a + (b + c) = (a + b) + c$

**B2'**  $a \cdot (b \cdot c) = (a \cdot b) \cdot c$

**B3:** 幺元:  $a + 0 = a$

**B3'**  $a \cdot 1 = a$

**B4:** 排中律:  $a + a' = 1$

**B4'** 矛盾律:  $a \cdot a' = 0$

**B5:** 分配律:  $a + (b \cdot c) = (a + b) \cdot (a + c)$

**B5'**  $a \cdot (b + c) = (a \cdot b) + (a \cdot c)$

其中 0 称为零元（下界），1 称为幺元（上界）， $a'$  称为  $a$  的补元。运算  $+$  称为布尔和（OR）， $\cdot$  称为布尔积（AND）。

##### 对偶原理

布尔代数的公理具有完全对称的对偶性——将  $+$  与  $\cdot$  互换、0 与 1 互换，所得命题仍成立。

### 9.1.2 亨廷顿公理体系

数学家 E. V. Huntington 在 1904 年给出了更精简的公理系统：

**H1:**  $a + b = b + a$

**H2:**  $(a + b) + c = a + (b + c)$

**H3:**  $a + b$  由  $a, b$  唯一确定

**H4:** 存在  $0, 1$  使得  $a + 0 = a$ ，且对每个  $a$  存在  $a'$  满足  $a + a' = 1$

亨廷顿还证明：若  $B$  有至少两个元素，这组公理足以推出所有布尔代数定理。

### 9.1.3 作为有补分配格的布尔代数

#### 格论视角的布尔代数

布尔代数是一个有补的分配格。格中有最大元  $1$  和最小元  $0$ ，每个元素  $a$  有补元  $a'$ ： $a \vee a' = 1$  且  $a \wedge a' = 0$ 。在分配格中，补元若存在则唯一。

偏序关系： $a \leq b \iff a + b = b \iff a \cdot b = a$ 。

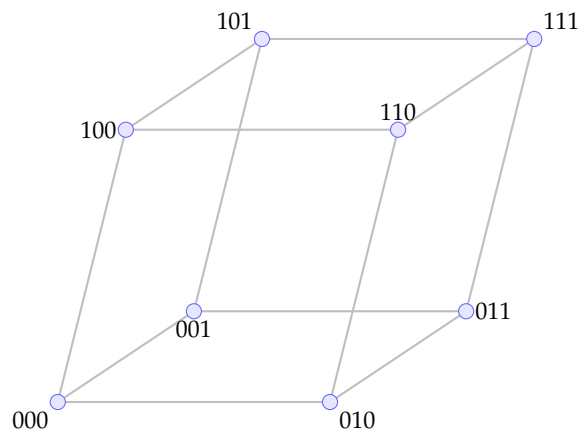


图 9.1: 三元布尔代数的立方体格结构——每个顶点对应一个三元组，顶点间的边表示相邻关系。

### 9.1.4 重要例子

| 布尔代数                | 元素                    | $+/ \vee$ | $\cdot / \wedge$ | $'$    | $0 / 1$         |
|---------------------|-----------------------|-----------|------------------|--------|-----------------|
| 二值布尔代数 $\mathbf{B}$ | $\{0, 1\}$            | OR        | AND              | NOT    | $0 / 1$         |
| 集合代数                | $\mathcal{P}(X)$      | $\cup$    | $\cap$           | 补集     | $\emptyset / X$ |
| 命题逻辑                | $\{F, T\}$            | $\vee$    | $\wedge$         | $\neg$ | $F / T$         |
| 除数格                 | $\{d \mid d \mid n\}$ | lcm       | gcd              | $n/d$  | $1 / n$         |

## § 9.2 布尔代数的基本性质

### 9.2.1 幂等律

$$a + a = a, \quad a \cdot a = a$$

证明. 以  $a + a = a$  为例：

$$\begin{aligned} a &= a + 0 && (\text{B3: 幺元}) \\ &= a + (a \cdot a') && (\text{B4': 矛盾律}) \\ &= (a + a) \cdot (a + a') && (\text{B5: 分配律}) \\ &= (a + a) \cdot 1 && (\text{B4: 排中律}) \\ &= a + a && (\text{B3': 幺元}) \end{aligned}$$

另一式由对偶原理可得。 □

### 9.2.2 零律

$$a + 1 = 1, \quad a \cdot 0 = 0$$

证明. 以  $a + 1 = 1$  为例：

$$\begin{aligned} a + 1 &= (a + 1) \cdot 1 && (\text{B3'}) \\ &= (a + 1) \cdot (a + a') && (\text{B4}) \\ &= a + (1 \cdot a') && (\text{B5: 分配律}) \\ &= a + a' && (\text{B3'}) \\ &= 1 && (\text{B4}) \end{aligned}$$

□

### 9.2.3 吸收律

$$a + (a \cdot b) = a, \quad a \cdot (a + b) = a$$

证明. 以  $a + (a \cdot b) = a$  为例：

$$\begin{aligned} a + (a \cdot b) &= (a \cdot 1) + (a \cdot b) && (\text{B3'}) \\ &= a \cdot (1 + b) && (\text{B5': 分配律}) \\ &= a \cdot 1 && (\text{零律}) \\ &= a && (\text{B3'}) \end{aligned}$$

□

## 9.2.4 补元的唯一性

## 补元唯一性

布尔代数中，每个元素  $a$  的补元是唯一的。

证明. 设  $b$  和  $c$  都是  $a$  的补元，即  $a + b = 1$ ,  $a \cdot b = 0$  且  $a + c = 1$ ,  $a \cdot c = 0$ 。则：

$$\begin{aligned} b &= b \cdot 1 = b \cdot (a + c) = (b \cdot a) + (b \cdot c) \\ &= (a \cdot b) + (b \cdot c) = 0 + (b \cdot c) = (a \cdot c) + (b \cdot c) \\ &= (c \cdot a) + (c \cdot b) = c \cdot (a + b) = c \cdot 1 = c \end{aligned}$$

□

## 9.2.5 德摩根律

$$(a + b)' = a' \cdot b', \quad (a \cdot b)' = a' + b'$$

证明. 以  $(a + b)' = a' \cdot b'$  为例。验证  $a' \cdot b'$  是  $a + b$  的补元：

$$\begin{aligned} (a + b) + (a' \cdot b') &= [(a + b) + a'] \cdot [(a + b) + b'] \\ &= [1 + b] \cdot [a + 1] = 1 \cdot 1 = 1 \\ (a + b) \cdot (a' \cdot b') &= [a \cdot (a' \cdot b')] + [b \cdot (a' \cdot b')] \\ &= [0 \cdot b'] + [0 \cdot a'] = 0 + 0 = 0 \end{aligned}$$

由补元唯一性， $a' \cdot b' = (a + b)'$ 。

□

## 9.2.6 双重否定律

$$(a')' = a$$

证明.  $a'$  的补元满足  $a' + (a')' = 1$  且  $a' \cdot (a')' = 0$ ；同时  $a' + a = 1$  且  $a' \cdot a = 0$ 。由补元唯一性得  $(a')' = a$ 。

□

## 9.2.7 性质总表

| 名称    | OR 形式                                     | AND 形式                                      |
|-------|-------------------------------------------|---------------------------------------------|
| 幂等律   | $a + a = a$                               | $a \cdot a = a$                             |
| 交换律   | $a + b = b + a$                           | $a \cdot b = b \cdot a$                     |
| 结合律   | $a + (b + c) = (a + b) + c$               | $a \cdot (b \cdot c) = (a \cdot b) \cdot c$ |
| 分配律   | $a + (b \cdot c) = (a + b) \cdot (a + c)$ | $a \cdot (b + c) = a \cdot b + a \cdot c$   |
| 同一律   | $a + 0 = a$                               | $a \cdot 1 = a$                             |
| 零律    | $a + 1 = 1$                               | $a \cdot 0 = 0$                             |
| 吸收律   | $a + (a \cdot b) = a$                     | $a \cdot (a + b) = a$                       |
| 德摩根律  | $(a + b)' = a' \cdot b'$                  | $(a \cdot b)' = a' + b'$                    |
| 双重否定律 | $(a')' = a$                               | $(a')' = a$                                 |
| 对合律   | $0' = 1$                                  | $1' = 0$                                    |

§ 9.3

布尔函数

9.3.1 布尔函数的定义

布尔函数

从布尔代数  $B^n$  到  $B$  的一个映射  $f : B^n \rightarrow B$  称为一个  $n$  元布尔函数。在最基本的情形,  $B = \{0,1\}$ 。

布尔表达式

由布尔变量  $x_1, x_2, \dots, x_n$  通过运算  $+, \cdot, '$  和常量  $0, 1$  构成的表达式称为布尔表达式。每个布尔表达式定义一个布尔函数。

例如:  $f(x,y,z) = xy + x'z + yz'$  定义了一个三元布尔函数。

布尔函数的数量

$n$  个变量的布尔函数共有  $2^{2^n}$  个。

证明.  $n$  个布尔变量有  $2^n$  种输入组合。对每种组合, 输出可取 0 或 1。因此共有  $2^{2^n}$  种输出模式。□

| $n$ | 输入组合数 $2^n$ | 布尔函数个数 $2^{2^n}$ | 示例                 |
|-----|-------------|------------------|--------------------|
| 1   | 2           | 4                | NOT, 恒等, 恒 0, 恒 1  |
| 2   | 4           | 16               | AND, OR, XOR, NAND |
| 3   | 8           | 256              | 多数表决、全加器           |
| 4   | 16          | 65536            | 各种组合电路             |

9.3.2 极小项与极大项

极小项与极大项

- 极小项:  $n$  个变量的一个布尔积, 每个变量 (或其补) 恰好出现一次, 如  $xy'z$ 。
- 极大项:  $n$  个变量的一个布尔和, 每个变量 (或其补) 恰好出现一次, 如  $x + y' + z$ 。

每个极小项在恰好一种赋值下为 1, 每个极大项在恰好一种赋值下为 0。

标准形定理

每个非零布尔函数可唯一表示为极小项之和 (SOP); 每个非 1 的布尔函数可唯一表示为极大项之积 (POS)。



## §9.4 布尔表达式的化简

### 9.4.1 代数化简法

常用化简技巧：

1. 合并： $ab + ab' = a(b + b') = a \cdot 1 = a$
2. 吸收： $a + ab = a(1 + b) = a$
3. 消去： $a + a'b = a + b$ （因  $(a + a')(a + b) = 1 \cdot (a + b) = a + b$ ）
4. 一致项： $ab + a'c + bc = ab + a'c$ （一致项  $bc$  可消去）

【例题 9.1】化简  $F = AB + A'C + BC$ ：

$$\begin{aligned}
 F &= AB + A'C + BC \cdot 1 \\
 &= AB + A'C + BC(A + A') \\
 &= AB + A'C + ABC + A'BC \\
 &= AB(1 + C) + A'C(1 + B) \\
 &= AB + A'C
 \end{aligned}$$

### 9.4.2 卡诺图

卡诺图是 Maurice Karnaugh 于 1953 年提出的可视化化简工具。

#### 2 变量卡诺图

|         |         |         |
|---------|---------|---------|
|         | $y = 0$ | $y = 1$ |
| $x = 0$ | $x'y'$  | $x'y$   |
| $x = 1$ | $xy'$   | $xy$    |

图 9.2: 2 变量卡诺图——相邻方格仅一位变量不同。

#### 3 变量卡诺图

|           |          |         |
|-----------|----------|---------|
|           | $z = 0$  | $z = 1$ |
| $xy = 00$ | $x'y'z'$ | $x'y'z$ |
| $xy = 01$ | $x'yz'$  | $x'yz$  |
| $xy = 11$ | $xyz'$   | $xyz$   |
| $xy = 10$ | $xy'z'$  | $xy'z$  |

图 9.3: 3 变量卡诺图——采用格雷码排列  $00 \rightarrow 01 \rightarrow 11 \rightarrow 10$ 。

## 4 变量卡诺图

|           | $yz = 00$ | $yz = 01$ | $yz = 11$ | $yz = 10$ |
|-----------|-----------|-----------|-----------|-----------|
| $wx = 00$ | $m_0$     | $m_1$     | $m_3$     | $m_2$     |
| $wx = 01$ | $m_4$     | $m_5$     | $m_7$     | $m_6$     |
| $wx = 11$ | $m_{12}$  | $m_{13}$  | $m_{15}$  | $m_{14}$  |
| $wx = 10$ | $m_8$     | $m_9$     | $m_{11}$  | $m_{10}$  |

图 9.4: 4 变量卡诺图—— $m_k$  表示第  $k$  个极小项。

卡诺图化简步骤:

1. 将函数填入卡诺图（1 的格子填 1，0 的填 0）
2. 用尽可能大的矩形圈住所有的 1，每个矩形大小为  $2^k$  个格子
3. 矩形可跨越边界（循环相邻）；圈数尽量少
4. 每个圈对应一项——变量取值不变的保留，发生变化的消去

无关项

实际电路设计中，某些输入组合永远不会出现，或输出不影响结果。这些情况用  $\times$  或  $d$  标记，称为无关项。化简时，无关项可当做 0 或 1，怎么有利于化简就怎么取。

## 9.4.3 Quine-McCluskey 算法

当  $n \geq 5$  时，卡诺图不再适用。Q-M 算法是系统化的表格化简方法。

算法核心思想:

1. 列出所有极小项的二进制形式
2. 按含 1 的个数分组
3. 在相邻组间寻找海明距离为 1 的项合并，差异位标记为 “-”
4. 重复直到无法继续合并
5. 构建质蕴含表，选取覆盖所有极小项的最少质蕴含项

复杂度：最坏情况指数级（NP-hard 问题）。

## §9.5 逻辑门电路

### 9.5.1 基本逻辑门

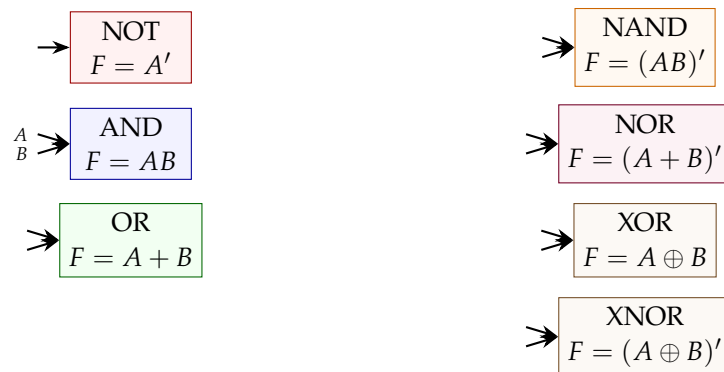


图 9.5: 七种基本逻辑门符号及其布尔表达式。

### 9.5.2 真值表

| A | B | AND | OR | NAND | NOR | XOR | XNOR |
|---|---|-----|----|------|-----|-----|------|
| 0 | 0 | 0   | 0  | 1    | 1   | 0   | 1    |
| 0 | 1 | 0   | 1  | 1    | 0   | 1   | 0    |
| 1 | 0 | 0   | 1  | 1    | 0   | 1   | 0    |
| 1 | 1 | 1   | 1  | 0    | 0   | 0   | 1    |

表 9.1: 基本逻辑门的真值表

### 9.5.3 功能完备性

#### 功能完备集

一组逻辑门称为**功能完备**的，如果仅用这组门就能实现任意布尔函数。

#### NAND 完备性

{NAND} 单独就是功能完备的。用 NAND 可实现：

$$\text{NOT } A = A \text{ NAND } A, \quad A \text{ AND } B = (A \text{ NAND } B) \text{ NAND } (A \text{ NAND } B)$$

$$A \text{ OR } B = (A \text{ NAND } A) \text{ NAND } (B \text{ NAND } B)$$

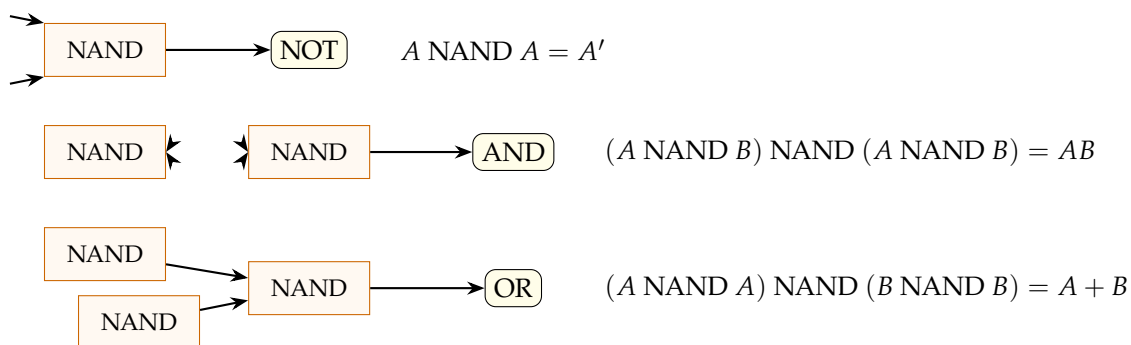


图 9.6: 用 NAND 门实现 NOT、AND、OR: NAND 门单独构成功能完备集。

### 9.5.4 加法器电路

半加器:  $S = A \oplus B$ ,  $C = A \cdot B$ 。

全加器:

$$S = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = AB + AC_{\text{in}} + BC_{\text{in}}$$

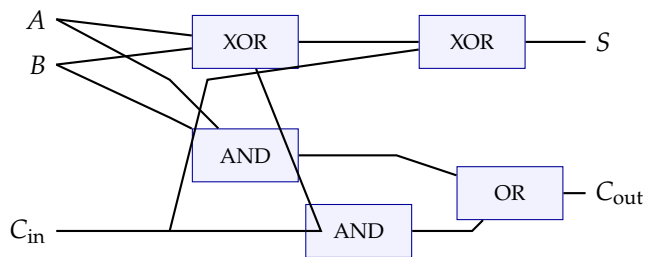


图 9.7: 全加器电路图——由 2 个 XOR 门、2 个 AND 门和 1 个 OR 门构成。

多个全加器级联可实现多位二进制加法器。

## §9.6 编程实现

### 布尔函数真值表生成

```
1 from itertools import product
2 from typing import Callable, List
3
4 def truth_table(variables: List[str],
5                 func: Callable[..., int]) -> str:
6     """生成布尔函数的真值表"""
7     n = len(variables)
8     header = " | ".join(variables) + " | F"
9     lines = [header, "-" * len(header)]
10    for values in product([0, 1], repeat=n):
11        result = func(*values)
12        row = " | ".join(str(v) for v in values) + f" | {result}"
13        lines.append(row)
14    return "\n".join(lines)
15
16 def majority(a: int, b: int, c: int) -> int:
17     """多数表决器:  $F = AB + AC + BC$ """
18     return (a & b) | (a & c) | (b & c)
19
20 print(truth_table(["A", "B", "C"], majority))
```

## Quine-McCluskey 算法

```

1 from typing import List, Set, Dict
2
3 def minterm_to_binary(m: int, n_vars: int) -> str:
4     return format(m, f'0{n_vars}b')
5
6 def hamming_distance(s1: str, s2: str) -> int:
7     return sum(c1 != c2 for c1, c2 in zip(s1, s2))
8
9 def combine_terms(t1: str, t2: str) -> str:
10    result = list(t1)
11    for i, (c1, c2) in enumerate(zip(t1, t2)):
12        if c1 != c2:
13            result[i] = '-'
14    return ''.join(result)
15
16 def qm_algorithm(minterms: List[int], n_vars: int) -> Set[str]:
17    implicants = set(minterm_to_binary(m, n_vars) for m in minterms)
18    while True:
19        groups: Dict[int, List[str]] = {}
20        for imp in implicants:
21            ones = imp.count('1')
22            groups.setdefault(ones, []).append(imp)
23        new_implicants, used = set(), set()
24        sorted_keys = sorted(groups.keys())
25        for i in range(len(sorted_keys) - 1):
26            k1, k2 = sorted_keys[i], sorted_keys[i + 1]
27            if k1 + 1 != k2: continue
28            for t1 in groups[k1]:
29                for t2 in groups[k2]:
30                    if hamming_distance(t1, t2) == 1:
31                        combined = combine_terms(t1, t2)
32                        new_implicants.add(combined)
33                        used.add(t1); used.add(t2)
34        prime = implicants - used
35        if not new_implicants:
36            return implicants
37        implicants = new_implicants | prime
38
39 def term_to_expression(term: str, variables: List[str]) -> str:
40    parts = []
41    for i, (ch, var) in enumerate(zip(term, variables)):
42        if ch == '1': parts.append(var)
43        elif ch == '0': parts.append(f"{var}'")
44    return ''.join(parts) if parts else '1'
45
46 # 示例
47 minterms = [0,1,2,4,5,6,8,9,12,13,14]
48 primes = qm_algorithm(minterms, 4)
49 for p in primes:
50     print(f" {p} -> {term_to_expression(p, ['w','x','y','z'])}")

```

## 逻辑电路仿真器

```

1 from itertools import product
2 from typing import Dict, List, Tuple, Callable
3
4 class LogicSim:
5     """组合逻辑电路仿真器"""
6     def __init__(self):
7         self.gates: List[Tuple[str, List[str], str, Callable]] = []
8         self.signals: Dict[str, int] = {}
9
10    def set_input(self, name: str, value: int):
11        self.signals[name] = value
12
13    def add_not(self, input_name: str, output_name: str):
14        self.gates.append(("NOT", [input_name], output_name, lambda a: 1 - a))
15
16    def add_and(self, in1: str, in2: str, output_name: str):
17        self.gates.append(("AND", [in1, in2], output_name, lambda a, b: a & b)
18        )
19
20    def add_or(self, in1: str, in2: str, output_name: str):
21        self.gates.append(("OR", [in1, in2], output_name, lambda a, b: a | b))
22
23    def add_xor(self, in1: str, in2: str, output_name: str):
24        self.gates.append(("XOR", [in1, in2], output_name, lambda a, b: a ^ b)
25        )
26
27    def simulate(self) -> Dict[str, int]:
28        for _, inputs, output, func in self.gates:
29            args = [self.signals[name] for name in inputs]
30            self.signals[output] = func(*args)
31        return dict(self.signals)
32
33    def truth_table(self, input_names: List[str],
34                    output_name: str) -> str:
35        results = []
36        header = " | ".join(input_names) + f" | {output_name}"
37        results.append(header)
38        results.append("-" * len(header))
39        for values in product([0, 1], repeat=len(input_names)):
40            self.signals.clear()
41            for name, val in zip(input_names, values):
42                self.signals[name] = val
43            self.simulate()
44            row = " | ".join(str(v) for v in values)
45            row += f" | {self.signals[output_name]}"
46            results.append(row)
47        return "\n".join(results)
48
49 sim = LogicSim()
50 sim.add_xor("A", "B", "S")
51 sim.add_and("A", "B", "C")
52
53 print("半加器真值表: ")
54 print(sim.truth_table(["A", "B"], "S"))

```

本章小结

| 概念     | 核心要点                                                       |
|--------|------------------------------------------------------------|
| 布尔代数   | 公理化代数系统 $(B, +, \cdot, ', 0, 1)$ ，也是有补分配格                  |
| 对偶原理   | 交换 $+ \leftrightarrow \cdot$ 、 $0 \leftrightarrow 1$ 得对偶命题 |
| 布尔函数   | $n$ 变量共有 $2^{2^n}$ 个布尔函数                                   |
| 标准形    | SOP（极小项之和）与 POS（极大项之积）唯一表示                                 |
| 卡诺图    | $\leq 4$ 变量可视化化简；无关项可灵活利用                                  |
| Q-M 算法 | 系统化表格化简，适合编程实现                                             |
| 逻辑门    | NAND 门单独功能完备                                               |
| 加法器    | 半加器 $S = A \oplus B$ ；全加器可级联                               |





## 递推关系与生成函数

如何研究序列中后项与前项之间的关系？递推关系与生成函数是离散数学中两个互为表里、相辅相成的重要工具。

## § 10.1

## 递推关系的基本概念

## 递推关系

设  $\{a_n\}_{n=0}^{\infty}$  是一个序列。若对某个  $k \geq 1$  和所有  $n \geq k$ ,  $a_n$  都可以用  $a_{n-1}, a_{n-2}, \dots, a_{n-k}$  表示, 即:

$$a_n = f(a_{n-1}, a_{n-2}, \dots, a_{n-k}, n)$$

则称该等式为一个  $k$  阶递推关系。确定唯一序列所必需的  $k$  个初始值  $a_0, a_1, \dots, a_{k-1}$  称为初始条件。

【例题 10.1】 •  $a_n = a_{n-1} + 3$  是 1 阶递推

•  $a_n = a_{n-1} + a_{n-2}$  是 2 阶递推 (Fibonacci 递推)

•  $a_n = na_{n-1}$  是 1 阶递推 (阶乘的递推定义)

若  $a_n = a_{n-1} + a_{n-2}$ , 初始条件为  $a_0 = 0, a_1 = 1$ , 则得到 Fibonacci 序列:  $0, 1, 1, 2, 3, 5, 8, 13, 21, \dots$ 。  
若初始条件为  $a_0 = 2, a_1 = 1$ , 则得到 Lucas 序列:  $2, 1, 3, 4, 7, 11, 18, 29, \dots$ 。

## § 10.2

## 线性递推关系的求解

## 10.2.1 一阶线性递推

## 一阶线性递推的通解

对  $a_n = ca_{n-1} + f(n)$ , 通解为:

$$a_n = c^n a_0 + \sum_{i=1}^n c^{n-i} f(i)$$

特例——齐次情形 ( $f(n) = 0$ ):  $a_n = a_0 \cdot c^n$ 。

【例题 10.2】  $a_n = 2a_{n-1} + 1, a_0 = 1$ :

$$a_n = 2^n \cdot 1 + \sum_{i=1}^n 2^{n-i} \cdot 1 = 2^n + (2^n - 1) = 2^{n+1} - 1$$

## 10.2.2 齐次线性递推的特征方程法

## 二阶齐次递推

考虑  $a_n = c_1 a_{n-1} + c_2 a_{n-2}$ 。

特征方程法：

1. 设  $a_n = r^n$  ( $r \neq 0$ )，代入得  $r^n = c_1 r^{n-1} + c_2 r^{n-2}$

2. 除以  $r^{n-2}$  得特征方程： $r^2 - c_1 r - c_2 = 0$

3. 解特征方程得两根  $r_1, r_2$

## 两个不等实根

若  $r_1 \neq r_2$ ，通解为：

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n$$

其中  $\alpha_1, \alpha_2$  由初始条件确定。

## 重根情形

若  $r_1 = r_2 = r$ ，通解为：

$$a_n = (\alpha_1 + \alpha_2 n) r^n$$

## 共轭复根情形

若  $r = \rho(\cos \theta \pm i \sin \theta)$ ，通解为：

$$a_n = \rho^n (\alpha_1 \cos n\theta + \alpha_2 \sin n\theta)$$

 $k$  阶齐次线性递推 $k$  阶齐次递推的通解

对递推  $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$ ，特征方程为：

$$r^k - c_1 r^{k-1} - c_2 r^{k-2} - \cdots - c_k = 0$$

设  $r_1, \dots, r_t$  为不同特征根，重数分别为  $m_1, \dots, m_t$  ( $\sum m_i = k$ )，则通解为：

$$a_n = \sum_{i=1}^t \left( \sum_{j=0}^{m_i-1} \alpha_{ij} n^j \right) r_i^n$$

重数为  $m$  的根  $r$  贡献  $m$  个线性无关解： $r^n, nr^n, n^2 r^n, \dots, n^{m-1} r^n$ 。

## 10.2.3 非齐次线性递推

## 非齐次递推的通解

$a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k} + f(n)$  的通解为:

$$a_n = a_n^{(h)} + a_n^{(p)}$$

其中  $a_n^{(h)}$  是对应齐次方程的通解,  $a_n^{(p)}$  是非齐次方程的一个特解。

待定系数法——根据  $f(n)$  的形式猜测特解形式:

| $f(n)$ 的形式    | 特解 $a_n^{(p)}$ 的尝试形式 | 条件             |
|---------------|----------------------|----------------|
| 常数 $C$        | $A$ (常数)             | 1 不是特征根        |
| $Cn$          | $An + B$             | 1 不是特征根        |
| $C \cdot s^n$ | $A \cdot s^n$        | $s$ 不是特征根      |
| $C \cdot s^n$ | $An^m s^n$           | $s$ 是 $m$ 重特征根 |
| 多项式 $P_t(n)$  | $Q_t(n)$ (同次多项式)     | 1 不是特征根        |

【例题 10.3】  $a_n = 3a_{n-1} + 2^n$ ,  $a_0 = 1$ :

齐次解  $a_n^{(h)} = A \cdot 3^n$ 。设特解  $a_n^{(p)} = B \cdot 2^n$ , 代入得  $B = -2$ 。

$$a_n = A \cdot 3^n - 2 \cdot 2^n$$

由  $a_0 = 1$ :  $A - 2 = 1 \Rightarrow A = 3$ , 故  $a_n = 3^{n+1} - 2^{n+1}$ 。

## 10.2.4 Fibonacci 数列的完整求解

## Fibonacci 数列

$$F_n = F_{n-1} + F_{n-2}, \quad F_0 = 0, F_1 = 1$$

特征根法求解:

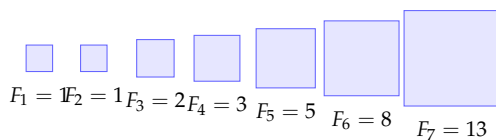
1. 特征方程:  $r^2 - r - 1 = 0$
2. 求根:  $r_1 = \frac{1+\sqrt{5}}{2} = \phi$ ,  $r_2 = \frac{1-\sqrt{5}}{2} = -\phi^{-1}$
3. 通解:  $F_n = \alpha_1 r_1^n + \alpha_2 r_2^n$
4. 由  $F_0 = 0, F_1 = 1$  解得  $\alpha_1 = \frac{1}{\sqrt{5}}$ ,  $\alpha_2 = -\frac{1}{\sqrt{5}}$

$$F_n = \frac{1}{\sqrt{5}} \left[ \left( \frac{1+\sqrt{5}}{2} \right)^n - \left( \frac{1-\sqrt{5}}{2} \right)^n \right]$$

这就是著名的 **Binet 公式**。有趣的性质:

$$\lim_{n \rightarrow \infty} \frac{F_{n+1}}{F_n} = \phi = \frac{1+\sqrt{5}}{2} \approx 1.618$$

$F_n$  是最接近  $\frac{\phi^n}{\sqrt{5}}$  的整数。



Fibonacci 正方形螺旋——边长依次为 Fibonacci 数

图 10.1: Fibonacci 数列的几何表示——正方形面积对应 Fibonacci 数，从中可画出黄金螺旋。

## § 10.3

## 分治递推与 Master 定理

## 10.3.1 分治算法的递推形式

分治算法的运行时间常具有如下形式：

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

其中  $a \geq 1$  是子问题个数， $b > 1$  是规模缩小因子， $f(n)$  是分解与合并的开销。

## 10.3.2 Master 定理

## Master Theorem

对  $T(n) = aT(n/b) + f(n)$ ，比较  $f(n)$  与  $n^{\log_b a}$  的增长率：

情形 1：若  $f(n) = O(n^{\log_b a - \epsilon})$ ，则  $T(n) = \Theta(n^{\log_b a})$

情形 2：若  $f(n) = \Theta(n^{\log_b a} \log^k n)$ ，则  $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

情形 3：若  $f(n) = \Omega(n^{\log_b a + \epsilon})$  且  $af(n/b) \leq cf(n)$  ( $c < 1$ )，则  $T(n) = \Theta(f(n))$

直观理解：

- 情形 1——叶子占主导（递归底层工作量最大）
- 情形 2——各层工作量相当
- 情形 3——根占主导（顶层合并工作量最大）

【例题 10.4】二分查找  $T(n) = T(n/2) + O(1)$ ： $a = 1, b = 2, \log_b a = 0$ ， $f(n) = \Theta(n^0) \Rightarrow$  情形 2 ( $k = 0$ )， $T(n) = \Theta(\log n)$ 。

归并排序  $T(n) = 2T(n/2) + O(n)$ ： $a = 2, b = 2, \log_b a = 1$ ， $f(n) = \Theta(n^1) \Rightarrow$  情形 2 ( $k = 0$ )， $T(n) = \Theta(n \log n)$ 。

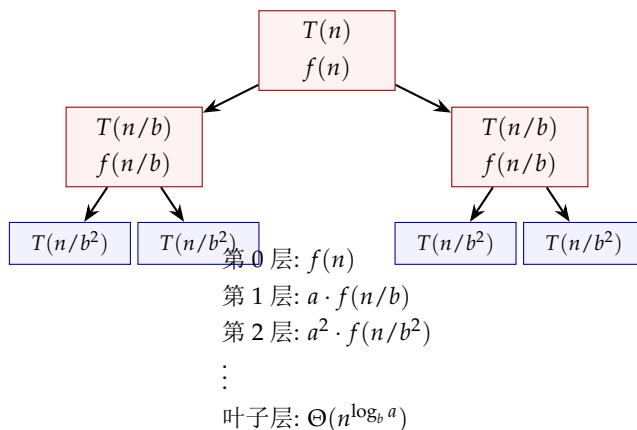


图 10.2: Master 定理的分治递归树——每层节点表示一个子问题，标注了该层的计算开销。

## § 10.4 生成函数（母函数）

### 10.4.1 普通生成函数的定义

#### 普通生成函数

给定序列  $\{a_n\}_{n=0}^{\infty}$ ，其普通生成函数定义为形式幂级数：

$$G(x) = \sum_{n=0}^{\infty} a_n x^n = a_0 + a_1 x + a_2 x^2 + \cdots$$

不关心收敛性，只把  $x$  当作占位符。记  $[x^n]G(x)$  表示提取  $x^n$  的系数。

### 10.4.2 常见序列的生成函数

| 序列 $\{a_n\}$             | 生成函数 $G(x)$                                | 闭合形式                |
|--------------------------|--------------------------------------------|---------------------|
| $a_n = 1$                | $\sum_{n=0}^{\infty} x^n$                  | $\frac{1}{1-x}$     |
| $a_n = c^n$              | $\sum_{n=0}^{\infty} c^n x^n$              | $\frac{1}{1-cx}$    |
| $a_n = n$                | $\sum_{n=0}^{\infty} n x^n$                | $\frac{x}{(1-x)^2}$ |
| $a_n = \binom{k}{n}$     | $\sum_{n=0}^k \binom{k}{n} x^n$            | $(1+x)^k$           |
| $a_n = \binom{n+k-1}{n}$ | $\sum_{n=0}^{\infty} \binom{n+k-1}{n} x^n$ | $\frac{1}{(1-x)^k}$ |

### 10.4.3 生成函数的运算

设  $F(x) = \sum f_n x^n$ ， $G(x) = \sum g_n x^n$ ：

| 运算         | 公式                                                               |
|------------|------------------------------------------------------------------|
| 加法         | $F(x) + G(x) = \sum (f_n + g_n) x^n$                             |
| 数乘         | $c \cdot F(x) = \sum (c f_n) x^n$                                |
| 乘法（卷积）     | $F(x)G(x) = \sum_{n=0}^{\infty} (\sum_{k=0}^n f_k g_{n-k}) x^n$  |
| 移位（乘 $x$ ） | $x F(x) = \sum_{n=1}^{\infty} f_{n-1} x^n$                       |
| 导数         | $F'(x) = \sum_{n=1}^{\infty} n f_n x^{n-1}$                      |
| 积分         | $\int_0^x F(t) dt = \sum_{n=0}^{\infty} \frac{f_n}{n+1} x^{n+1}$ |

### 10.4.4 用生成函数解 Fibonacci 递推

设  $G(x) = \sum_{n=0}^{\infty} F_n x^n$ 。将递推关系  $F_n = F_{n-1} + F_{n-2}$  乘以  $x^n$  并对  $n \geq 2$  求和：

$$\sum_{n=2}^{\infty} F_n x^n = \sum_{n=2}^{\infty} F_{n-1} x^n + \sum_{n=2}^{\infty} F_{n-2} x^n$$

用  $G(x)$  表达：

$$(G(x) - x) = xG(x) + x^2 G(x) \Rightarrow G(x) = \frac{x}{1-x-x^2}$$

分解为部分分式。设  $\phi = \frac{1+\sqrt{5}}{2}$ ,  $\hat{\phi} = \frac{1-\sqrt{5}}{2}$ :

$$G(x) = \frac{1}{\sqrt{5}} \left( \frac{1}{1-\phi x} - \frac{1}{1-\hat{\phi}x} \right) = \frac{1}{\sqrt{5}} \sum_{n=0}^{\infty} (\phi^n - \hat{\phi}^n) x^n$$

提取系数:  $F_n = [x^n]G(x) = \frac{1}{\sqrt{5}}(\phi^n - \hat{\phi}^n)$ ——正是 Binet 公式。

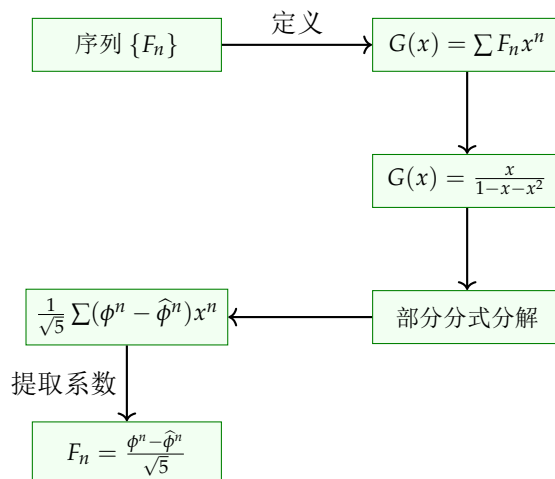


图 10.3: 生成函数法求解 Fibonacci 递推的流程图——生成函数将递推问题转化为代数问题。

### 10.4.5 指数生成函数

#### 指数生成函数

序列  $\{a_n\}$  的指数生成函数定义为:

$$E(x) = \sum_{n=0}^{\infty} a_n \frac{x^n}{n!}$$

当序列涉及排列、有标号结构时使用 EGF。

**【例题 10.5】**  $a_n = 1$  的 EGF 是  $e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$ ;  $a_n = 2^n$  的 EGF 是  $e^{2x}$ 。

EGF 的乘法 (二项卷积):

$$F(x) \cdot G(x) = \sum_{n=0}^{\infty} \left( \sum_{k=0}^n \binom{n}{k} f_k g_{n-k} \right) \frac{x^n}{n!}$$

## § 10.5

### 生成函数与组合计数

#### 10.5.1 多重集的组合数

从  $k$  种无穷供应的物品中选  $n$  个的组合数。生成函数方法:

$$G(x) = \left( \frac{1}{1-x} \right)^k = \sum_{n=0}^{\infty} \binom{n+k-1}{n} x^n$$

答案为  $\binom{n+k-1}{n} = \binom{n+k-1}{k-1}$ 。

### 10.5.2 整数拆分的生成函数

#### 整数拆分

将正整数  $n$  表示为若干正整数之和的方式数记为  $p(n)$ 。如  $p(4) = 5$ : 4,  $3 + 1$ ,  $2 + 2$ ,  $2 + 1 + 1$ ,  $1 + 1 + 1 + 1$ 。

无限制拆分的生成函数:

$$P(x) = \prod_{k=1}^{\infty} \frac{1}{1-x^k}$$

欧拉五边形数定理给出了  $p(n)$  的递推公式:

$$p(n) = p(n-1) + p(n-2) - p(n-5) - p(n-7) + p(n-12) + p(n-15) - \dots$$



## § 10.6

### 编程实现

#### Fibonacci: 递推/矩阵快速幂/Binet 公式

```

1 import math
2 from functools import lru_cache
3
4 @lru_cache(maxsize=None)
5 def fib_memo(n: int) -> int:
6     if n <= 1:
7         return n
8     return fib_memo(n - 1) + fib_memo(n - 2)
9
10 def fib_iter(n: int) -> int:
11     if n <= 1:
12         return n
13     a, b = 0, 1
14     for _ in range(2, n + 1):
15         a, b = b, a + b
16     return b
17
18 def fib_matrix(n: int) -> int:
19     """矩阵快速幂  $O(\log n)$ """
20     if n <= 1:
21         return n
22     def mat_mul(a, b):
23         return [[a[0][0]*b[0][0] + a[0][1]*b[1][0],
24                 a[0][0]*b[0][1] + a[0][1]*b[1][1],
25                 a[1][0]*b[0][0] + a[1][1]*b[1][0],
26                 a[1][0]*b[0][1] + a[1][1]*b[1][1]]
27     def mat_pow(mat, exp):
28         res = [[1, 0], [0, 1]]
29         while exp:
30             if exp & 1: res = mat_mul(res, mat)
31             mat = mat_mul(mat, mat)
32             exp >>= 1
33         return res
34     return mat_pow([[1, 1], [1, 0]], n - 1)[0][0]
35
36 SQRT5 = math.sqrt(5)
37 PHI = (1 + SQRT5) / 2
38 def fib_binet(n: int) -> int:
39     return round(PHI ** n / SQRT5)

```

## Fibonacci 矩阵快速幂 C++ 实现

```
1 #include <iostream>
2 using namespace std;
3 using ll = long long;
4
5 struct Matrix {
6     ll m[2][2] = {{0,0},{0,0}};
7 };
8
9 Matrix mul(Matrix a, Matrix b) {
10     Matrix c;
11     for (int i = 0; i < 2; i++)
12         for (int j = 0; j < 2; j++)
13             for (int k = 0; k < 2; k++)
14                 c.m[i][j] += a.m[i][k] * b.m[k][j];
15     return c;
16 }
17
18 Matrix power(Matrix a, ll n) {
19     Matrix res;
20     res.m[0][0] = res.m[1][1] = 1;
21     while (n) {
22         if (n & 1) res = mul(res, a);
23         a = mul(a, a);
24         n >>= 1;
25     }
26     return res;
27 }
28
29 ll fib_matrix(int n) {
30     if (n <= 1) return n;
31     Matrix base;
32     base.m[0][0] = base.m[0][1] = base.m[1][0] = 1;
33     return power(base, n - 1).m[0][0];
34 }
35
36 int main() {
37     cout << fib_matrix(50) << endl; // 12586269025
38     return 0;
39 }
```

## 线性递推的求解程序

```

1 def solve_linear_recurrence(coeffs, initials, n_terms):
2     """
3     coeffs: [c1, c2, ..., ck] -- a_n = c1*a_{n-1} + ... + ck*a_{n-k}
4     initials: [a_0, a_1, ..., a_{k-1}]
5     """
6     k = len(coeffs)
7     n = len(initials)
8     seq = initials[:]
9     for i in range(k, n_terms):
10         next_val = sum(coeffs[j] * seq[-1-j] for j in range(k))
11         seq.append(next_val)
12     return seq
13
14 fib_seq = solve_linear_recurrence([1, 1], [0, 1], 15)
15 print(fib_seq) # [0,1,1,2,3,5,8,13,21,34,55,89,144,233,377]

```

## 本章小结

| 知识模块         | 核心内容                                             |
|--------------|--------------------------------------------------|
| 递推关系         | $k$ 阶递推的定义，初始条件的唯一性                              |
| 齐次线性递推       | 特征方程 $\rightarrow$ 特征根 $\rightarrow$ 通解公式（含重根情形） |
| 非齐次递推        | 通解 = 齐次通解 + 特解（待定系数法）                            |
| Master 定理    | 比较 $f(n)$ 与 $n^{\log_b a}$ ，分三种情形                |
| 普通生成函数       | 序列 $\rightarrow G(x) = \sum a_n x^n$ ；四则运算与系数提取  |
| 指数生成函数       | $E(x) = \sum a_n x^n / n!$ ；有标号组合计数              |
| Fibonacci 求解 | 特征方程法 $\rightarrow$ Binet 公式；生成函数法殊途同归           |
| 组合计数         | 多重集组合数、整数拆分的生成函数表示                               |

整数有哪些“深处”的结构？为什么古代数学家研究的问题会成为现代密码学的基石？数论研究整数的性质，是RSA加密、椭圆曲线密码等现代信息技术的理论基础。

### §11.1 整除理论

#### 11.1.1 整除的定义与基本性质

##### 整除

设  $a, b \in \mathbb{Z}$ ,  $a \neq 0$ 。若存在整数  $c$  使  $b = ac$ ，则称  $a$  整除  $b$ ，记作  $a \mid b$ 。否则记作  $a \nmid b$ 。此时  $a$  称为  $b$  的因数， $b$  称为  $a$  的倍数。

基本性质：

| 性质  | 表达式                                                                             |
|-----|---------------------------------------------------------------------------------|
| 自反性 | $a \mid a$                                                                      |
| 传递性 | 若 $a \mid b$ 且 $b \mid c$ ，则 $a \mid c$                                         |
| 线性性 | 若 $a \mid b$ 且 $a \mid c$ ，则 $a \mid (mb + nc)$ , $\forall m, n \in \mathbb{Z}$ |
| 反身性 | 若 $a \mid b$ 且 $b \mid a$ ，则 $a = \pm b$                                        |

线性性的证明. 由  $b = ak_1$ ,  $c = ak_2$  得:  $mb + nc = m(ak_1) + n(ak_2) = a(mk_1 + nk_2)$ , 故  $a \mid (mb + nc)$ 。□

#### 11.1.2 带余除法

##### 带余除法

设  $a \in \mathbb{Z}$ ,  $b \in \mathbb{Z}^+$ ，则存在唯一的整数  $q$  和  $r$  满足：

$$a = bq + r, \quad 0 \leq r < b$$

其中  $q$  称为商， $r$  称为余数。余数常记作  $a \bmod b = r$ 。

证明. 考虑集合  $S = \{a - bk \mid k \in \mathbb{Z}, a - bk \geq 0\}$ 。  $S$  非空（取  $k$  足够小），由良序原理  $S$  有最小元  $r$ 。令  $q$  满足  $r = a - bq$ ，则  $r \geq 0$ 。若  $r \geq b$ ，则  $a - b(q+1) = r - b \geq 0$  是  $S$  中更小的非负元，矛盾。  
唯一性：若有两组  $(q_1, r_1)$  和  $(q_2, r_2)$ ，则  $b(q_1 - q_2) = r_2 - r_1$ 。由于  $|r_2 - r_1| < b$ ，只能  $r_1 = r_2$ ，从而  $q_1 = q_2$ 。□

## 11.1.3 最大公因数与辗转相除法

## 最大公因数

设  $a, b$  是不全为 0 的整数。集合  $\{d \mid d \mid a \text{ 且 } d \mid b\}$  中的最大元素称为  $a$  和  $b$  的最大公因数，记作  $\gcd(a, b)$ 。

性质： $\gcd(a, b) = \gcd(a \bmod b, b)$ 。

## 辗转相除法

输入:  $a, b$  (不全为 0)

输出:  $\gcd(a, b)$

```
while b != 0:
    r = a mod b
    a = b
    b = r
return |a|
```

## 时间复杂度

辗转相除法的时间复杂度为  $O(\log \min(a, b))$ 。

【例题 11.1】  $\gcd(252, 105)$ :

$$252 = 105 \times 2 + 42, \quad 105 = 42 \times 2 + 21, \quad 42 = 21 \times 2 + 0$$

故  $\gcd(252, 105) = 21$ 。

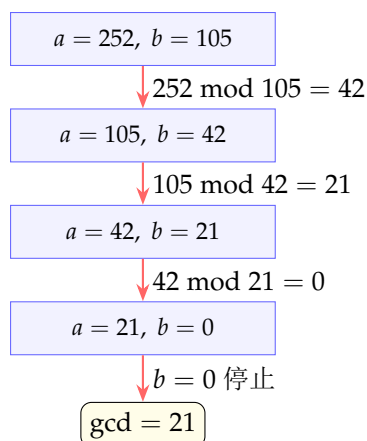


图 11.1: 辗转相除法流程图——每次迭代用余数替换较大的数，直到余数为 0。

## 11.1.4 最小公倍数

## 最小公倍数

$a$  和  $b$  的正公倍数中最小的一个称为最小公倍数，记作  $\text{lcm}(a, b)$ 。

$$|ab| = \gcd(a, b) \cdot \text{lcm}(a, b)。$$

### 11.1.5 扩展欧几里得算法与 Bézout 定理

#### Bézout 定理

设  $a, b$  为不全为 0 的整数，则存在整数  $x, y$  满足：

$$ax + by = \gcd(a, b)$$

且  $\gcd(a, b)$  是能被  $a$  和  $b$  的整系数线性组合表示的最小正整数。

扩展欧几里得算法在辗转相除的每一步保持关系  $r_i = ax_i + by_i$ ：

$$(x_i, y_i) = (x_{i-2} - q_i x_{i-1}, y_{i-2} - q_i y_{i-1})$$

从初始值  $(x_0, y_0) = (1, 0)$ ,  $(x_1, y_1) = (0, 1)$  开始。

**【例题 11.2】** 求  $252x + 105y = \gcd(252, 105) = 21$ ：

| $i$ | $q_i$ | $x_i$                 | $y_i$                   | $r_i$ |
|-----|-------|-----------------------|-------------------------|-------|
| 0   | —     | 1                     | 0                       | 252   |
| 1   | —     | 0                     | 1                       | 105   |
| 2   | 2     | $1 - 2 \times 0 = 1$  | $0 - 2 \times 1 = -2$   | 42    |
| 3   | 2     | $0 - 2 \times 1 = -2$ | $1 - 2 \times (-2) = 5$ | 21    |

得  $(x, y) = (-2, 5)$ ，验证： $252 \times (-2) + 105 \times 5 = -504 + 525 = 21$ 。

## § 11.2 素数

### 11.2.1 素数与算术基本定理

#### 素数

大于 1 的整数  $p$  若只有 1 和  $p$  两个正因数，称为素数；否则称为合数。1 既不是素数也不是合数。

#### 算术基本定理

每个大于 1 的整数  $n$  都可以唯一地（不计顺序）表示为素数的乘积：

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$$

其中  $p_1 < p_2 < \cdots < p_k$  为素数， $e_i \geq 1$ 。

## 11.2.2 素数有无穷多个

## 欧几里得定理

素数有无穷多个。

**证明.** 假设只有有限个素数  $p_1, p_2, \dots, p_k$ 。考虑  $N = p_1 p_2 \cdots p_k + 1$ 。 $N$  不能被任何  $p_i$  整除（余 1），故  $N$  要么本身是新的素数，要么有不在列表中的素因子。矛盾。  $\square$

## 11.2.3 素数判定与筛法

## 埃拉托斯特尼筛法

1. 列出 2 到  $n$  的所有数
2. 从  $p = 2$  开始，标记所有  $p$  的倍数（除  $p$  本身外）为合数
3. 找到下一个未标记的数作为新的  $p$ ，重复步骤 2
4. 直到  $p > \sqrt{n}$

复杂度： $O(n \log \log n)$ 。



图 11.2: 埃氏筛法可视化——素数被保留（红色），合数被逐一筛掉（灰色）。

欧拉线性筛：每个合数仅被其最小素因子筛掉一次，复杂度  $O(n)$ 。

**Miller-Rabin 概率素性测试：**基于费马小定理及其推广。单次测试错误概率  $\leq 1/4$ ，执行  $k$  次独立测试后错误概率  $\leq (1/4)^k$ 。

## 11.2.4 素数定理

## 素数定理

设  $\pi(x)$  为不超过  $x$  的素数个数，则：

$$\pi(x) \sim \frac{x}{\ln x}, \quad (x \rightarrow \infty)$$

即  $\lim_{x \rightarrow \infty} \frac{\pi(x)}{x/\ln x} = 1$ 。 $n$  附近的素数密度约为  $\frac{1}{\ln n}$ 。

## § 11.3 同余与模算术

### 11.3.1 同余的定义

#### 同余

设  $m \in \mathbb{Z}^+$ 。若  $m \mid (a - b)$ ，则称  $a$  与  $b$  模  $m$  同余，记作：

$$a \equiv b \pmod{m}$$

即  $a$  和  $b$  除以  $m$  的余数相同。

### 11.3.2 模算术基本性质

设  $a \equiv b \pmod{m}$  且  $c \equiv d \pmod{m}$ ，则：

| 运算  | 结果                                                |
|-----|---------------------------------------------------|
| 加法  | $a + c \equiv b + d \pmod{m}$                     |
| 减法  | $a - c \equiv b - d \pmod{m}$                     |
| 乘法  | $ac \equiv bd \pmod{m}$                           |
| 幂   | $a^k \equiv b^k \pmod{m} \ (k \geq 0)$            |
| 多项式 | $f(a) \equiv f(b) \pmod{m} \ (f \text{ 为整系数多项式})$ |

### 11.3.3 线性同余方程

#### 线性同余方程的解

方程  $ax \equiv b \pmod{m}$  有解当且仅当  $g = \gcd(a, m) \mid b$ 。此时恰好有  $g$  个模  $m$  不同余的解。

解法：

1. 用扩展欧几里得算法求  $ax_0 + my_0 = g$
2. 特解：  $x_0 \cdot (b/g) \pmod{m}$
3. 通解：  $x_k = x_0 + k \cdot (m/g), k = 0, 1, \dots, g-1$

**【例题 11.3】**  $14x \equiv 8 \pmod{30}$ ：

$\gcd(14, 30) = 2 \mid 8$ ，有解。约简为  $7x \equiv 4 \pmod{15}$ 。扩展欧几里得得  $7 \cdot 13 \equiv 1 \pmod{15}$ 。 $x \equiv 13 \times 4 = 52 \equiv 7 \pmod{15}$ 。模 30 下有两个解：  $x \equiv 7, 22 \pmod{30}$ 。

### 11.3.4 模逆元与费马小定理

#### 模逆元

若  $ab \equiv 1 \pmod{m}$ ，则  $b$  称为  $a$  的模  $m$  逆元，记作  $a^{-1} \pmod{m}$ 。



## 逆元存在条件

$a$  在模  $m$  下有逆元当且仅当  $\gcd(a, m) = 1$ 。逆元唯一（模  $m$  意义下）。用扩展欧几里得算法求  $ax + my = 1$ ，则  $x \equiv a^{-1} \pmod{m}$ 。

## 费马小定理

若  $p$  是素数且  $\gcd(a, p) = 1$ ，则：

$$a^{p-1} \equiv 1 \pmod{p}$$

等价形式：对任意整数  $a$ ，有  $a^p \equiv a \pmod{p}$ 。

应用——求模逆元：在模素数  $p$  下， $a^{-1} \equiv a^{p-2} \pmod{p}$ （因为  $a \cdot a^{p-2} = a^{p-1} \equiv 1$ ）。

## § 11.4 中国剩余定理

## 中国剩余定理

设  $m_1, m_2, \dots, m_k$  是两两互素的正整数，则同余方程组：

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \vdots \\ x \equiv a_k \pmod{m_k} \end{cases}$$

在模  $M = m_1 m_2 \cdots m_k$  下有唯一解。

**构造性证明。** 令  $M = \prod m_i$ ， $M_i = M/m_i$ 。由于  $\gcd(m_i, M_i) = 1$ ，存在  $t_i$  使  $M_i t_i \equiv 1 \pmod{m_i}$ 。  
构造解：

$$x = \sum_{i=1}^k a_i M_i t_i \pmod{M}$$

对任意  $j$ ，当  $i \neq j$  时  $m_j \mid M_i$ ，故该项  $\equiv 0 \pmod{m_j}$ ；当  $i = j$  时  $M_j t_j \equiv 1$ ，该项  $\equiv a_j$ 。因此  $x \equiv a_j \pmod{m_j}$ 。唯一性：若  $x, y$  都是解，则  $x \equiv y \pmod{m_i}$  对所有  $i$ ，由互素性得  $x \equiv y \pmod{M}$ 。□

**【例题 11.4】** 解  $x \equiv 2 \pmod{3}$ ， $x \equiv 3 \pmod{5}$ ， $x \equiv 2 \pmod{7}$ ： $M = 105$ 。 $M_1 = 35$ ： $35t_1 \equiv 1 \pmod{3} \Rightarrow t_1 = 2$ 。 $M_2 = 21$ ： $21t_2 \equiv 1 \pmod{5} \Rightarrow t_2 = 1$ 。 $M_3 = 15$ ： $15t_3 \equiv 1 \pmod{7} \Rightarrow t_3 = 1$ 。

$$x = 2 \cdot 35 \cdot 2 + 3 \cdot 21 \cdot 1 + 2 \cdot 15 \cdot 1 = 233 \equiv 23 \pmod{105}$$

验证： $23 \equiv 2 \pmod{3}$ ， $23 \equiv 3 \pmod{5}$ ， $23 \equiv 2 \pmod{7}$ 。

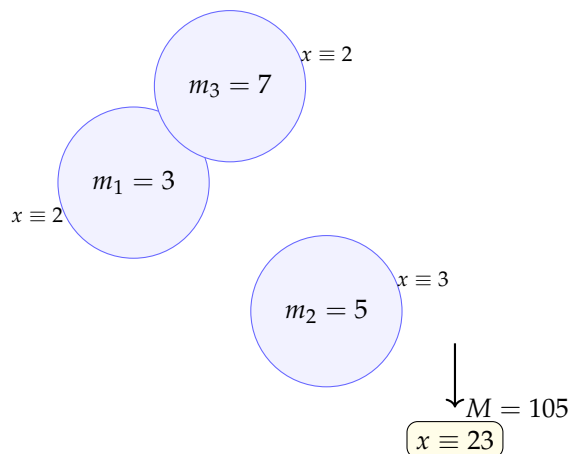


图 11.3: 中国剩余定理的几何理解——三个独立的模约束（环）在模 105 下相交于唯一解  $x = 23$ 。

## § 11.5 欧拉定理与 RSA 密码体制

### 11.5.1 欧拉函数与欧拉定理

#### 欧拉函数 $\varphi(n)$

$\varphi(n)$  表示 1 到  $n$  中与  $n$  互素的正整数个数：

$$\varphi(n) = |\{a \in [1, n] : \gcd(a, n) = 1\}|$$

#### 欧拉函数的计算

若  $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ （素因数分解），则：

$$\varphi(n) = n \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right) = \prod_{i=1}^k p_i^{e_i-1} (p_i - 1)$$

特例： $\varphi(p) = p - 1$ ,  $\varphi(pq) = (p - 1)(q - 1)$ ,  $\varphi(p^k) = p^k - p^{k-1}$ 。

#### 欧拉定理

若  $\gcd(a, n) = 1$ ，则：

$$a^{\varphi(n)} \equiv 1 \pmod{n}$$

当  $n$  为素数  $p$  时， $\varphi(p) = p - 1$ ，即得费马小定理。

### 11.5.2 RSA 密码体制

RSA 由 Rivest、Shamir、Adleman 于 1977 年提出，是第一个实用的公钥加密体制。其安全性基于大整数分解的困难性。

#### 密钥生成

1. 随机选择两个大素数  $p$  和  $q$ （各约 1024 位以上）

2. 计算  $n = pq$ ,  $\varphi(n) = (p-1)(q-1)$
3. 选择加密指数  $e$ : 满足  $1 < e < \varphi(n)$  且  $\gcd(e, \varphi(n)) = 1$
4. 计算解密指数  $d$ : 满足  $ed \equiv 1 \pmod{\varphi(n)}$
5. 公钥:  $(n, e)$ ; 私钥:  $(n, d)$  ( $p, q$  必须保密)

### 加密与解密

消息  $M$  需满足  $0 \leq M < n$ 。密文  $C = M^e \bmod n$ 。解密  $M = C^d \bmod n$ 。

### 正确性证明

需证  $(M^e)^d \equiv M \pmod{n}$ 。由  $ed \equiv 1 \pmod{\varphi(n)}$ , 存在  $k$  使  $ed = 1 + k\varphi(n)$ 。

若  $\gcd(M, n) = 1$  (绝大多数情况), 由欧拉定理:

$$M^{ed} = M^{1+k\varphi(n)} = M \cdot (M^{\varphi(n)})^k \equiv M \pmod{n}$$

若  $\gcd(M, n) \neq 1$ , 不妨设  $p \mid M$  且  $q \nmid M$ 。由费马小定理  $M^{q-1} \equiv 1 \pmod{q}$ , 于是  $M^{ed} \equiv M \pmod{q}$ 。又显然  $M^{ed} \equiv 0 \equiv M \pmod{p}$ 。由中国剩余定理得  $M^{ed} \equiv M \pmod{n}$ 。

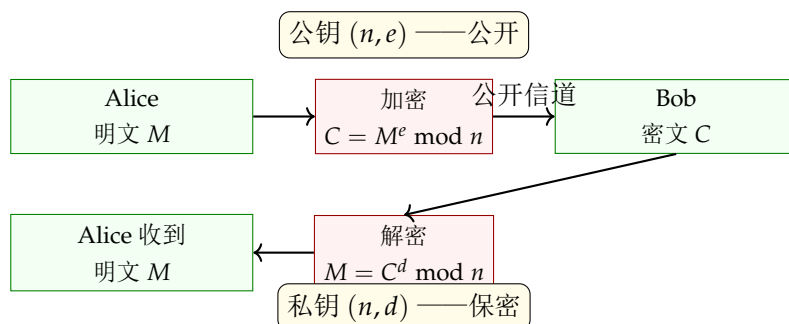


图 11.4: RSA 加密通信示意图——加密使用公钥 (任何人都可加密), 解密使用私钥 (仅接收方持有)。

### 11.5.3 RSA 的安全性

- 大整数分解: 若能分解  $n = pq$ , 则可算出  $\varphi(n)$  和  $d$
- 小指数攻击: 若  $e = 3$ , 当  $M^3 < n$  时计算普通立方根即可破解 (必须用填充方案如 OAEP)
- 共模攻击: 不同  $e$  共享同一个  $n$  非常危险
- 推荐参数:  $n \geq 2048$  位,  $e = 65537$

## § 11.6 二次剩余

### 二次剩余

设  $p$  是奇素数,  $a \in \mathbb{Z}$ 。若同余方程  $x^2 \equiv a \pmod{p}$  有解, 则称  $a$  是模  $p$  的二次剩余; 否则称为二次非剩余。

勒让德符号：

$$\left(\frac{a}{p}\right) = \begin{cases} 0, & p \mid a \\ 1, & a \text{ 是二次剩余且 } p \nmid a \\ -1, & a \text{ 是二次非剩余} \end{cases}$$

#### 欧拉判别法

$$\left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \pmod{p}.$$

### §11.7 编程实现

#### 扩展欧几里得算法——Python

```

1 def exgcd(a: int, b: int):
2     """返回 (g, x, y) 使 ax + by = g = gcd(a,b)"""
3     if b == 0:
4         return a, 1, 0
5     g, y, x = exgcd(b, a % b)
6     y -= (a // b) * x
7     return g, x, y
8
9 def mod_inv(a: int, m: int) -> int:
10    """求 a 在模 m 下的逆元"""
11    g, x, _ = exgcd(a, m)
12    if g != 1:
13        raise ValueError("无逆元")
14    return x % m
15
16 # 测试: 252x + 105y = 21
17 g, x, y = exgcd(252, 105)
18 print(f"gcd={g}, x={x}, y={y}")
19 print(f"验证: {252*x + 105*y}")

```

## 扩展欧几里得算法——C++

```
1 #include <iostream>
2 using namespace std;
3 using ll = long long;
4
5 ll exgcd(ll a, ll b, ll &x, ll &y) {
6     if (b == 0) {
7         x = 1; y = 0;
8         return a;
9     }
10    ll g = exgcd(b, a % b, y, x);
11    y -= (a / b) * x;
12    return g;
13 }
14
15 ll mod_inv(ll a, ll m) {
16     ll x, y;
17     ll g = exgcd(a, m, x, y);
18     if (g != 1) return -1;
19     return (x % m + m) % m;
20 }
21
22 int main() {
23     ll x, y;
24     ll g = exgcd(252, 105, x, y);
25     cout << "gcd=" << g << " x=" << x << " y=" << y << endl;
26     cout << "252*" << x << "+105*" << y << "=" << 252*x+105*y << endl;
27     return 0;
28 }
```

## 快速幂取模与 RSA 加解密

```

1 import random
2 from math import gcd
3
4 def mod_pow(a: int, b: int, m: int) -> int:
5     """快速幂取模  $O(\log b)$ """
6     res = 1 % m; a %= m
7     while b:
8         if b & 1: res = (res * a) % m
9         a = (a * a) % m; b >>= 1
10    return res
11
12 def miller_rabin(n: int, k: int = 20) -> bool:
13     """Miller-Rabin 概率素性测试"""
14     if n < 2: return False
15     if n in (2, 3): return True
16     if n % 2 == 0: return False
17     s, d = 0, n - 1
18     while d % 2 == 0:
19         s += 1; d //= 2
20     def check(a):
21         x = pow(a, d, n)
22         if x == 1 or x == n - 1: return True
23         for _ in range(s - 1):
24             x = (x * x) % n
25             if x == n - 1: return True
26         return False
27     for _ in range(k):
28         a = random.randrange(2, n - 1)
29         if not check(a): return False
30     return True
31
32 def generate_prime(bits: int) -> int:
33     while True:
34         n = random.getrandbits(bits) | (1 << (bits - 1)) | 1
35         if miller_rabin(n): return n
36
37 def generate_rsa_keys(bits: int = 512):
38     p = generate_prime(bits // 2)
39     q = generate_prime(bits // 2)
40     n = p * q
41     phi_n = (p - 1) * (q - 1)
42     e = 65537
43     d = mod_inv(e, phi_n)
44     return (n, e), (n, d)
45
46 def rsa_encrypt(m: int, pub_key: tuple) -> int:
47     return pow(m, pub_key[1], pub_key[0])
48
49 def rsa_decrypt(c: int, priv_key: tuple) -> int:
50     return pow(c, priv_key[1], priv_key[0])
51
52 pub, priv = generate_rsa_keys(256)
53 msg = 123456789
54 print(f"原文: {msg}")

```

## 中国剩余定理与素数筛法

```
1 def crt(remainders: list, moduli: list) -> int:
2     """x    remainders[i] (mod moduli[i]), 要求 moduli 两两互素"""
3     M = 1
4     for m in moduli:
5         M *= m
6     x = 0
7     for a, m in zip(remainders, moduli):
8         Mi = M // m
9         ti = mod_inv(Mi, m)
10        x = (x + a * Mi * ti) % M
11    return x
12
13 print(crt([2, 3, 2], [3, 5, 7])) # 输出 23
14
15 def sieve_euler(n: int) -> list:
16     """欧拉线性筛  $O(n)$ """
17     primes = []
18     min_pf = [0] * (n + 1)
19     for i in range(2, n + 1):
20         if min_pf[i] == 0:
21             min_pf[i] = i
22             primes.append(i)
23         for p in primes:
24             if p > min_pf[i] or p * i > n:
25                 break
26             min_pf[p * i] = p
27     return primes
28
29 print(sieve_euler(50))
```

## 本章小结

| 知识模块      | 核心内容                                                                  |
|-----------|-----------------------------------------------------------------------|
| 整除理论      | 整除定义、带余除法、gcd 与 lcm、辗转相除法 $O(\log \min(a, b))$                        |
| Bézout 定理 | $\exists x, y : ax + by = \gcd(a, b)$ ; 扩展欧几里得算法求解                    |
| 算术基本定理    | 每个大于 1 的整数可唯一分解为素数乘积                                                  |
| 素数判定      | 试除法 $O(\sqrt{n})$ 、埃氏筛 $O(n \log \log n)$ 、欧拉线性筛 $O(n)$ 、Miller-Rabin |
| 同余与模算术    | 线性同余方程 $ax \equiv b \pmod{m}$ , $\gcd(a, m) \mid b$ 时有解               |
| 模逆元       | $\gcd(a, m) = 1$ 时逆元存在; 费马小定理下 $a^{-1} \equiv a^{m-2} \pmod{m}$       |
| 中国剩余定理    | 处理多模同余方程组, 模两两互素时有唯一解                                                 |
| 欧拉定理      | $a^{\varphi(n)} \equiv 1 \pmod{n}$ ( $\gcd(a, n) = 1$ )               |
| RSA 密码    | 公钥 $(n, e)$ 加密 $C = M^e \bmod n$ , 私钥 $(n, d)$ 解密 $M = C^d \bmod n$   |
| 二次剩余      | 勒让德符号、欧拉判别法 $a^{(p-1)/2} \bmod p$                                     |





## 算法与计算理论初步

什么是算法？如何判断算法好坏？计算机“能”解决什么、“不能”解决什么？本章是理论计算机科学的入门，从算法分析到计算理论。

### § 12.1 算法的基本概念

#### 12.1.1 算法的定义与特性

##### 算法

算法是一组定义明确的有穷指令序列，用于解决特定问题。它必须满足以下五个特性：

1. **有限性**：必须在执行有限步骤后终止
2. **确定性**：每一步操作必须精确定义、无歧义
3. **可行性**：每一步都必须足够基本，在有限时间内可执行
4. **输入**：有零个或多个输入
5. **输出**：至少有一个输出（问题的解）

算法 vs 程序：程序是算法在具体编程语言中的实现。程序可以不满足有限性（如操作系统永远运行），但算法必须终止。

#### 12.1.2 算法的表示

伪代码是介于自然语言和编程语言之间的描述方式，强调算法思想而非具体语法。

##### 冒泡排序

```
算法：BubbleSort(A[1..n])
for i = 1 to n-1 do
  for j = 1 to n-i do
    if A[j] > A[j+1] then
      swap(A[j], A[j+1])
    end if
  end for
end for
```

流程图通过图形元素（开始/结束框、处理框、判断框、箭头）直观表达算法流程。

## § 12.2 算法复杂度分析

### 12.2.1 渐进记号

核心思想：当输入规模  $n$  趋于无穷大时，关心算法运行时间的增长阶，忽略常数因子和低阶项。

#### 大 $O$ 记号——上界

$f(n) = O(g(n))$  若存在正常数  $c$  和  $n_0$ ，使得对所有  $n \geq n_0$ ，有  $0 \leq f(n) \leq c \cdot g(n)$ 。

#### 大 $\Omega$ 记号——下界

$f(n) = \Omega(g(n))$  若存在正常数  $c$  和  $n_0$ ，使得对所有  $n \geq n_0$ ，有  $0 \leq c \cdot g(n) \leq f(n)$ 。

#### 大 $\Theta$ 记号——紧确界

$f(n) = \Theta(g(n))$  若  $f(n) = O(g(n))$  且  $f(n) = \Omega(g(n))$ 。

直观理解： $O$ ——不快于， $\Omega$ ——不慢于， $\Theta$ ——恰等于（同阶）。

**【例题 12.1】**  $3n^2 + 5n - 7 = O(n^2)$ （取  $c = 4, n_0 = 6$ ）， $3n^2 + 5n - 7 = \Omega(n^2)$ （取  $c = 1, n_0 = 1$ ），故  $3n^2 + 5n - 7 = \Theta(n^2)$ 。

### 12.2.2 常见复杂度排序

从快到慢：

| 复杂度           | 名称   | 常见算法           |
|---------------|------|----------------|
| $O(1)$        | 常数   | 哈希表查找、数组索引     |
| $O(\log n)$   | 对数   | 二分查找、平衡树操作     |
| $O(n)$        | 线性   | 线性搜索、遍历数组      |
| $O(n \log n)$ | 线性对数 | 归并排序、堆排序       |
| $O(n^2)$      | 平方   | 简单排序、简单 DP     |
| $O(n^3)$      | 立方   | Floyd-Warshall |
| $O(2^n)$      | 指数   | 穷举搜索、子集枚举      |
| $O(n!)$       | 阶乘   | 旅行商问题穷举        |

**空间复杂度：**描述算法执行所需的额外内存空间。如冒泡排序  $O(1)$ ，归并排序  $O(n)$ 。

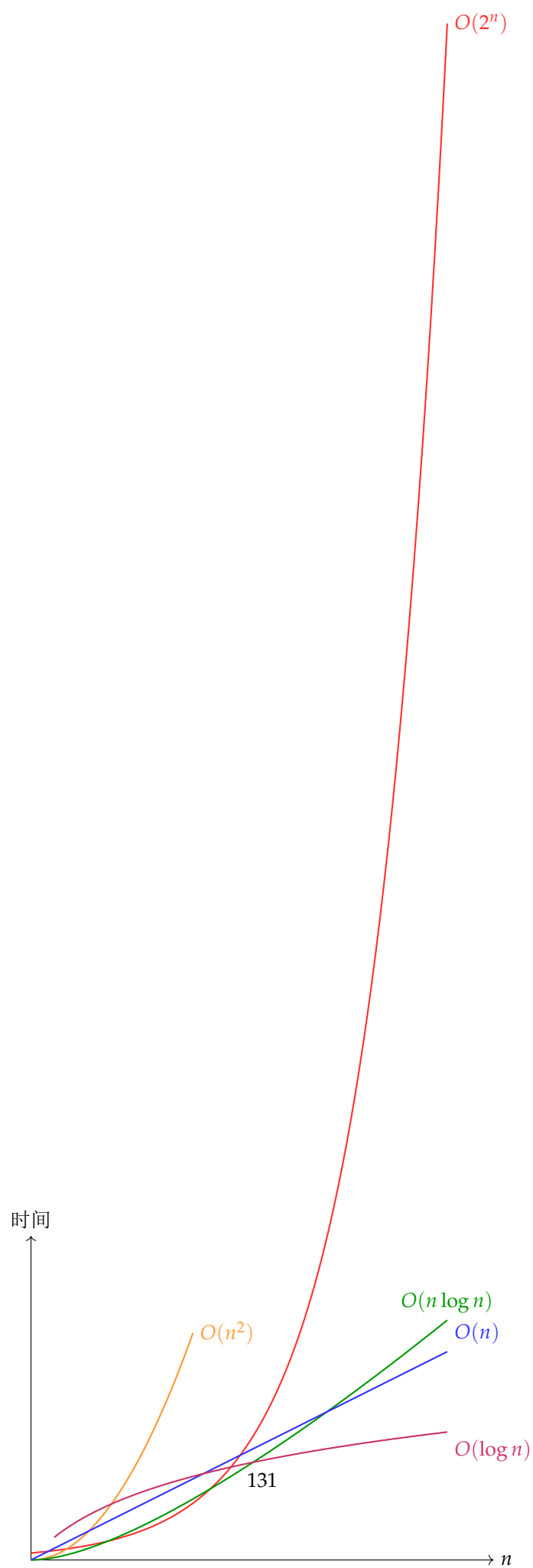


图 12.1: 渐进复杂度增长率对比图—— $O(2^n)$  以指数爆炸式增长， $O(\log n)$  极为平缓。

## § 12.3

### 经典算法设计策略

#### 12.3.1 贪心算法

核心思想：在每一步都做出当前看起来最优的选择（局部最优），希望通过局部最优的累积得到全局最优解。适用条件：贪心选择性质 + 最优子结构。

**【例题 12.2】(找零钱问题)** 人民币面值  $\{100, 50, 20, 10, 5, 1\}$ 。找零 187 元： $187 = 100 + 50 + 20 + 10 + 5 + 1 + 1$ ，共 7 张。策略：每次选不超过余额的最大面值。

注意：贪心不一定总是最优。若面值为  $\{1, 3, 4\}$ ，找零 6 元：贪心给  $6 = 4 + 1 + 1$ （3 张），最优是  $6 = 3 + 3$ （2 张）。

**【例题 12.3】(活动选择问题)** 给定  $n$  个活动，各有开始时间和结束时间。选择最大数量的不冲突活动。贪心策略：每次选结束时间最早的兼容活动——可获得最优解。

#### 12.3.2 分治算法

核心思想：将原问题分解为若干个规模较小的、互相独立的子问题，递归求解后合并。

三部曲：分解 → 递归解决 → 合并。

##### 【例题 12.4】(归并排序)

```
MergeSort( $A[l..r]$ ) :  
    if  $l \geq r$  : return  
     $mid = (l + r) / 2$   
    MergeSort( $A[l..mid]$ ) // 分解 + 解决左半  
    MergeSort( $A[mid + 1..r]$ ) // 分解 + 解决右半  
    Merge( $A[l..mid], A[mid + 1..r]$ ) // 合并
```

复杂度： $T(n) = 2T(n/2) + O(n) \Rightarrow T(n) = O(n \log n)$ 。

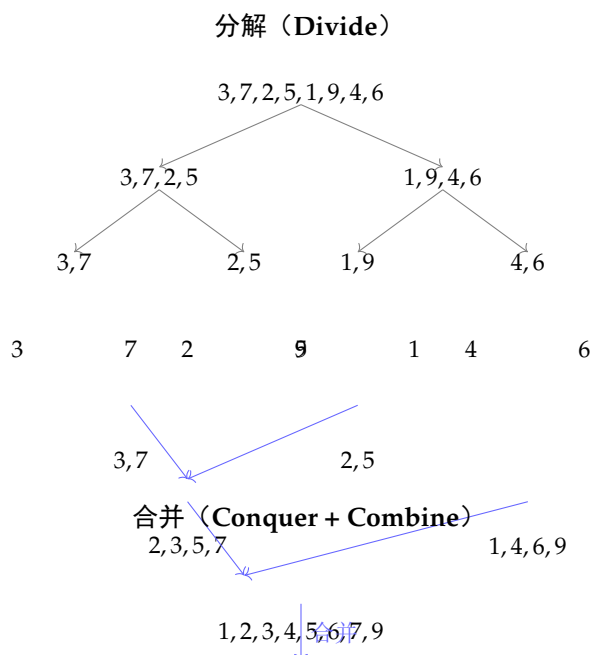


图 12.2: 归并排序的分治树——先自顶向下分解, 再自底向上合并有序子数组。

### 12.3.3 动态规划

核心思想: 将问题分解为重叠子问题, 自底向上求解, 用表格存储子问题的解以避免重复计算。

适用条件:

- **最优子结构:** 问题的最优解包含子问题的最优解
- **重叠子问题:** 递归过程中同一个子问题被多次求解

DP vs 分治: 分治的子问题是独立的; DP 的子问题是重叠的。

#### 0-1 背包问题

有  $n$  件物品, 第  $i$  件重量  $w_i$ , 价值  $v_i$ , 背包容量  $W$ 。每件物品要么全放入要么不放入。求最大总价值。

##### DP 解法

- **状态定义:**  $dp[i][j]$  = 考虑前  $i$  件物品, 容量为  $j$  时的最大价值

- **状态转移:**

$$dp[i][j] = \begin{cases} dp[i-1][j], & \text{若 } w_i > j \\ \max(dp[i-1][j], dp[i-1][j-w_i] + v_i), & \text{否则} \end{cases}$$

- **时间复杂度:**  $O(nW)$ ; **空间优化:** 可降至  $O(W)$

| 物品<br>backslash | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|-----------------|---|---|---|---|---|---|---|---|---|
| 0               | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1               | 0 | 0 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| 2               | 0 | 0 | 3 | 4 | 7 | 7 | 6 | 7 | 8 |

$w_1 = 2, v_1 = 3$   
 $w_2 = 3, v_2 = 4$   
 $W = 8$  时最大价值 = 10

图 12.3: 0-1 背包问题的 DP 表—— $dp[i][j]$  的值由上方格子和左上方格子转移而来。

### 最长公共子序列 (LCS)

求两字符串的最长公共子序列（不要求连续）。

- 定义:  $dp[i][j]$  =  $X[1..i]$  和  $Y[1..j]$  的 LCS 长度

- 转移:

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1, & X[i] = Y[j] \\ \max(dp[i-1][j], dp[i][j-1]), & X[i] \neq Y[j] \end{cases}$$

- 复杂度:  $O(mn)$

## § 12.4 计算理论入门

### 12.4.1 形式语言与 Chomsky 谱系

#### 基本概念

- 字母表  $\Sigma$ : 有限的非空符号集合
- 字符串: 由  $\Sigma$  中符号组成的有限序列; 空串记为  $\varepsilon$
- 语言:  $\Sigma^*$  的任意子集



图 12.4: Chomsky 谱系的金字塔——类型 3 在最内层（最小），类型 0 在最外层（最大），依次包含。

## 12.4.2 有限自动机

## 确定型有限自动机

DFA 是一个五元组  $M = (Q, \Sigma, \delta, q_0, F)$ :

- $Q$ : 有限状态集合
- $\Sigma$ : 输入字母表
- $\delta: Q \times \Sigma \rightarrow Q$ : 状态转移函数
- $q_0 \in Q$ : 起始状态
- $F \subseteq Q$ : 接受状态集合

从  $q_0$  开始, 依次读入输入字符, 每步根据当前状态和输入决定下一状态。读完整个字符串后若处于接受状态  $F$ , 则接受该字符串。

**【例题 12.5】(识别以 01 结尾的二进制串)**  $Q = \{q_0, q_1, q_2\}$  ( $q_0$  起始,  $q_2$  接受),  $\Sigma = \{0, 1\}$ 。

| 当前状态  | 输入 0  | 输入 1  |
|-------|-------|-------|
| $q_0$ | $q_1$ | $q_0$ |
| $q_1$ | $q_1$ | $q_2$ |
| $q_2$ | $q_1$ | $q_0$ |

运行举例: 输入 '1001':  $q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{0} q_1 \xrightarrow{1} q_2$  (接受)。

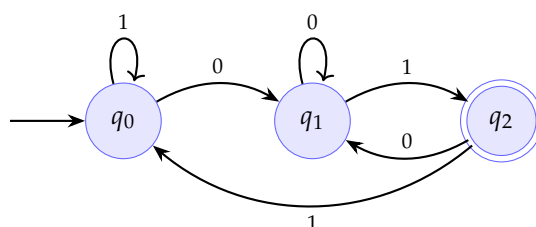


图 12.5: 识别以 01 结尾字符串的 DFA 状态转移图。 $q_2$  为接受状态 (双圈)。

## 12.4.3 非确定型有限自动机

NFA 与 DFA 的主要区别:

- $\delta$  可能返回多个后继状态 (或空集)
- 允许  $\epsilon$  转移 (不消耗输入的状态转移)

## NFA 与 DFA 的等价性

每个 NFA 都存在一个等价的 DFA (通过子集构造法)。但 DFA 的状态数最坏可能达到指数级  $2^{|Q|}$ 。

## 12.4.4 正则表达式

正则表达式是描述正则语言的代数表示。



| 表达式            | 含义                            |
|----------------|-------------------------------|
| $\emptyset$    | 空语言                           |
| $\varepsilon$  | 仅含空串的语言                       |
| $a$            | 单字符“a”                        |
| $R_1 R_2$      | 连接：任何 $R_1$ 匹配的串后接 $R_2$ 匹配的串 |
| $R_1 \mid R_2$ | 并：匹配 $R_1$ 或 $R_2$ 的串         |
| $R^*$          | Kleene 星闭包： $R$ 匹配的串的零次或多次连接  |

【例题 12.6】 •  $[01]^*$  ——所有二进制串

•  $0^*10^*$  ——恰好含一个 1 的二进制串

•  $(a|b)^*abb$  ——以  $abb$  结尾的、由  $a, b$  组成的串

#### Kleene 定理

正则表达式描述的语言与有限自动机接受的语言完全相同——即都是正则语言。

### 12.4.5 图灵机简介

#### 图灵机

图灵机是一个七元组  $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$ ，其中：

- 有一条无限长的纸带，划分为格子，每个格子存一个符号
- 有一个读写头，可在纸带上左右移动、读写符号
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$  为转移函数

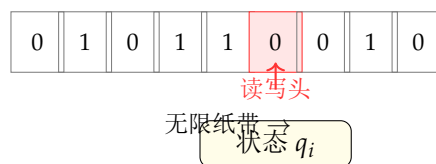


图 12.6: 图灵机示意图——无限长纸带上的每个格子存储一个符号，读写头根据当前状态进行读写和移动。

图灵机是计算理论中最核心的模型，它可以模拟任何已知的计算过程。

#### Church-Turing 论题

任何直觉上“可计算”的函数都可以被图灵机计算。这不是定理（无法证明），而是对计算能力的信仰声明——目前没有反例。

## § 12.5

## P 与 NP 问题简介

## 12.5.1 P 类与 NP 类

## P 类

存在多项式时间确定性算法求解的判定问题的集合。例如：排序、最短路径、最大流——这些都属于 P。

## NP 类

给定一个证书，可以在多项式时间验证其正确性的判定问题的集合。例如：合数判定、哈密顿回路。显然有  $P \subseteq NP$ 。

## 12.5.2 NP 完全性

## 多项式时间归约

问题  $A$  可多项式时间归约到问题  $B$  (记作  $A \leq_P B$ )，若存在多项式时间的变换  $f$ ，使得  $x \in A \iff f(x) \in B$ 。

## NP-难与 NP-完全

- **NP-难**：问题  $B$  称为 NP-难，若对任意  $A \in NP$ ，有  $A \leq_P B$
- **NP-完全**：问题  $B$  称为 NP-完全，若  $B \in NP$  且  $B$  是 NP-难的

经典 NPC 问题：SAT（布尔可满足性）、3SAT、顶点覆盖、团问题、哈密顿回路、0-1 背包判定问题、旅行商问题判定版。

12.5.3 Cook-Levin 定理与  $P=NP$  问题

## Cook-Levin 定理

SAT（布尔可满足性问题）是 NP-完全的。任何 NP 问题都可多项式时间归约到 SAT。

世纪难题： $P = NP$  还是  $P \neq NP$ ？这是 Clay 数学研究所的七大千年难题之一。

- 如果  $P = NP$ ：所有 NPC 问题都存在多项式时间算法；密码学（RSA 等）将崩溃；AI 中的许多困难问题可高效求解
- 如果  $P \neq NP$ ：目前已知的 NPC 问题真的没有多项式算法；基于某些困难假设的密码学依然安全

绝大多数计算机科学家相信  $P \neq NP$ ，但尚无严格证明。

## §12.6

### 编程实现

#### DFA 模拟器

```
1 class DFA:
2     def __init__(self, states, alphabet, transitions,
3                   start_state, accept_states):
4         self.states = set(states)
5         self.alphabet = set(alphabet)
6         self.transitions = transitions
7         self.start_state = start_state
8         self.accept_states = set(accept_states)
9
10    def run(self, input_string: str) -> bool:
11        current = self.start_state
12        for ch in input_string:
13            if (current, ch) not in self.transitions:
14                return False
15            current = self.transitions[(current, ch)]
16        return current in self.accept_states
17
18 # 识别以 "01" 结尾的二进制串
19 dfa = DFA(
20     states=['q0', 'q1', 'q2'], alphabet=['0', '1'],
21     transitions={
22         ('q0', '0'): 'q1', ('q0', '1'): 'q0',
23         ('q1', '0'): 'q1', ('q1', '1'): 'q2',
24         ('q2', '0'): 'q1', ('q2', '1'): 'q0',
25     },
26     start_state='q0', accept_states=['q2']
27 )
28 for s in ['', '01', '101', '1001', '110']:
29     print(f"  '{s}' -> {'接受' if dfa.run(s) else '拒绝'}")
```

## 0-1 背包 DP——Python

```

1 def knapsack_01(weights: list, values: list, capacity: int) -> int:
2     """0-1背包DP, 空间优化版  $O(nW)$  时间  $O(W)$  空间"""
3     n = len(weights)
4     dp = [0] * (capacity + 1)
5     for i in range(n):
6         w, v = weights[i], values[i]
7         for j in range(capacity, w - 1, -1):
8             dp[j] = max(dp[j], dp[j - w] + v)
9     return dp[capacity]
10
11 print(knapsack_01([2, 3, 4, 5], [3, 4, 5, 6], 8)) # 10

```

## 0-1 背包 DP——C++

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 int knapsack_01(const vector<int>& w, const vector<int>& v, int W) {
7     int n = w.size();
8     vector<int> dp(W + 1, 0);
9     for (int i = 0; i < n; i++) {
10         for (int j = W; j >= w[i]; j--) {
11             dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
12         }
13     }
14     return dp[W];
15 }
16
17 int main() {
18     vector<int> weights = {2, 3, 4, 5};
19     vector<int> values = {3, 4, 5, 6};
20     cout << knapsack_01(weights, values, 8) << endl; // 10
21     return 0;
22 }

```

## 最长公共子序列

```
1 def lcs_length(X: str, Y: str) -> int:
2     m, n = len(X), len(Y)
3     dp = [[0] * (n + 1) for _ in range(m + 1)]
4     for i in range(1, m + 1):
5         for j in range(1, n + 1):
6             if X[i-1] == Y[j-1]:
7                 dp[i][j] = dp[i-1][j-1] + 1
8             else:
9                 dp[i][j] = max(dp[i-1][j], dp[i][j-1])
10    return dp[m][n]
11
12 def get_lcs(X: str, Y: str) -> str:
13     m, n = len(X), len(Y)
14     dp = [[0]*(n+1) for _ in range(m+1)]
15     for i in range(1, m+1):
16         for j in range(1, n+1):
17             if X[i-1]==Y[j-1]:
18                 dp[i][j]=dp[i-1][j-1]+1
19             else:
20                 dp[i][j]=max(dp[i-1][j],dp[i][j-1])
21     chars = []
22     i, j = m, n
23     while i>0 and j>0:
24         if X[i-1]==Y[j-1]:
25             chars.append(X[i-1])
26             i-=1; j-=1
27         elif dp[i-1][j] > dp[i][j-1]:
28             i-=1
29         else:
30             j-=1
31     return ''.join(reversed(chars))
32
33 print(lcs_length("ABCBADAB", "BDCABA")) # 4
34 print(get_lcs("ABCBADAB", "BDCABA"))    # BCBA
```

## 本章小结

| 知识模块    | 核心内容                                                                                        |
|---------|---------------------------------------------------------------------------------------------|
| 算法定义    | 有限、确定、可行、有输入/输出的指令序列                                                                        |
| 复杂度分析   | $O$ (上界)、 $\Omega$ (下界)、 $\Theta$ (紧确界); $O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$ |
| 贪心算法    | 每步选局部最优; 找零钱、活动选择                                                                           |
| 分治算法    | 分解 $\rightarrow$ 递归解决 $\rightarrow$ 合并; 归并排序 $O(n \log n)$                                  |
| 动态规划    | 最优子结构 + 重叠子问题; 0-1 背包 $O(nW)$ 、LCS $O(mn)$                                                  |
| 形式语言    | Chomsky 谱系: 正则 (3) $\rightarrow$ CFL(2) $\rightarrow$ CSL(1) $\rightarrow$ RE(0)            |
| 有限自动机   | DFA (确定)、NFA (非确定); 子集构造法: NFA $\rightarrow$ DFA                                            |
| 图灵机     | 无限纸带 + 读写头 + 转移函数; Church-Turing 论题                                                         |
| P vs NP | P = 多项式可解; NP = 多项式可验证; NPC = NP 中最难的; Cook-Levin 定理                                        |